Internet-based programming-WEB DESIGN

information communication technology (Kisii University)

INTERNET BASED PROGRANMMING

DIPLOMA IN INFORMATION COMMUNICATION TECHNOLOGY

MODULE III

MORWABE N. ALEX BSC. (KISII)

CHAPTER 1

1. Introduction

1.1 TCP/IP Overview

Computer network protocols are formal rules of behavior that govern network communications. The *Transmission Control Protocol* (TCP) and *Internet Protocol* (IP) are just two of the data communication protocols encompassed by the Internet Protocol Suite. This protocol suite is usually referred to as TCP/IP partly because TCP and IP are two of the most important protocols of the collection.

TCP/IP includes a set of standards that specify how networked computers communicate and how data is routed through the interconnected computers. TCP/IP provides the application programmer with two primary services: *connectionless* packet delivery and *reliable* stream transport. TCP/IP has several distinguishing features that have led to its popularity, including:

- **Network Topology Independence** - TCP/IP is used on bus, ring, and star networks. It's used in local-area networks as well as wide-area networks.

- **Physical Network Hardware Independence** - TCP/IP can utilize Ethernet, token ring, or any number of physical hardware variations.

- **Open Protocol Standard** - The TCP/IP protocol suite standard is freely available for independent implementation on any computer hardware platform or operating system.

- **Universal Addressing Scheme** - Each computer on a TCP/IP network has an address that uniquely identifies it so that any TCP/IP enabled device can communicate with any other on the network. Each packet of data sent across a TCP/IP network has a header that contains the address of the destination computer as well as the address of the source computer.

- **Powerful Client-Server Framework** - TCP/IP is the framework for powerful and robust client-server applications that operate in local-area networks and wide-area networks.

2. Internet Protocol

2.1 Addresses for the Virtual Internet

The goal of internetworking is to provide a seamless communication system. To achieve this, the internet protocol software must hide the details of physical networks and offer facilities of a large virtual network. The virtual internet operates like a normal network, allowing computers to send and receive packets of information. The major difference between an internet and a physical network is that an internet is merely an abstraction created entirely by software. The designers are free to choose addresses, packet formats and delivery techniques independent of the details of the physical hardware.

To give the appearance of a single, uniform system, all host computers must use a uniform addressing scheme and each address must be unique. Physical network addresses do not suffice because an internet can include different technologies and each technology defines its address format. Addresses used by two different technologies may be incompatible because they are of different sizes or have different formats.

The protocol software defines an addressing scheme that is uniform and independent of the underlying physical address. To send a packet across an internet, the sender places the destination's protocol address

in the packet and passes the packet to the protocol software for delivery. The software uses the destination protocol address to forward the packet to the destination computer.

Uniform addressing helps create the illusion of a large seamless network because; it hides the details of the underlying physical network addresses. Applications communicate without knowing their hardware addresses.

2.2 The IP Addressing Scheme

In the TCP/IP, addressing is specified by the *Internet Protocol* (IP). The IP standard specifies that each computer be assigned a unique 32-bit number known as the host's IP address. Each packet sent across the internet has the IP address of both the sender and the destination computer. Thus, to transmit information across a TCP/IP internet, a computer must know the IP address of the remote computer to which information is being sent.

2.3 The IP Address Hierarchy

Conceptually, each IP address is divided into two parts: a prefix and a suffix. The address prefix identifies the physical network to which a computer is attached (each network is assigned a network number), while the suffix identifies a particular computer on that network. No two networks can be assigned the same network number and no two computers on the same network can have the same number.

The IP address hierarchy guarantees two important properties:

- Each computer is assigned a unique address
- Although network number assignments can be coordinated globally, suffixes can be assigned locally without global coordination.

2.4 Classes of IP Addresses

The prefix of an IP address needs sufficient bits to allow a unique network number to be assigned to each physical network on the Internet. The suffix needs sufficient bits to permit each computer attached to a network to be assigned a unique suffix. Choosing a large prefix accommodates many networks, but limits the size of each network, while choosing a large suffix means each physical network can contain many computers, but limits the total number of networks.

The internet can consist of few large physical networks or many small networks. More important, a single internet can contain a mixture of large and small networks. Consequently, the designers chose a compromise addressing scheme that can accommodate large and small networks. The original scheme, which is known as *classful IP* addressing, divides the IP address space into three primary classes, where each class has a different size prefix and suffix.

The first four bits determine the class to which the address belongs, and specifies how the remainder is divided into prefix and suffix.

Class A – First bit 0 used for class identification, prefix consists of next 7 and rest 24 for suffix
Class B – First 2 bits start with 10 used for class identification, prefix consists of next 14 and rest 16 for suffix
Class C – First 3 bits start with 110 used for class identification, prefix consists of next 21 and rest 8 for suffix
Class D – First 4 bits start with 1110 used for class identification, and rest used as multicast address
Class E – First 4 bits start with 1111 used for class identification, and rest reserved for future use

First Four Bits Of Address	Table Index (in decimal)	Class of Address
0000	0	A
0001	1	A
0010	2	A
0011	3	A
0100	4	A
0101	5	A
0110	6	A
0111	7	A
1000	8	B
1001	9	B
1010	10	B
1011	11	B
1100	12	C
1101	13	C
1110	14	D
1111	15	E

Figure 2.1 - the mapping between the first four bits of an IP address and the class of the address. The mapping was used with the original classful scheme.

Class A, B and C are called *primary classes* because they are used for host addresses. Class D is used for multicasting which allows for delivery to a number of computers. To use IP multicasting, a set of hosts must agree to share a multicast address. Each host will, receive a copy of any packet sent to the multicast address.

2.5 Dotted Decimal Notation

Although IP addresses are 32-bit numbers, users seldom use binary. Instead, the software uses a notation that is more convenient for humans to understand*: dotted decimal notation*. This form expresses each 8-bit section of a 32-bit number as a decimal value and uses periods to separate sections. Figure 2.3 illustrates examples of binary numbers and their equivalent dotted decimal forms.

32-bit Binary Number	Equivalent Dotted Decimal
10000001 00110100 00000110 00000000	129 . 52 . 6 . 0
11000000 00000101 00110000 00000011	192 . 5 . 48 . 3
00001010 00000010 00000000 00100101	10 . 2 . 0 . 37
10000000 00001010 00000010 00000011	128 . 10 . 2 . 3
10000000 10000000 11111111 00000000	128 . 128 . 255 . 0

Figure 2.2: Examples of 32-bit binary numbers and their equivalent in dotted decimal notation.

As the example shows, the smallest possible value, 0, occurs when all bits of an octet are zero, and the largest possible value, 255, when all bits are one. Thus, dotted decimal addresses range from 0.0.0.0 through 255.255.255.255.

2.6 Division of address Space

The IP class scheme does not divide the 32-bit address space into equal size classes, and the classes do not contain the same number of networks. For example, half of all the IP addresses (i.e., those addresses in which the first bit is zero) lie in class A. Surprisingly, class A can contain only 128 networks because the first bit of a class A address must be zero and the prefix occupies one octet. Thus only seven bits remain to use for numbering class A networks. Figure 2.3 summarizes the maximum number of networks available in each class and the maximum number of hosts per network.

Address Class	Bits In Prefix	Maximum Number of Networks	Bits In Suffix	Maximum Number Of Hosts Per Network
A	7	128	24	16777216
B	14	16384	16	65536
C	21	2097152	8	256

Figure 2.3 the number of networks and hosts per network in each of the three primary IP address classes.

In addition to assigning an IP address to each host, the IP protocol specifies that routers should be given IP addresses as well. In fact, each router is assigned two ore more IP addresses. To understand why, recall two facts:

- A router has connections to multiple physical networks.
- Each IP address contains a prefix that specifies a physical network.

Thus, a single IP address does not suffice for a router because each router connects multiple networks. This is the fundamental principle of IP addresses:

An IP address does not identify a specific computer. Instead, each IP address identifies a connection between a computer and a network. A computer with multiple network connections (e.g. a router) must be assigned one IP address for each connection.

CONCEPTS OF WEB DESIGN

Introduction

INTERNET

This is a world-wide system of interconnected computers cooperating with each other to exchange data using a common software standard through telephone lines and satellite links.

USES OF INTERNET

1) **For Business/ To make money**: The Internet offers a wide range of business opportunities and facilities. One is able to trade online thus putting away all the tariffs and barriers experienced. I.e. The Internet is used to advertise and sell product and services.
2) **To Communicate**: The Internet has enabled users to get faster and more reliable communication. Communication ranges from electronic mail to Internet access. Through chatting and emails the Internet can be used to meet people. www.yahoo.com
3) **To have Fun:** The Internet provides access to many games that one can download to play online. (Entertainment in form of music, movies etc)
4) **Research:** Anyone can be able to find articles or information readily available on the Internet. It is an open library with access to some university online libraries.www.google.com
5) **News:** Ranging from headlines around the world to sports it is readily available. E.g www.nation.co.ke, www.cnn.com
6) **Education:** the Internet is a great learning tool. Many tutorials are available in various subjects thus allowing users to learn more.
7) **To find software:** The Internet contains a wealth of useful downloadable shareware (software you can use for free on a trial basis) e.g shareware.com
8) **To shop:** The Internet offers a wide range of goods that can be bought online e.g. cars, books e.t.c. However, security online is still questionable. E.g www.amazon.com

What Internet users can do.

- Post information for others to access and update it frequently

- Access multimedia information that include sound, video and images
- Exchange emails with friends
- Connect easily through personal computer and phone numbers.

Information available in the Internet is in form of
- Text documents
- Graphic files, video and charts
- Digitized sound
- Downloadable software (shareware)
- Host interactive forums

INTRANET
- Organizations can use Internet networking standards and web technology to create private networks called intranets.
- An Intranet is an internal organizational network that can provide access to data across the enterprise.
- It uses the existing company network infrastructure along with Internet connectivity standards and software developed for the World Wide Web.
- Intranets can create networked applications that can run on many different kinds of computers throughout the organization.
  - √ The principal difference between the Internet and an Intranet is that whereas the Internet is open to anyone, the Intranet is private and is protected from public visits by firewalls.
  - √ A **firewall** is a hardware or software placed between an organization's internal network and an external network to prevent outsiders from invading private networks.

EXTRANET
- Private intranet that is accessible to select outsiders.
- They are extended to authorized users outside the company eg authorized buyers could link to a portion of the company's intranet to obtain information about the cost and features of its products.
- The company can use firewalls to make sure that only authorized people can access the site.
- Extranets are especially useful for linking organizations with customers or business partners. They often are used for providing product availability, pricing and shipment data and electronic data interchange (EDI) or for collaborating with other companies on joint development or training efforts.

WEB TECHNOLOGIES/ INTERNET SERVICES/ INTERNET TOOLS

(1) WWW

What is the World Wide Web?

The official definition of the WWW is *"wide-area hypermedia information retrieval initiative aiming to give universal access to a large universe of documents."*

wide-area: The World Wide Web spans the whole globe.

hypermedia: It contains various types of media (text, pictures, sound, movies ...) and hyperlinks that connect pages to one another.

information retrieval: Viewing a WWW document (commonly called a Web page) is very easy thanks to the help of Web browsers. They allow you to retrieve pages just by clicking links, or entering addresses.

universal access: It doesn't matter what type of computer you have, or what type of computer the page you want is stored on - your Web browser allows you to connect seamlessly to many different systems.

large universe of documents: Anyone can publish a Web page - and nearly anyone has! No matter what obscure information you want to find, there is bound to be someone out there who has written a Web page about it.

What's the relationship between the WWW and the Internet?
The World Wide Web is just one of the many services that the Internet provides. Some other services provided by the Internet are email, FTP, gopher, telnet and usenet.
Almost every protocol type available on the Internet is accessible on the web including the following components: Email, FTP, Telnet, User news, HTTP

Features of WWW
- It has its own protocol i.e. HTTP
- It creates a convenient and user friendly environment
- It is the fastest components of Internet since it gathers together all the protocols into a single system.
- It relies on the hypertext as means of Information retrieval.
- It has the ability to work with multimedia and advanced programming languages i.e. text, graphics, video and audio.
- It is a delivery medium, content provider and subject matter.
- It connects users to almost any part of the Internet.
- It is used to explore intellectual, verbal knowledge and effective learning.
- It contains complex virtual web of connections and consist of files.
  It provides real-time collaboration, interactive pages and automatic push of information to client computers.

(2) FTP
- The Internet allows you to copy files between your computer and other computers on the Internet by using file transfer protocol (ftp). You connect your computer to an ftp server, an Internet host computer that stores files for transfer. You may be required to log in to retrieve a file, which varies from software, and text files to graphic files.

(3) TCP/IP (Transmission Control Protocol/Internet Protocol)
- A special set of protocols that is used to send data in a more reliable way.

(4) E-mail
- This is online communication between computer users. It is quick, convenient, efficient and cheap way to communicate with both individuals and groups.
- It's the most popular internet service.

(5) TELNET
It's a service that enables remote log in. Users are permitted to log in onto a host and perform tasks as if they are working on the remote computer itself.

(6) USENET/newsgroups, mailing lists
A huge network of discussion groups
(7) Gopher
This is a menu based system that allows a user to access information from a remote computer. Menu items point to a file which may be located on the same computer or on a different one.

(8) IRC
This is an Internet service that allows a number of users to connect to the same network node and communicate in real time.

role of web site in organizations

1. Improve Your Advertising Effectiveness

Placing your website address on all of your promotional material will help you gain additional exposure and encourage the visitors to first check your site for the information they are seeking.

2. Save Money on Printing and Distribution Costs

A website can act as your online brochure or catalog that can be changed or updated at anytime. If you employ a content management system (CMS) you can make changes quickly and at no charge.

3. Easy Access to New Customers

You can have your existing customers refer you to their friends and relatives using only your web address or URL.

4. Easy to Use and Update

If maintained properly your website will always be up-to-date and current. Easily make updates, edits and deletions from any computer on the Internet. No more having to pay a programmer every time you want to chance a date or add a product.

5. Improve Productivity

A website increases your company's productivity because less time is spent explaining product or service details to customers because all this information is available 24 hours a day on your website.

6. Educate Your Customers

Your website can offer free advice about your products and services. This information can be delivered at any hour in a well thought out and consistent way.

7. Expand Your Market

The Internet allows businesses to break through the geographical barriers and become accessible from any of the world by a potential customer that has an Internet connection. Selling products online is cheaper and easier for you and your customers.

8. Extend Your Local Reach

Extend the local reach of your brick-and-mortar store to consumers around the world. You are open for business 24/7 365 days/year with all the information the visitors needs to make an informed decision.

9. Promote & Sell Products & Services

Provide photos and detailed descriptions of your products or services. Explain why your products or services are superior to your competitors. Show visitors how your products or services can help them in their personal or professional lives.

10. Promote Your Brick and Mortar Footprint

When customers and potential customers are out and about they will still be able to find you via their phone. Your phone number, address and full selection can be made available form your website or mobile-friendly site.

11. When You Change Locations

If you move your business to a new location your customers can still find you because your main marketing tool, your website, is easily changed and updated. Your website is flexible and if your search engine optimization is done properly your business will appear to online visitors who search for you.

12. Great Tool for Finding New Employees

You can post job opportunities for available positions and applicants can investigate your company and apply online.

13. Your Own Internet Identity

Your own domain name (www.yourcompany.com) establishes a strong online brand identity.

14. Set-up Email Addresses

You can set-up a personalized email addresses for the company, yourself and your employees. If you set-up a system to accept emails on your site you can also email updates, notices, sales and holiday store hours to your customers.

15. Two-Way Communication

Customers can quickly and easily contact you, give feedback on your products or ask about product availability.

16. Cheap Market Research

You can feature visitor polls and online surveys to take the pulse of your customers.

17. Build Your Reputation

Become or remain the expert by demonstrating knowledge and expertise in your area of work. Write blog posts and articles on the site that educate visitors and help them understand your business and offerings.

18. Improve Customer Service

Information requests can be processed immediately via online forms and auto-responders automatically day or night.

Web programming

Web programming refers to the writing, markup and coding involved in Web development, which includes Web content, Web client and server scripting and network security. The most common languages used for Web programming are XML, HTML, JavaScript, Perl 5 and PHP. Web programming is different from just programming, which requires interdisciplinary knowledge on the application area, client and server scripting, and database technology.

Web programming can be briefly categorized into client and server coding. The client side needs programming related to accessing data from users and providing information. It also needs to ensure there are enough plug ins to enrich user experience in a graphic user interface, including security measures.

1. To improve user experience and related functionalities on the client side, JavaScript is usually used. It is an excellent client-side platform for designing and implementing Web applications.

2. HTML5 and CSS3 supports most of the client-side functionality provided by other application frameworks.

The server side needs programming mostly related to data retrieval, security and performance. Some of the tools used here include ASP, Lotus Notes, PHP, Java and MySQL. There are certain tools/platforms that aid in both client- and server-side programming. Some examples of these are Opa and Tersus.

Approaches to web programming

Simply put, Web Applications are dynamic web sites combined with server side programming which provide functionalities such as interacting with users, connecting to back-end databases, and generating results to browsers.

Examples of Web Applications are Online Banking, Social Networking, Online Reservations, eCommerce / Shopping Cart Applications, Interactive Games, Online Training, Online Polls, Blogs, Online Forums, Content Management Systems, etc..

Technologies

There are two main categories of coding, scripting and programming for creating Web Applications:

I. Client Side Scripting / Coding - Client Side Scripting is the type of code that is executed or interpreted by browsers.

Client Side Scripting is generally viewable by any visitor to a site (from the view menu click on "View Source" to view the source code).

Below are some common Client Side Scripting technologies:

- HTML (HyperText Markup Language)
- CSS (Cascading Style Sheets)
- JavaScript
- Ajax (Asynchronous JavaScript and XML)
- jQuery (JavaScript Framework Library - commonly used in Ajax development)
- MooTools (JavaScript Framework Library - commonly used in Ajax development)
- Dojo Toolkit (JavaScript Framework Library - commonly used in Ajax development)

II. Server Side Scripting / Coding - Server Side Scripting is the type of code that is executed or interpreted by the web server.

Server Side Scripting is not viewable or accessible by any visitor or general public.

Below are the common Server Side Scripting technologies:

- PHP (very common Server Side Scripting language - Linux / Unix based Open Source - free redistribution, usually combines with MySQL database)
- Zend Framework (PHP's Object Oriented Web Application Framework)
- ASP (Microsoft Web Server (IIS) Scripting language)
- ASP.NET (Microsoft's Web Application Framework - successor of ASP)
- ColdFusion (Adobe's Web Application Framework)
- Ruby on Rails (Ruby programming's Web Application Framework - free redistribution)
- Perl (general purpose high-level programming language and Server Side Scripting Language - free redistribution - lost its popularity to PHP)
- Python (general purpose high-level programming language and Server Side Scripting language - free redistribution)

Program Libraries

Program libraries are a collection of commonly used functions, classes or subroutines which provide ease of development and maintanance by allowing developers to easily add or edit functionalities to a frame-worked or modular type application.

Web Application Frameworks

Web Application Frameworks are sets of program libraries, components and tools organized in an architecture system allowing developers to build and maintain complex web application projects using a fast and efficient approach.

Web Application Frameworks are designed to streamline programming and promote code reuse by setting forth folder organization and structure, documentation, guidelines and libraries (reusable codes for common functions and classes).

Web Application Frameworks - Benefits and Advantages

- Program actions and logic are separated from the HTML, CSS and design files. This helps designers (without any programming experience) to be able to edit the interface and make design changes without help from a programmer.
- Builds are based on the module, libraries and tools, allowing programmers to easily share libraries and implement complex functionalities and features in a fast and efficient manner.
- The structure helps produce best practice coding with consistent logic and coding standards, and provides other developers the ability to become familiar with the code in a short time.

Popular Web Frameworks for Web App Development

Websites have become imperative for businesses in this evolving world for creating a prominent online existence. Creating sophisticated and intuitive websites is the best way to attract new users. The right web framework can assist you in the development of such web applications and websites.

To make it easy for you to choose the top web frameworks, we have made a list of some of the **most popular web frameworks** that you can use to make web development hassle-free.

We will look at more such features in detail to understand why you should use frameworks.

In this blog, we will be looking at the most in-demand web development frameworks of 2021. Some of the most loved frameworks are **Ruby on Rails, Django, Meteor,** and **Vue**, to name a few. Most developers find them extremely productive and useful.

But before we discuss their features, let's first understand what a framework is and what is the difference between frontend and backend frameworks.

With the rise in the standards of web applications, it is absolutely irrational to recreate such refined techniques. Also, web developers have to keep an eye on the latest trends relevant to the best frameworks.

Table of Content

What is a web framework?

A web framework is a software platform for developing web applications and websites.

Web application frameworks offer a wide range of pre-written components, code snippets, and whole application templates. Web development frameworks can be used for the development of web services, web APIs (Application Programming Interface) and other web resources.

But why are frameworks necessary?

- They help in increasing your traffic
- Make the developing and maintaining process much quicker and easier.
- Include **standardized coding practices** and conventions for code structures.
- Systemize the developers' programming process and avoid errors and bugs.
- Allows you to focus entirely on your app, taking care of all the background details like data binding and configuration efficiently.

There are generally two types of development frameworks – **client side** and **server side frameworks.** While client side frameworks are used for dealing with the user interface, a server side framework works in the background to ensure the smooth functioning of the website.

Front-end Frameworks	Back-end Frameworks
The frontend is the part of the website visible to the users.	Backend refers to the background functioning of the sites.
Also known as client-side frameworks	Also known as Server side frameworks
Involves UI-UX designing, SEO optimization, performance and scalability enhancing, creating reusable templates	Involves database management, security, URL routing, designing site architecture, the server handling
Frontend Languages – HTML, CSS, JavaScript, JQuery	Backend Languages – Python, JavaScript, PHP, Ruby, .NET
Frontend Frameworks- React, Vue, BootStrap, Ember, Angular	Backend frameworks- Django, Ruby On Rails, Express, Spring, ASP.NET Core
Front end frameworks provide pre-written code snippets, reusable templates, integrable elements and manage user interaction	Database manipulation, user authorization, privacy encryptions, reusable components are some benefits of using backend frameworks

Classification of Framework Architectures

The architecture of a framework defines the relation between the different components of the framework. *The architecture decides how the various layers will interact with each other*. It is important to use a well-understood architecture as it largely defines how your application functions.

- **Model View Controller**

  Many frameworks for web apps work on **Model View Controller** (MVC) models. To split data models with business standards from the UI, MVC architecture is preferred. It is a good practice as it usually fosters code reuse, modularized code, and permits various interfaces.
- **Model-View-ViewModel (MVVM)**

  Frameworks, such as KnockoutJS and VueJS use the **Model-View-ViewModel** or the Model-View-Binder architecture model. In this architecture, the view layer acts as a controller and converts the data objects from the Model layer into manageable components. Since the View layer handles all the user requests directly, the data binding is much more straightforward.
- **Push-based vs Pull-based**

  Generally, MVC frameworks observe a push-based architecture, also known as **'action-based.'** They adopt actions that do the necessary processing and then accordingly 'push' the data to a view layer to furnish the outcome. Some examples of frameworks are *Spring MVC, Ruby on Rails, Sails.js, Django,* and *CodeIgniter.*

Another option to this architecture is 'pull-based,' also termed as **'component-based.'** This is a kind of framework that starts with the view layer, which in turn can 'pull' outcomes from diverse controllers as required. Some examples of pull-based architectures are *JBoss, Tapestry, Lift, Wicket, Micro,* and *JavaServer Faces*.

Struts, ZK, Play, and RIFE use both push-based and pull-based application controller calls.

- **Three-tier organization**

  The three-tier organization applications are well-regulated in three physical tiers: application, database, and client-side. This database is usually a **relational database.**

  The application possesses business logic that runs on a server and corresponds with the client utilizing HTTP. The client uses a web browser that runs the HTML code developed by the application layer.

Most Popular Web Frameworks

A framework has become critical for the development and maintenance of websites. With a lot of choices available, it becomes extremely difficult to decide on one. Here we have curated a list of the best libraries and frameworks along with their numerous features that are used today for ease of development.

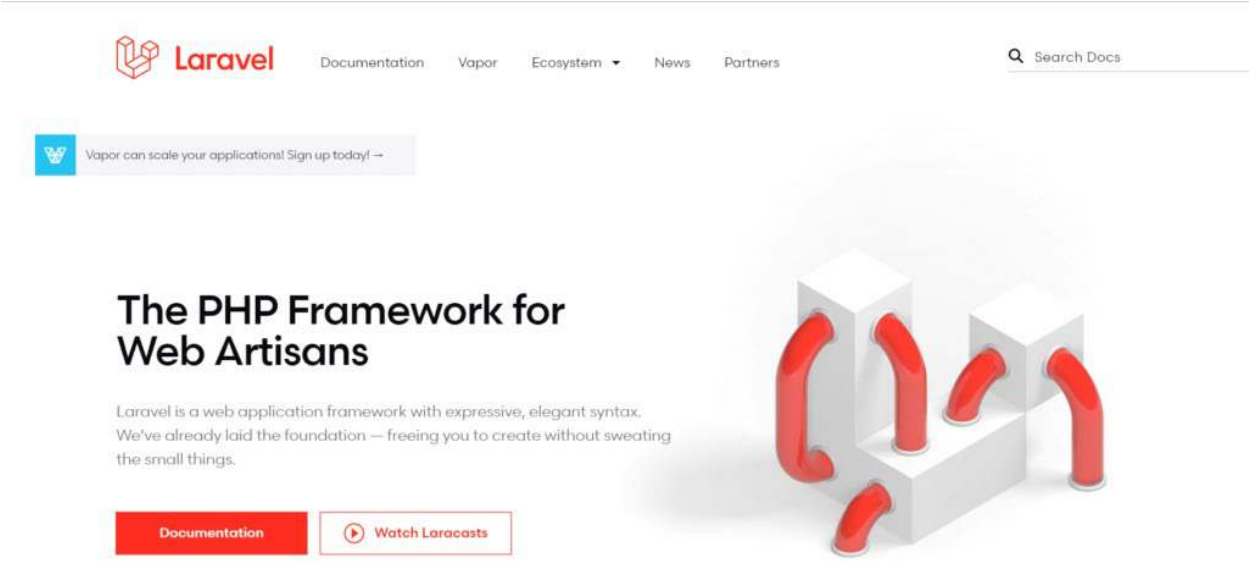
1. Ruby on Rails

Ruby on Rails is a dynamic web application framework, perfect for developing a high-speed application. Discovered by David Heinemeier Hansson, Ruby on Rails applications are generally ten times faster. It is one of the best backend frameworks as it *comprises everything that is needed to form a* ***database-driven application***.

Websites that have incorporated Ruby on Rails are GitHub, Airbnb, GroupOn, Shopify, Hulu, to name a few. Not only this, but you can also explore top companies that use Ruby on Rails for their websites and apps.

- Language: Ruby
- Framework Link: http://rubyonrails.org
- Github Link: https://github.com/rails/rails
- Stable Release: Rails 6.0.3.1

2. Laravel



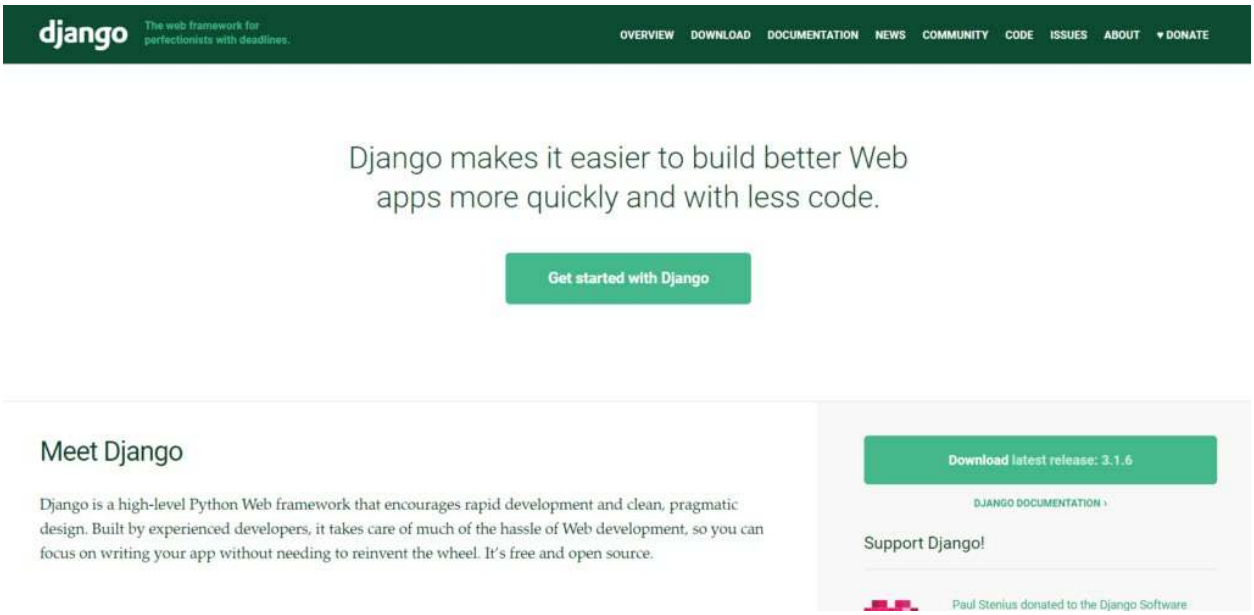
Simple, elegant, and readable are the USPs of Larval. Taylor Otwell developed Laravel in 2011. Similar to other contemporary development frameworks, it functions on an MVC architectural model that uses PHP.

Laravel has API support and contains a *good quantity of packages* that could expand its reach. **Laracasts** is a tutorial website with thousands of videos on Laravel, PHP, and frontend technologies in the ecosystem of Laravel.

Laravel includes a lot of features such as dependency injection, server side rendering It is proving to be the best framework for websites like Neighbourhood, Travel, Deltanet, Lender, and many more.

- Language: PHP
- Framework Link: https://laravel.com/
- Github Link: https://github.com/laravel/laravel
- Stable Release: Laravel 7.15.0

3. Django

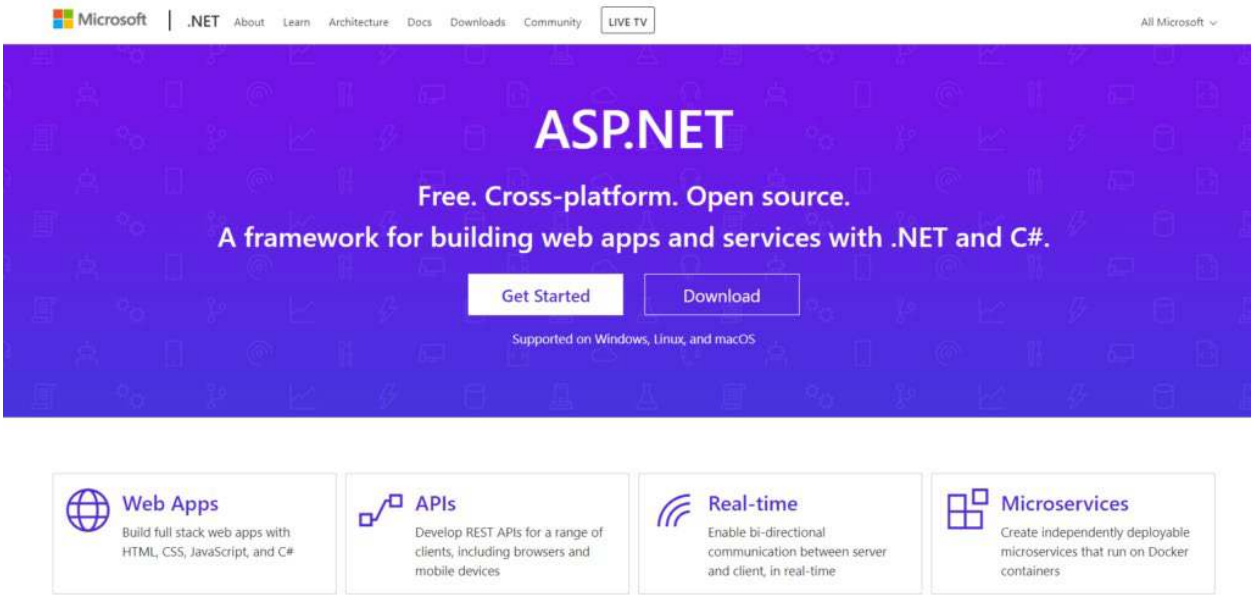


Django is one of the reliable backend web development frameworks that aid in developing a robust web application. It uses **'Convention over Configuration,'** as well as **DRY pattern**. Django provides tools and techniques to developers for building a safe website, implementing the security features in the framework for best web development.

It is perfect when competing with fast-moving deadlines and the challenging requirements of veteran developers. The Django applications are tremendously scalable, fast, versatile, and secure. The companies that use Django are Pinterest, Disqus, YouTube, Spotify, Instagram, and other popular giants.

- Language: Python
- Framework Link: https://www.djangoproject.com
- Github Link: https://github.com/django/django
- Stable Release: Django 3.0.7

4. ASP.NET

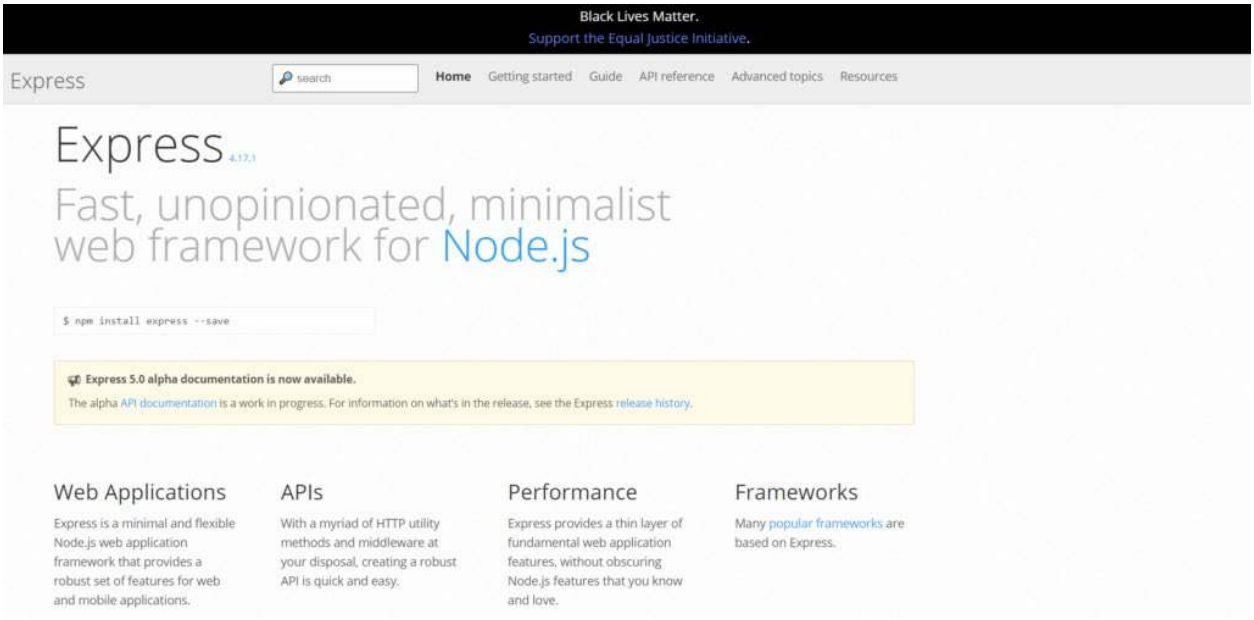


A popular web development framework that is immensely helpful to build dynamic web applications for PC and mobiles is ASP.NET. **Microsoft** invented it to let programmers build vibrant websites, applications, and services.

Asp.net Core is a novel version of Asp.net and is renowned for its speed, productivity, and power. It is a high-performing and lightweight. Some famous companies using ASP.NET are TacoBell, GettyImages, StackOverflow, to name a few.

- Language: C#
- Framework Link: http://www.asp.net/
- Github link: https://github.com/aspnet
- Stable Release: Asp.net Core 4.8

5. Express



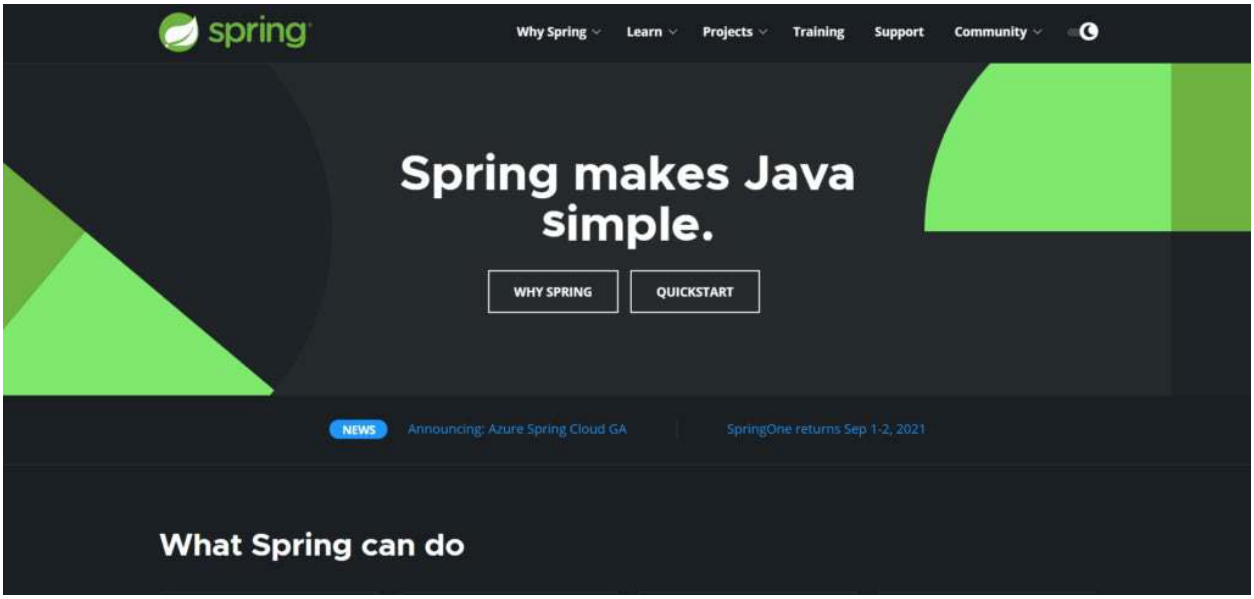
Express is an open-source backend framework that supplies a variety of features for website and mobile development. It is considered one of the best web development frameworks as it is a minimal, flexible, framework for backend development on **Node.js**.

It functions on Java, one of the major programming languages, for building up backend web applications. It is preferred by developers all across the globe. Additionally, Express helps develop **competent APIs**.

It is one of the significant Express components of the mean software bundle. Companies using Express for their projects are *MuleSoft, Accenture, Myntra, Uber, Myspace, etc.*

- Language: JavaScript
- Framework Link: http://expressjs.com/
- Github Link: https://github.com/strongloop/express
- Stable Release: 4.17.1

6. Spring



The Spring Framework, invented by **Pivotal Software**, is one of the most popular backend web frameworks. Built for the Java platform, you can form simple, flexible, and fast applications and systems with the Spring framework.

Though it does not oblige any precise programming pattern, this backend framework has gained popularity in the Java Community, with the addition of the Enterprise JavaBeans model. A Java application can utilize its main features, but the extensions help in forming apps on top of the Java Enterprise Edition platform.

The developers mainly use Spring for creating high-performing and vigorous applications. Companies that utilize Spring are Deleokorea, Intuit, Zalando, MIT, and Zillow, to name a few.

- Language: Java
- Framework Link: http://projects.spring.io/spring-framework/
- Github Link: https://github.com/spring-projects/spring-framework
- Stable Release: Spring 5.2.5

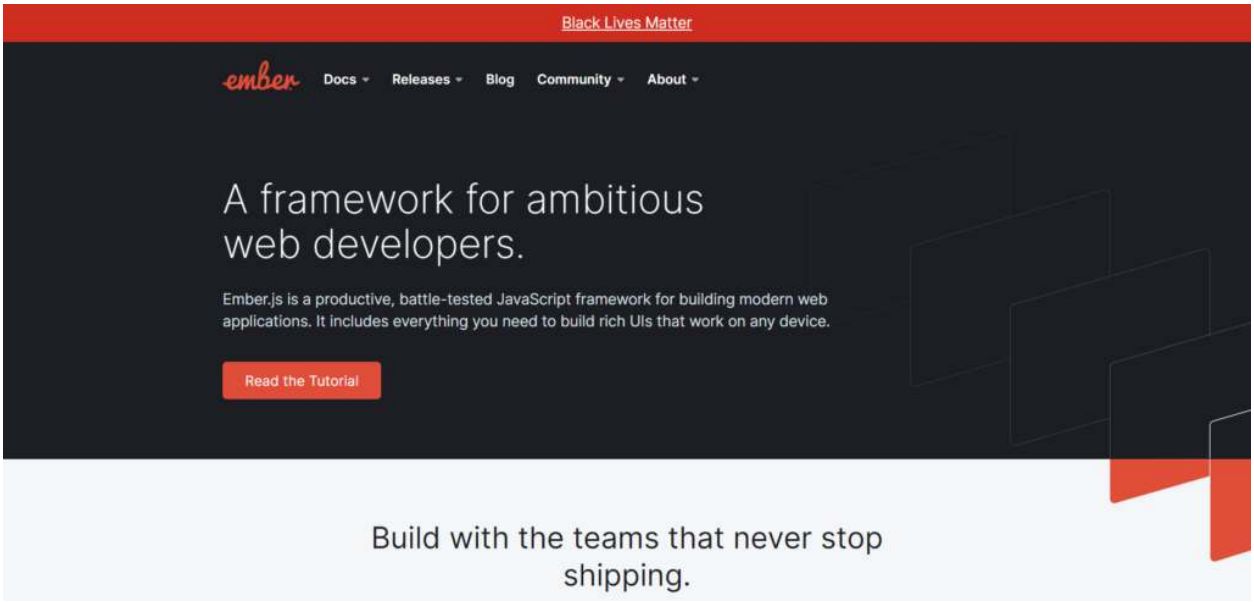
7. Angular

Angular is one of the best front-end web development frameworks for developers. It is a TypeScript-based, open-source web app framework managed by Google Angular Team. You can use Angular to construct vast, **high-functioning web applications**.

Numerous apps are built with the assistance of Angular as its applications are much easy to maintain. Websites that use Angular are Upwork, Lego, PayPal, Netflix, etc.

- Language: JavaScript
- Framework Link: https://angular.io/
- Github Link: https://github.com/angular/angular
- Stable Release: Angular 9.1.11

8. Ember



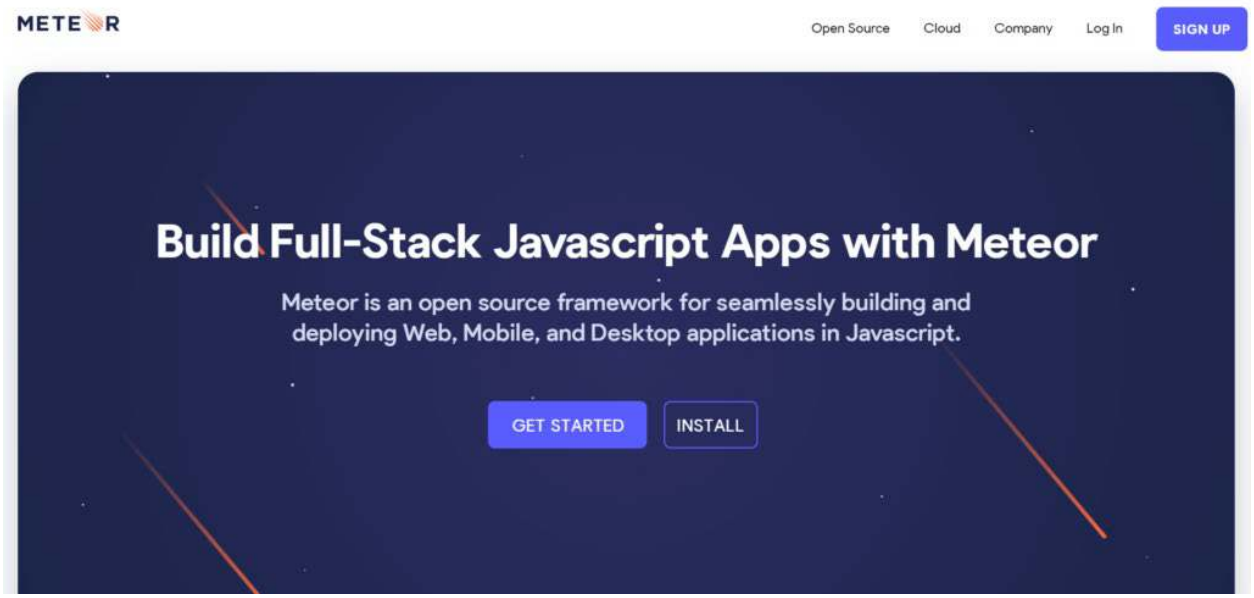
Ember.js is an open-source JavaScript framework based on the model-view-view-model (MVVM) architecture pattern. It aids web developers to form extensible single-page applications by utilizing general idioms and advanced practices.

Though mainly considered the best web framework, it also supports building mobile and desktop applications. A noteworthy example of Ember desktop application is **Apple Music**, an attribute of the iTunes desktop app.

The Ember community of developers is ever-expanding, and new releases and features are added frequently. Heroku, Microsoft, Netflix, and Google utilize this framework entirely on a regular basis.

- Language: JavaScript
- Framework Link: https://emberjs.com/
- Github Link: https://github.com/emberjs
- Stable Release: Ember 3.18.1

9. Meteor



MeteorJS or Meteor is a framework that can create real-time web and mobile apps quickly. It supports instant prototyping and yields cross-platform code for iOS, Android, desktops and browsers. **Galaxy**, its cloud platform, exceptionally simplifies scaling, deployment, and monitoring.

According to **Builtwith**, there are currently 19,913 websites using Meteor. Some of them are WishPool, HaggleMate, and Telescope.

- Language: JavaScript
- Framework Link: https://www.meteor.com/
- Github Link: https://github.com/meteor/meteor
- Stable Release: Meteor 1.10

10. Vue

Vue.js is the most high-performing lightweight JavaScript library of UI components. You can build **dynamic UIs** and **Single Page Applications** extremely fast using Vue. It provides a lot of components that can assist you in developing a seamless user interface.

Vue is a flexible framework that allows you to integrate multiple third-party solutions. It also employs both one-way and two-way data binding. You can effortlessly apply this open-source framework to amplify applications. Companies using this framework for their projects are Gitlab, Behance, Grammarly, 9GAG, to name a few.

- Language: JavaScript
- Framework Link: https://vuejs.org/
- Github Link: https://github.com/vuejs/vue
- Stable Release: Vue 2.6.11

These top frameworks have different uses and benefits. It is important to identify your ideal fit instead of going with whichever is on the top.

For building simple, lightweight client-side applications, we suggest using ***Vue***. For dynamic web apps and single-page applications, ***React*** would be a better choice. You can also use ***React Native*** for developing highly performative mobile apps.

Meteor is a good choice for building cross-platform hybrid apps. You should opt for ***Ruby on Rails*** if you want increased productivity and work under short deadlines. ***Angular*** is a complete solution for high-functioning robust applications built using JavaScript or you can use ***Django*** for Python-based complex websites and apps.

On the plus side, Django and Angular both support server side rendering. Use the ***Spring*** framework if you prefer working with Java and want to develop rich, testable apps.

We recommend using ***Express*** for creating REST APIs for web and mobile applications using Node.JS. ***ASP.NET*** is a lightweight solution for you if you want to build fast and responsive mobile and desktop apps using .NET, while ***Laravel*** is better for database-driven sites such as blogs or eCommerce apps.

Finally, you should go for ***Ember*** if you want a complete client-side application framework with data management and testing. So now that you know which are the most popular frameworks, let's talk about why frameworks are important. Why do we even need to use them?

Why Should You Use Web Frameworks?

A web development framework is not absolutely necessary for building a web application. But it surely provides tools that can simplify the process and makes it hassle-free. Using a framework is a good practice as developing, testing, and maintaining apps becomes almost impossible without it.

Let us discuss some features of the best web application frameworks that make it a useful development tool for developers.

1. **Simplifies Programming Languages**

   Frameworks are sometimes necessary for working with complicated programming languages as they can provide understandable and integrable functions. Working with JavaScript, for example, would require writing huge lines of code for maintaining data consistency throughout the app. But frameworks like React do this automatically.

   Similarly, they can automate a lot of other tasks by providing **built-in functions** that would be complicated for us. All major languages have multiple frameworks for different usage needs. They also are a great way to explore and learn a new language. Just understanding the basic programming concepts of the language, you can start coding in the framework.
2. **Optimizes Development Process**

   Frameworks are a great way to **bootstrap the development cycle**. Developers working under a time constraint or a budget constraint should opt for popular open-source frameworks like mentioned above. We suggest using React or Vue for quickly developing client-side applications and Meteor or Ruby on Rails for developing MVPs(Minimum Value Products) overnight.

   You can develop custom applications within a short time, building up ready-made templates according to one's needs. They also provide great flexibility to developers for experimenting and implementing creative solutions.
3. **Reusability**

   Another great advantage is **reusability**. It fosters the building of reusable components that can be integrated into other areas of the code and even in different projects.

   Reusability increases productivity, maintainability, and data consistency, especially in large applications with a huge developing team. Since the developers do not have to write everything from scratch, it gives them time to focus on the creation of the application logic.
4. **Standardized Code Practices**

   Most frameworks employ a specific code structure that has to be followed. This helps maintain **uniformity** throughout the application and **streamlines the debugging process**. It also makes the code simple and concise.

   It fosters the usage of standardized coding and designing practices. For example, many frameworks and libraries follow the Material Design Guidelines by Google. While this can be a hindrance for some developer's flexibility, if you are a beginner or working in a huge team, having standardized practices ensures consistency and readability.
5. **Testing and Debugging**

   Most frameworks provide **in-built testing and debugging** functions. With the code reusability and standard practices, your application is already better maintained and understandable.

   With features such as unit testing, code completion, simultaneous code editing, and more, maintaining and updating your app becomes extremely easy.
6. **Community**

   All of the well-known web application frameworks in this list have a huge community of developers to their name. With easy **access to solutions** and a huge amount of knowledge, working with frameworks comes with extensive support and guidance.

   Plus, users have started developing numerous tools and libraries that assist in working with these frameworks. This means you have access to a world of additional features that extend the capabilities of already robust frameworks.

Which is the easiest web framework?

There are various frameworks that are obtainable to choose from. Some of them contain pre-packaged folders, files, and ways to get started quickly. These are Ruby in Rails with convention over configuration principle, Django with MVC design pattern, ExpressJS, Angular.js with model-controller-view architecture, and Laravel that values readability, elegance, and simplicity.

Which is the most popular framework for web development?

The web framework plays a crucial part in the arena of website development. The best web frameworks are Laravel, Django, Rails, AngularJS, and Spring to name a few that facilitates a quick and effective setup, along with a flexible environment.

Why are frameworks useful?

Web development frameworks act as building blocks for websites and web applications. They provide multiple features that make client-side and server-side development hassle-free. The most popular sites and apps use these frameworks for generating business. You can use the templates that the frameworks provide for cutting down the development time and libraries and additional tools help in optimizing performance and increasing user traffic.

Conclusion

A web framework acts as a crucial element in the software development process. Each of the frameworks mentioned above has its characteristics and advantages.

Using these popular web development frameworks is highly recommended as it can speed up your development process. Along with these, you can also use some web development tools to assist you in the process. But it is important to choose the right framework for your project or you can get stuck with a robust framework like Ruby on Rails for developing a simple blogging site or a SPA.

We, at Monocubed, can help you find your ideal web framework by focusing on your business requirements and clients to avail higher traffic rates for your website. Our team of expert developers will assist you throughout the whole development process.

https://www.monocubed.com/10-most-popular-web-frameworks/

https://www.geeksforgeeks.org/top-10-frameworks-for-web-applications/

Coding Guidelines, Standards & Convention

Coding guidelines are sets of rules and standards used in programming a web application project.

These rules and standards apply to coding logic, folder structure and names, file names, file organization, formatting and indentation, statements, classes and functions, and naming conventions. These rules also enforce writing clear comments and provide documentation.

Important benefits of using Coding Guidelines

- Creates the best environment for multiple programmers to work on the same project
- Provides ease of maintainability and version management
- Delivers better readability and understanding of the source code
- Insures that other developers can understand and become familiar with the code in a short time

Web Applications Lifecycle Model

Web Application Lifecycle is the process of developing a web application and involvement of the multiple teams that are engaged in the development process. Each organization may set forth its own unique style of operating.

Some companies follow a certain standard model such as SDLC (System Development Life Cycle) or Agile Software Development Model.

- SDLC is the traditional process of developing software or web applications by including research to identify and define the application requirements, information analysis, architectural design and specifications blueprint, team involvement, programming, testing and bug fixing, system testing, implementation and maintenance.

- Agile Software / Web Application Development is the iterative development process and development process practices that focus on collaboration of people involved and provide a better procedure to allow revisions and evolution of web application requirements. Agile methodology includes research, analysis, project management, design, programming, implementation, frequent testing, adaptation and maintenance.

Web Application Development Process

Web Application Development Process organizes a practical procedure and approach in application development.

For detail information: Web Application Development Process

The following list of procedures and suggested documents provide a good outline for a Web Application Lifecycle and Process:

- Roadmap Document: Defining Web Application, Purpose, Goals and Direction
- Researching and Defining Audience Scope and Security Documents
- Creating Functional Specifications or Feature Summary Document
- Team Collaboration and Project Management Document
- Technology Selection, Technical Specifications, Illustrative Diagram of Web Application Architecture and Structure, Development Methodology, Versions Control, Backups, Upgrades, Expansion and Growth Planning Document, Server Hardware / Software Selection
- Third Party Vendors Analysis and Selection (Merchant Account and Payment Gateway, SSL Certificate, Managed Server / Colocated Server Provider, Fulfillment Centers, Website Visitor Analytics Software, Third Party Checkout Systems, etc.)
- Application Visual Guide, Design Layout, Interface Design, Wire Framing
- Database Structure Design and Web Application Development
- Testing: Quality Assurance, Multiple Browser Compatibility, Security, Performance - Load and Stress Testing, Usability
- Maintenance

Web Application Testing

Testing is an important part of the Web Application Development process. On occasion, testing would consume more manpower and time than development itself.

Below are some of the most common testing needed for any web application development process:

- Quality Assurance and Bug Testing
- Multiple Browser Compatibility
- Application Security

- Performance - Load and Stress Testing
- Usability

Trends and Popularity

The demands for companies to build Web Applications are growing substantially.

If planned and built correctly, web applications can:

- Reach and service millions of consumers and businesses
- Generate substantial, multi-layer / multi-category income from consumers, businesses and advertisers
- Easily build business goodwill and assets based on audience reach, popularity, technology and potential growth

Below are good reasons for companies to build web applications:

- Companies want to streamline their internal departments and functions, operations, sales and project management, etc.
- Companies want to take advantage of a web based application's flexibility and versatility, by moving away from the traditional desktop application platform to the web application platform
- Companies want to gain more clients or better service their current clients by offering convenient services and solutions online
- Companies want to build new web applications to offer innovative services or solutions to online users and businesses

Business Impact

Today's web applications have substantial business impact on the way companies and consumers do business such as:

- There are opportunities to gain the upper hand and bypass the traditional brick and mortar companies when this type of opportunity was rarely possible or existed before the explosion of the web
- The new web created a global business environment which challenges the way in which traditional companies do business
- Companies need to reinvent and evolve in order to compete in today's trends, online business and global marketplace
- Businesses and consumers have more options and resources to research and easily compare and shop around for the best deals
- Information and resources are immense and available to everyone who seeks it
- Businesses or companies who used to profit from consulting or advice, that can now be easily acquired online are struggling, and will need to take a new business direction if they want to stay solvent

By reading the following descriptions, you can get a greater idea about languages in trend this year. Here is a list of top programming languages of 2019:

1. Javascript:

Javascript is no doubt the most trending language these days. It plays a chief role in front-end development. It can be used to develop interactive web pages very conveniently. Most of the full-stack software developers in the contemporary market prefer to use it.

Stack Overflow, a popular website which is mainly used by developers surveyed 100,000 developers in 2018 to find the most preferred language. Javascript topped the list, followed by HTML, CSS, SQL, and others.

Here is the representation of their survey in the following chart:

This year too, it has been expected that Javascript will top the list again. The reason behind this popularity is probably its compatibility with all browsers. Moreover, it is also used in server-side through node.js.

2. Python:

Python has grown more than any other language in recent years. The major reason behind its growing popularity python is the ***rise of Artificial intelligence*** as well as the elimination of semicolon as an indication of the end of the statement. Python is one of the ***preferred languages for creating AI*** or machine learning based web & mobile apps. In many ways, it is similar and different from other programming languages.

The picture above shows the traffic of different Python packages. Pandas which was introduced in 2011 is the fastest growing python package. It is a multi-purpose language which can be used by software product development companies for data science as well as web designing.

It would not be wrong to say that the growth of data science has enhanced the development of python as a programming language. Moreover, Python is also replacing Javascript as a teaching language in institutes.

3. Java:

This language has survived at the peak in the programming industry from the past 20 years. It is widely used for building enterprise scale web application. Android mobile app developers also rely on this language for programming.

It is considered as one of the most stable languages. This is why it is the most preferred language by large enterprises. Another important factor which has kept its magic intact among web development companies is its independence from platforms.

It is known for high cross-platform compatibility. The Java Virtual Machine (JVM) allows it to run on various devices and platforms. Most of the fortune 500 companies use Java to develop back-end applications.

4. PHP:

It stands for Hypertext Preprocessor and is a popular scripting language found in 1995. Newer languages could not make any difference to the popularity of PHP frameworks. This is mainly because it kept on evolving throughout these years.

Most of the frameworks are free of cost and provide strong security features. It is the best server-side programming language according to a survey by w3techs.com.

It has over-the-top benefits like libraries and modules which assure dynamic software development. This is why most of the websites and content management systems are written in the PHP programming language. There are many PHP development companies, which use this language for creating

Criteria for choosing a programming language

1. Learning curve - if your team finds it difficult to adopt and there is no good introductions and tutorial, better skip it
2. Active Community - around this technology/framework/language is very important, this is driving force
3. Tooling - how much productivity tools exist around it
4. Comfort level - of developers in a team or company
5. Availability of developer(s) who has experience and build some proto-types with this language/framework
6. Suitability for the problem at hand (Is the project web-centric? Does it use a lot of concurrency? Is real-time performance important? Is there a lot of number-crunching involved? Is it mainly transactional? etc.)
7. Overall quality of available tools and third-party libraries
8. Bindings for other technology (doesn't make sense to use a language that doesn't have any decent libraries for your choice DBMS)
9. Legal and licensing issues (not usually a problem, but licensing the tools can be a cost factor)
10. Platform support (even more important if you have to support multiple platforms, or want to be able to switch platforms down the road)
11. Integration with other systems (e.g., .NET works best if combined with other Microsoft technology; PHP and Apache go well together; etc.)
12. Paradigm support
13. Philosophy
14. Semantic properties (static vs. dynamic, etc.)
15. Maturity
16. Stability (Will your code break in 5 years? Will you be able to fix it?)
17. Quality of available programmers (it's easy to find a hundred PHP programmers, but about 95 of them will be incompetent; it's hard to round up 5 Lisp programmers, but those that you do find know their stuff)
18. Security - no programming language is inherently secure or insecure, but some provide more and better tools for secure programming than others; for example, avoiding XSS is much easier in Haskell (use the type system to distinguish between raw strings and HTML) than it is in PHP (remember to call `htmlspecialchars` appropriately)

Common web programmimg interfaces

CCI (Common Client Interface)

NCSA Mosaic CCI (Common Client Interface) is an interface specification (protocol & API) that enables client-side applications to communicate with NCSA Mosaic, the original web browser, to control Mosaic or to obtain information off the web via Mosaic. Note that this is not for invoking client-side applications (applets) from Mosaic, but for controlling Mosaic from the application. Invocation of client-side applications from a browser is currently specific to the browser, but most support NCSA helpers. Once the application is running, it can communicate with the browser with CCI. CCI is not the only interface currently defined for this purpose, but it seems to be meeting with some acceptance, as Tcl and Perl now support it.

CGI (Common Gateway Interface)

is an interface specification that enables web servers to execute an external program, typically to process user requests.

A Web daemon *executes* a CGI program on the server and returns the results to the client (e.g. a query against a database server), rather than simply returning a copy of the referenced file, as it does with an HTML reference. Parameters are passed from the server to the CGI program as environment variables. The program is sometimes written in C, C++, or some other compiled programming language, but more often it is written in a scripting language (e.g. Perl, Tcl, sh). To prevent damage to the server, CGI programs generally are stored in a protected directory under the exclusive control of the webmaster.

6 Benefits Of Web Authoring Tools For Your Company

In a digital and cloud-based world, companies still tend to work the old way. A lot of trainers are still using traditional authoring tools, locally saved into their desktop. Even though this is a great way to work, trends are evolving: It is now time to move to web authoring tools. As it might sound a little bit scary, we decided to gather in this article the 6 main benefits provided by these new authoring softwares.

The Advantages Of Using Web Authoring Tools

Human beings are naturally reluctant to change. That being said, trends are evolving faster and faster in our digital corporate world. Especially in the eLearning field. Nowadays, a lot of our leads are hesitating to use web authoring tools instead of classical ones. Maybe because they fear to lose control over their material. Maybe because they fear to get overwhelmed by technical stuff. If you are in this case, do not worry. Web authoring tools are actually even easier to use than traditional ones.

In this article, we are going to highlight 6 main benefits provided by web authoring tools in your everyday life. This way, you will discover how easy it is to use, and to implement as a working tool.

1. Web Authoring Tools Are Time And Cost Effective.

Often, because people are used to paying a one shot fee for traditional authoring softwares, they find web authoring tools expensive, with their recurrent billing system. However, they do not realize the cost effectiveness potential of such solutions. First and foremost, maintenance costs are included in your recurrent fees. This alone is worth paying your license on a recurrent basis.

Besides, with classical authoring tools, one license is dedicated to one user, on a single computer. With web authoring tools, this is not the case. You can easily share your license between 3 to 4 trainers.

The only condition is not to be connected with the same license at the same time. By working like that, you can spare about 3 licenses, and just share the access between co-workers for an enhanced efficiency.

2. Web Authoring Tools Are Always Up To Date.

Are you tired of updating your authoring software all day long? Indeed, this is a tedious task: it takes time, it is non-productive, it is boring… What if there was a solution to avoid it? Well, this is what web authoring tools are all about. As it is cloud-based, updates are made automatically by your provider, without any efforts from your part.

This feature is even more valuable when your authoring tool is providing you with ready-made content libraries, because they will be up to date automatically.

3. Easy To Publish, Update, And Translate.

When you only have about 30 learners, grouped in one single country and language, it is quite easy to manage it. Nevertheless, how about spreading your learning simulations to ten thousand learners, working all over the globe, in several different languages?

This is not the same, right? Fortunately, web authoring tools expand your possibilities. First off, this will simplify your publishing process. All you have to do is exporting your training simulations at the expected format, and then publish it to your Learning Management System. If, for any reason, you do need to update your training files, simply update the file concerned in your LMS platform to enable your learners to access it.

Finally, chances are that you will need to create your training modules into several languages. To do so, web authoring tools usually have in place translation tools, to manage your modules into several languages.

4. Create Anywhere, Anytime.

Whether you are at work, at home, with a customer, in a colleague's office: you can access your data anytime. Web authoring tools bring on the table this awesome feature. You do not have to choose a dedicated place to work from anymore. Basically, all you need is a computer, a browser, and an internet connection. That is it. Simply fill in your username and password on the login page of your web authoring tool, and you are good to go.

Another great benefit of this feature is that you do not need to worry about using a Mac or a PC anymore. You can use both. This will end this eternal conflict in your workforce.

5. Get Started Right Away.

A great thing about web authoring tools is that it is plug and play. Once you have your license up and running, you can start working instantly, within seconds. No need to install the software. No need to wait hours before the very last patches are up to date. Not to mention that step-by-step guides, as well as F.A.Q and online support are all fully reachable from your web authoring tools interface.

This ensures a great customer experience, and efficiency in the use of your authoring tool.

6. Web Authoring Tools Keep Your Work Safe.

With traditional standalone authoring tools, everything seems safer. Indeed, your work is saved into your computer. You know precisely where files are located. You have this great feeling of getting the situation under control. Well, honestly, this is not the case. Why? Think about it. Image that your hard disk suddenly breaks up. Imagine that someone steals your computer. Imagine that you forget your computer in the train, and that you do not find it back.

Under control, right? Web authoring tools actually secure your work and information drastically. Simply put, all your data is gathered into your provider storage, and he is in charge of it. Chances are he will be really well equipped to fulfill its role. Consequently, you can lose your computer, your hard disk may break up, whatever unfortunate event can occur: your data will be safe. All of this thanks to web authoring tools.

Web Authoring Software

What is it?

A web authoring package is specifically designed to allow you to create web pages and web sites. Examples include Dreamweaver and Microsoft Front Page.

What is it used for?

Web page authoring

How were things done before web authoring?

Web designers would have to write the page using HTML code. Whilst this is very powerful, it is a lot slower than using a web authoring package.

It is also possible to save word processed documents or dtp publications as a web page. However, this is not recommended because they produce large amounts of code which is not required.

- Web authoring software allows you to easily build websites by using the integrated features and user-friendly UI.
- Good web authoring programs will allow you to create stunning websites without any coding skills or programming knowledge whatsoever.
- In order to help you easily create websites, web authoring software options often allow you to use a variety of templates and make good use of their extensive libraries of assets.
- From established names such as Adobe to bold open-source projects, you will discover here the best website builder programs, each filled with unique characteristics and special perks.

Website creation can be a complicated process that requires a certain level of programming knowledge. In most cases, developers tend to use HTML editors or even text editors to create websites.

However, these tools can be a bit complicated to use for average users, so many of them tend to use web authoring software.

HTML editors usually don't have a live preview, so you can't see how your web page looks without opening it in a browser.

In addition, web authoring programs allow you to create a website by simply adding elements to a visual interface.

This makes the website creation similar to editing a Word document since you can add all the necessary elements without writing any code.

Of course, you can always switch to HTML view and make the necessary adjustments using HTML code.

Web authoring software is relatively simple to use and if you're interested in such tools, be sure to check some of the following software.

Features of web authoring applications

WYSIWYG INTERFACE	Web pages are created using Hypertext Markup Language (HTML). Whilst it definitely helps to have some basic knowledge of this, it is not essential when using a web authoring package because there is the option of using a WYSIWYG interface. Generally this works well, but unlike word processing packages, sometimes don't always go exactly where you want them or display as you had planned. In this case, the only solution is to switch to the HTML code and edit that directly. Although many of the features in a web authoring package are similar to those you find in standard office applications, generally you need some technical skill in order to be able to create suitable web pages and use the tools effectively.
Templates	Templates can be set up to ensure that the main features on a web page are the same throughout the site, e.g. the banner, the side menu. Many web authoring packages contain a selection of templates. There are plenty available on the Internet to download for little or no cost. It is also fairly easy to create your own template.
Wizards	Most web authoring software has a number of wizards to help you with things such as setting up the site, specifying the FTP etc
Multimedia	Web pages can contain text, images, animation, videos and sound files.
CGI Web forms	They can also contain interactive CGI forms (Common Gateway

Interface) which enable people to fill in their details using a form on a webpage and then submit the completed form to the website. A web authoring package makes it easy for the user to set up these interactive forms.

Hyperlinks and hotspots

The software also allows makes it possible to set up hyperlinks and hotspots which when clicked will take the user to other pages, other websites or a place within the same page. If a web page is moved to another place in the website by using the web authoring application then any links to that page will automatically be updated.

Site Manager

This is a tool which enables you to view the overall structure of your website and to update links quickly if any pages are moved from within the web authoring software

Using web authoring software to create pages

WYSIWYG interface	Not intuitive to just pick up and use, normally needs some training and/or technical knowledge
Many of the features available in more commonly used office applications	Need a basic understanding of HTML to overcome any formatting problems
Can set up and use templates to ensure consistency between webpages	Doesn't normally come as part of a general office suite and can be quite expensive to purchase.
Can set up hyperlinks and hotspots	
Can insert multi-media into webpages	
The site manager facility can be used to update links within the website	

Using general application software for pages

Very easy to create a web page, no technical knowledge required	If you want to move pages there is no site manager facility to track and update the hyperlinks
Do not need to pay the extra costs of purchasing web authoring software because most computers already have general office packages which are capable of creating web pages	A lot of extra HTML code is created which is unnecessary and can interfere with browser compatibility. This is known as 'bulky code'
The output is compatible with most browsers	May not be able to create all of the features needed on the website, for example CGI forms
Company does not need to employ specialist web design/development staff	It can be difficult/impossible to edit the code - especially if no one understands HTML
Generally faster to produce web pages using standard application software	

What is the best web authoring software

Adobe Dreamweaver

Adobe Dreamweaver is a very powerful and easy to use website creator that provides the full range of features needed in order to make the creation process user-friendly and efficient.

This software's best features include a wide range of site templates, fast coding options, and the capability to showcase your creation in different formats, depending on the devices you choose to preview your work in.

Here are some other notable features found in DreamWeaver:

- Live view editing with instant preview features that allow you to observe how your website looks with just a few clicks
- Multi-monitor support for Windows
- Redesigned user interface for easy access to all the tools and options
- Full Git support

- Bootstrap 4.4.1 integration

Adobe Dreamweaver

The best website creator is more easy to work with than you have imagined – why not give it a go?
Free trial Visit website

Corel Website Creator

If you want to create websites without writing a single line of code, you might want to consider Corel Website Creator.

This application allows you to create interactive and modern websites simply by dragging and dropping the desired elements.

There are also various templates that you can customize any way you want.

The application supports HTML5 video and audio, and you can even add YouTube or Vimeo videos to your website.

In order to speed up website creation, the application relies on CSS3 grid system. There's also support for various effects and gradients so you can create eye-catching designs quickly and with ease.

Corel Website Creator offers support for web fonts, but it also has a SiteStyles feature. Thanks to this feature you can add typographical and graphic elements to your page in a matter of seconds.

If needed, you can also create your own styles and add them to future projects.

There's also a Site Safe feature that creates an automatic backup of your files. Using this feature you can back up your projects to an external hard drive, network drive or a web host.

The application supports a wide range of web development technologies thus allowing you to create interactive websites with ease.

Thanks to the WYSIWYG editor you can see changes in real-time, but you can also inspect your HTML and CSS code if needed.

We have to mention that the application has a simple graphics editor so you can perform image adjustments right from this application.

Xara Web Designer

Creating websites from scratch isn't a simple task, but you can make this task a lot simpler and faster by using Xara Web Designer.

This web authoring program allows you to create visually appealing websites without any coding skills.

Creating a website with Xara Web Designer is as simple as creating a text document.

This web authoring tool gives you the option to create your own designs from scratch or to use the templates included.

There are over 250 website themes and many hundreds of templates for individual elements like buttons and NavBars. It also offers photo editing features, so you won't have to rely on third-party tools.

To create a website you just need to drag and drop elements to the page and the application will generate the code.

Xara Web Designer is a WYSIWYG application, and your website will look identical in both editor and browser.

The application has some useful features and here are their latest updates:

- new advanced text options such as hyphenation and full Open Type support (for creating stand-out typography, great for memorable and eye-catching headings)
- easily add HTML elements to your website
- a lot of new, ultra-editable smart elements including tables, text and photo panels, arrows, and charts
- added support in Premium for the latest animation effects such as parallax scrolling
- the ability to create thumbnails and mouseover effects
- make site-wide changes simply by editing a single element thus speeding up the website creation process
- support for various widgets
- drag and drop MP4, FLV, MP3, or PDF files to your project

Xara Web Designer follows the latest standards, and all your websites will be compatible with all major browsers.

In addition to cross-browser compatibility, your websites are also mobile-friendly, so they will work perfectly on any iOS and Android devices.

This application supports cloud storage so you can easily collaborate with your coworkers.

Xara Web Designer is a powerful tool that you can use to create modern websites without writing a single line of code.

WebSite X5 Evo

Another web authoring software that we want to show you is WebSite X5 Evolution.

This application lets you create your website without writing a single line of code. Instead of writing code, you can create your website simply by dragging and dropping the desired elements.

Recently, the company has released a brand new update of the software.

With the new version, they've made the user interface friendlier and placed great emphasis on revising the templates included within the program.

They've re-designed the entire library of template presets from scratch:

- Enjoy 100 new, mobile-friendly templates.
- Each template is a complete project with graphics, pages, and content: making it even easier to start creating.
- A live preview of each template lets you choose more quickly.

You can create an online store website that supports credit card payments using this tool. Of course, you can also create a static website, blog, or even a guestbook.

All websites are mobile-friendly, so they will work perfectly on any tablet or mobile device.

This tool creates SEO-friendly code so your website will be indexed quickly and without any issues.

You get to see the statistics of your website in the application. This allows you to see the number of visitors, orders, or comments with ease.

WebSite X5 Evolution supports Parallax and other various effects that you can use to enhance the look of your website.

In addition to effects, you can easily import photo and video galleries and the application will automatically generate the required code.

If needed, you can also view the code at any time and make the necessary changes.

The application also has a built-in graphics editor thus allowing you to edit images and customize templates without using third-party solutions.

Lastly, there's also a built-in FTP client so you can easily upload your website online.

HTML Kit

This is a web authoring tool that allows you to create websites by writing your own code.

To make the creation process faster, you can use various toolbar icons to add elements to your page.

The application also offers code helpers and generators that simplify the creation process.

There's also support for revisions and you can revert to older versions of your web page without configuring the server.

Another extremely useful feature is the ability to preview your page in its editor. Preview supports two modes: a full-screen or split-screen preview.

In addition, you can preview your page in another window or on a different monitor.

HTML-Kit also supports preview on multiple browsers and external devices such as smartphones and tablets.

The application comes with a built-in HTML Tidy feature you can use to find and replace invalid markups with ease.

In addition, there's full support for HTML and CSS validators. HTML-Kit also has simple features that can make the development process simpler. For example, you can see the color below each color code with ease.

This tool also supports highlighting matching blocks simply by clicking them. If you want to enhance the functionality of this tool, you can do that by using a wide range of available plugins.

⇒ **Download HTML-Kit**

First Page

Another web authoring software that we have to mention is the First Page. The application has a simple user interface with which you can see a live preview of your page as well as your code.

If needed, you can switch to preview mode to better examine your page.

In addition, you can use Design mode if you're not familiar with HTML. In Design mode, you can edit your page as a standard document without code.

To do that, you can just insert all the necessary elements right from the toolbar menu.

The application has a SmartHistory feature that will help you during coding, remembering all the values and attributes you used so you can easily add them later.

There's a CSS insight feature to help you with CSS code, allowing you to easily position any element or change its properties.

First Page remembers CSS classes, so you can easily assign the desired class to any object. Of course, there's also a syntax highlighting feature that helps you organize your code.

In order to make changes to your page, the application offers a tag selection tool. With it, you can easily view element hierarchy and modify any element with ease.

If you're a webmaster, you'll be happy to hear that this application supports a wide range of webmaster tools.

As a result, you can easily add your website to search engines, check accessibility, validate documents, check the number of links, etc.

The application offers a tag inspection feature that allows you to modify tags with ease.

First Page auto-completes tags as well, a feature that will help you write code while ensuring all your tags are properly closed.

If you're a novice user, we're happy to inform you that this application has a reference guide so you can easily learn more about any tag.

To speed up website creation, the application offers an asset management feature allowing you to manage and reuse your snippets and templates.

To ensure your website follows certain standards and practices, the application offers Tidy HTML Power Tools.

Using these tools, you can perform various actions to ensure your code follows certain standards. The application supports color themes, and you can change the color of your website in a matter of seconds.

First Page comes with its own Popup maker, and there's also an Image Mapper that allows you to create clickable regions on images.

Another useful feature is the ability to check the validity of your links.

The application offers file management tools and allows you to view and add multiple images simply by clicking their thumbnail.

Speaking of images, you can create rollover images with ease, too. It's worth mentioning that the application has a built-in photo album gallery generator.

Thanks to this feature, you can easily create and customize your own photo album.

KompoZer

If you're looking for a free web authoring tool, you might be interested in KompoZer. The application is simple to use, so even basic users will be able to create websites using this app.

KompoZer is based on the Gecko layout engine, so it offers a high performance to its users.

The application offers a simple interface with which you can easily see your code at any time.

In addition, the application supports page previews, and the ability to add new elements to the page without writing any code. Thanks to this feature, website creation is as simple as editing a text document.

The application supports FTP, so you can easily publish your website online. There's also an extended color picker that allows you to adjust the amount of red, green, and blue with ease.

You can set hue, saturation, or brightness right from the color picker.

This tool works with CSS stylesheets and you can see changes from these stylesheets instantly in the preview window.

As for styles, you can easily assign a style to any element simply by right-clicking it from the hierarchical toolbar at the bottom.

KompoZer also has a built-in code cleaner that can remove unnecessary code and validate your page.

By validating your page, you'll ensure that your code is following the latest standards and practices.

We also have to mention that this tool has a built-in spellchecker that will automatically highlight misspelled words.

OpenElement

Another great web authoring software that we want to highlight is openElement.

Offering a great user interface, you can easily create a website from scratch with along with support for layers and reusable styles and elements.

The application offers cross-browser support, support for HTML5 and CSS3 technologies, and support for multi-language responsive websites.

You can also easily integrate your own scripts using this application. There's support for databases and you can even run local web servers on your PC for PHP and database projects.

In order to optimize your code and website for the web, openElement offers support for both image and code optimization.

The application allows you to add any element simply by choosing it from the menu on the right.

There's also a live preview that allows you to freely move your elements on the page and organize them any way you want.

If you're familiar with programming, you can view the source code of your page and edit it as you like.

CoffeeCup HTML Editor

If you're looking for web authoring software with a sleek user interface, you might want to consider Coffeecup HTML Editor.

Besides its visually appealing user interface, the application allows you to create both HTML and CSS files from scratch. If you prefer, you can also use various designs and templates.

This application can work with files stored locally on your PC as well as open and edit files from your web server.

Thanks to the Open from Web feature, you can even open any website from the web and use it as a template.

The Coffeecup HTML Editor also has a useful Components Library feature that you can use to save different elements, allowing you to reuse the same elements on different pages or projects to speed up your work.

In fact, simply by updating the element in the Components Library, you'll update it throughout your project.

The application has a Tag Reference feature and thanks to the Code Completion, you'll automatically get code suggestions as you type.

In order to ensure your code is completely valid, there's a built-in Validation Tool that can check your code.

There's a split-screen preview mode so you can view live preview alongside your code. If needed, you can also display the preview on a different page or even on a different monitor.

The preview is updated in real-time, so you can see even the slightest changes.

Mobirese Free Website Builder

If you're looking for a tool that will allow you to create websites with ease, you might want to consider the Free Website Builder.

This application requires no coding knowledge and can be used to create modern-looking websites with ease, offering a minimalistic and simple user interface so that even the most basic users can use it.

This application can create mobile-friendly websites, so your websites should work on any screen size when complete.

It comes with a set of building blocks; you just need to drag and drop those blocks to your page in order to create a website.

After adding a specific element, you can change its properties like in any text document.

Each element comes with a wide range of options so you can customize your blocks any way you like.

As previously mentioned, the application creates mobile-friendly websites so they should look perfect on any screen.

In addition, you can switch between tablet, mobile, and desktop views to see how your website will look on different screen sizes.

This Free Website Builder has more than 400 blocks available. You can easily add sliders, galleries, articles, accordions, videos, contact forms, lightboxes, and other elements.

All templates are based on Bootstrap 3 and Bootstrap 4 technology, ensuring your website looks perfect on any browser or device.

The application supports FTP, but you can also host your website on Amazon S3, Google Cloud, and Github Pages.

Amaya

Another free web authoring application you need to check out is Amaya. The application works as a web browser, but you can also create your own websites with this tool.

Unlike most applications on our list, you can edit any online website with this tool.

In order to create a website, you just have to choose elements from the right pane and add them to your project. Amaya fully supports CSS and Math ML with the ability to customize all the elements by adding various styles to them.

There's also support for SVG and many SVG-related functions such as transformation, transparency, and animation.

Amaya is a simple web authoring software that offers limited features. The project hasn't been updated in a while, which is why Amaya lacks modern features and interface.

Despite this flaw, it's a decent tool and is available for macOS, Linux, and Windows.

In addition, the application is completely free, so you can use it without any restrictions.

BlueFish

Another free and powerful web authoring software that we would like to mention is Bluefish. This application is released under the GNU GPL license and it's available for most major computer platforms.

The application is lightweight and fast, and it should work without issues on any PC.

Like many other tools on our web authoring software list, this one has a project manager so you can easily switch between different projects.

Bluefish has multi-threaded support for remote files, and it also supports FTP, SFTP, HTTP, HTTPS, WebDAV, and CIFS.

The application has a powerful search and replaces feature that supports regular expressions. In addition, you can perform sub-pattern replacing and even replace files on your hard drive.

It's worth mentioning that you can open multiple files that match a specific pattern using this tool.

Bluefish also has a snippets sidebar thus allowing you to quickly reuse parts of your code. In order to write your code faster, you can assign shortcut key combinations to your snippets.

The application offers in-line reference information for functions and tags as well as code block folding.

In order to organize your code, Bluefish will highlight the start and the end of a selected code block. Of course, there's also an autocomplete feature that will offer coding suggestions and close open tags.

It's worth mentioning that you can fully customize each element's attributes right from the wizard. In addition, there's also image insert dialog and the ability to create thumbnails.

EditPlus

Another web authoring software that we want to show you is EditPlus. The application has a simple user interface and it can work as a Notepad replacement or a web authoring tool.

We have to mention that this tool supports syntax highlighting for HTML, CSS, PHP, ASP, Perl, C/C++, Java, JavaScript, and VBScript.

The application works as a seamless web browser, and you can preview your web page right from this tool. In addition to local files, you can also view any website right from this tool.

EditPlus also has a built-in FTP feature so you can easily upload files over FTP.

In addition, you can also edit remote files directly which can be rather useful. In order to keep your code organized, the application supports code folding and you can easily hide certain lines of code based online indention.

The application has an auto-completion feature, and you can use it to transform abbreviations into a string with ease.

This feature supports Perl and C++ by default, but you can also create your own auto-completion file for other programming languages.

EditPlus also has a Cliptext window thus allowing you to easily access your text clips.

We have to mention that this tool also has a powerful search and replace feature, support for multiple undo and redo steps, and hex viewer.

PSPad

Another application that can help you create websites is PSPad. The application works with various programming languages and also offers syntax highlighting.

In addition to coding, you can use this application to write plain or rich text. PSPad allows you to work on multiple projects and documents at the same time, and you can also save desktop sessions for later use.

There's also a built-in FTP client so you can upload and download your files with ease. In addition, you can also edit files directly from the web.

Since website creation can be tedious at times, you can speed up the process by using macros.

The tool also offers a search and replace feature as well as the ability to see text differences between two files. In addition, the application also has an internal preview so you can see all changes with ease.

In order to ensure that your code is properly formatted, there's a TiDy library available that will check and optimize your code.

Speaking of additional features, the tool also has a TopStyle Lite CSS editor available. The application also has code explorer for many languages such as C++, HTML, and PHP.

UltraEdit

If you're looking for a powerful web authoring software, you might want to consider UltraEdit.

The application has a modern and sleek user interface and you can choose between several available themes. If needed, you can also create unique themes from scratch.

UltraEdit supports multiple selections so you can edit your document on several different locations simultaneously. The application offers an extensive search feature and you can search multiple files with ease.

In addition, you can even use regular expressions in order to customize your search. The application also supports column mode and you can select text along Y-axis which can be rather useful in some cases.

UltraEdit has a built-in FTP feature so you can upload or download your projects or even edit them on the server. There's also support for SSH and Telnet.

The application is perfect for web developers because it supports syntax highlighting for any coding language.

Another useful feature is a file comparison tool that allows you to compare two or three files and see the differences between them.

Organizing multiple projects in UltraEdit is also simple so you can easily manage your files and projects.

The tool also has a function listing feature, and you can easily locate any function, variable, class, or macro right from the list.

If you're not familiar with HTML, you can easily insert HTML elements right from the menu.

Pinegrow Web Editor

Another great web authoring software that we want to show you is Pinegrow Web Editor.

The application offers a sleek user interface so you should be able to create websites with ease.

All elements are available in the right pane, and you can easily move, delete, copy, or edit them. In this regard, the application feels more like Photoshop than a web authoring software.

The application supports live editing so you'll see your changes in real-time. There's no preview option and you can test and edit your page at any time. As a result, website creation feels more natural and streamlined.

It's worth mentioning that this application supports Bootstrap, Foundation, AngularJS, and plain HTML. It's also worth mentioning that all elements will be optimized for mobile devices, so your website should work on any display.

Since many users tend to use tools such as WordPress, Pinegrow Web Editor also allows you to create WordPress themes. Simply add WordPress actions to HTML elements and set the required parameters.

Pinegrow Web Editor lets you create websites by combining different blocks, thus making website creation simple and straightforward. Of course, you can customize every block that you add to the website.

If you're a developer, you can also view the code of your website and adjust it at any time.

As for CSS, you can edit it visually or using code. It's worth mentioning that you can add variables and expressions in order to create custom themes with ease.

Thanks to the Stylesheet manager you can easily clone, attach, and organize your stylesheets. The application also has a Media query helper tool so you can add custom breakpoints with ease.

Another great feature of Pinegrow Web Editor is the ability to edit any online website. Simply enter the website URL and you'll be able to change any element.

This is rather useful if you want to update your website and save changes locally.

NetObjects Website Design

If you're a beginner, NetObjects Website Design software will be perfect for you. The application uses the drag and drop method thus allowing users to create websites in a matter of minutes.

Thanks to an intuitive user interface and built-in wizards you can upload your website online with ease.

It's worth mentioning that this application supports E-Commerce and popular payment processors.

To make website creation easier, the application offers access to an online library of free templates, photos, and styles.

Creating a website is rather simple, and you just need to drag and drop elements into your page and the application will generate the code automatically.

This means that you can create a website without writing a single line of code.

If you're an advanced user, you can view the HTML code at any time and make the required adjustments.

NetObjects Website Design supports HTML5 audio and video, and you can add videos to your website by using drag and drop method.

Speaking of videos, you can also embed YouTube and Vimeo videos with ease.

The application has a custom CSS Framework and Grid System thus allowing you to create a multi-column website easily.

There are even visual aids in the Page View that can help you align your content. The application supports image carousels so you can easily highlight specific pages.

WebEasy Professional

Another web authoring software that we want to show you is WebEasy Professional. The application offers thousands of professional templates, so you can create a website without any programming knowledge.

There's a template gallery available and you can easily browse through the available templates.

In addition to templates, there's also a style gallery so you can add various fonts, colors, links, and backgrounds to your website with ease.

The application also supports Google Maps and you can add a map to your website in a matter of seconds.

The application supports drag and drop method, so you can easily organize all your elements and create unique layouts.

It's worth mentioning that WebEasy Professional supports e-commerce and you can easily add PayPal Shopping Cart to your website.

In addition to an e-commerce website, you can also create a personal blog or create podcasts right from this tool.

WebEasy Professional can create cross-browser and mobile-friendly websites that will work on any device and platform without issues.

There's also support for YouTube videos and social media websites. The application supports online photo albums, but you can also generate a photo album from your own pictures.

⇒ **Download WebEasy Professional**

Creating a website isn't hard if you have the proper tool for it.

Elearning authoring tools examples – at a glance

Don't have time to read through each tool example now? No problem – here's a quick overview:

1. **Elucidat** – cloud-based authoring tool – designed for big employers aiming to drive down the cost of producing business-critical training.

2. **Articulate Rise** – cloud-based authoring tool – part of the Articulate 360 package with the ability to create device-responsive content

3. **Easygenerator** – cloud-based authoring tool – user-friendly, requires no coding, no installation and is fully responsive

4. **Articulate Storyline** – desktop-based authoring tool – solely for Windows PC, PowerPoint style, with high-quality output.

5. **Adobe Captivate** – desktop-based authoring tool – a powerful tool with the ability to create complex interactions, but comes with a steep learning curve.

Content authoring tools come in all shapes and sizes. Some are simple, some complex, some are even included with learning management systems! What it really boils down to when choosing an authoring tool is the **type** of elearning you want to create, what your **learners** are expecting and the **team** behind it.

CHAPTER 2
WORLD-WIDE WEB MODEL

WEB DESIGN

This is a user centered multi disciplinary design pursuit pertaining to planning and production of web sites. It includes influences from visuals arts technology, information structure and networked delivery.

Components of web design

It consists of 5 components:

(i) Content
(ii) Visuals
(iii) Technology
(iv) Delivery
(v) Purpose

Factors to consider while designing a web site

(i) The subject content - Should be relevant
(ii) The coverage of the topic/scope of the resource - Size of page, wide or narrow scope
(iii) Presentation of the information - Should be accurate, should be logical
(iv) Interest of the authority - those responsible for the site, Is the author an expert on the subject?
(v) The objectivity of the site – How balanced or bias is the coverage?
(vi) Presentation – keep in mind the interest of the users
(vii) Currency - availability of the updates - date of publication
(viii) Usability – is information useful

Factors that influence the significant impact of the web design on web marketing efforts

(i) How long it takes to load a web page
(ii) How often visitors visit the site, register or buy products
(iii) How long users use the site
(iv) How users find the site
(v) How likely users revisit the site

Modern web design

1. Web - centered versus designer - centered site design.

One has to consider the designer and user needs hence the site should be developed with the users in mind.

2. Balance of form and function

The site should contain function with form to inspire the user and to break the site boredom. There should be a clear and continuous relationship between form and function. The designer has to make sure that the visual form of the page related to its function.

3. Quality of execution

A web site is considered excellent if it is useful, usable, correct and pleasing. The components of web development have to be compatible e.g. HTML, XML, JavaScript, Java, flash browser, compatibility and server capacity.

4. Conformity of conversion and innovation

This includes how fast the site loads and how attractive the site is.

WEB SITES

- This is a group of web pages that are related and logically connected.
- Web sites can be viewed using software called a web browser eg Internet Explorer, Netscape navigator
- A web site may contain a single web page or many interconnected web pages.

1. Web page

- This is a single web document that is everything you can see on your PC's browser window at one time. Web pages are multiple documents.
- A web page can perform the following:
    - Present information in an interesting way.
    - Provide course material for students
    - Share information globally
    - Direct potential employees to information

2. Entry Page

- This is the first page a visitor views when entering a web site. It is not necessarily the home page.

3. Home Page

- This is the introductory, starting, first, or welcome page for a web page.
- It is where other pages branch off.
- It is the main page of a web site.
- It acts as the introductory page by providing visitors with an overview of the web site and links to the rest of the site.

4. Exit page

- This is the last page a visitor views before leaving the web site.

5. Hypertext

- This is text that contains links to other texts or documents.
- It refers to any word or phrase in an electronic document that can be used as a pointer to a related text page.

6. Hypermedia

- This is a system that have links between text or media that takes users to another web page.
- It contains various type of media hyperlinked to connect a page to other page.

7. Link

- This is a connection made on a piece of text or media that takes users to another web page.
- It is a part of web that can be clicked to get to somewhere else.

8. Hyperlink

- This is a connection or links from one document to another or to any resource or within a document.
- It is the most basic navigational element in a web browser.

9. Broken Link

It is a link that references a page that no longer exists.

10. Content

These are all the words, images and link which a user can read and interact with in a web page.

11. In - Line

- This is a resource of some type, which is placed directly into a document.
- It is always used in an image i.e. inline image.

12. Web design tools

- HTML documents are plain-text (also known as ASCII) files that can be created using any text editor. Eg of text editors include notepad, web-edit, word processors like MS Word, DOS edit, Netscape composer
- Some WYSIWYG editors can also be used eg Front page, Outlook Express, Claris Home page, Adobe PageMill

One can also use graphic tools like photoshop, Paint, Animated GIF construction set, PageMaker etc

13.Web browsers

The piece of software that runs on your computer and allows you to view Web pages. The most common browsers are Netscape and Internet Explorer.

Types of Servers

The Internet is made up of millions of machines, each with a unique IP address. Many of these machines are **server machines**, meaning that they provide services to other machines on the Internet. You have heard of many of these servers: e-mail servers, Web servers, FTP servers, Gopher servers and Telnet servers, to name a few. All of these are provided by server machines.

WEB SERVERS

At its core, a web server serves static content to a web browser by loading a file from a disk and serving it across the network to a user's web browser. This entire exchange is mediated by the browser and server talking to each other using HyperText Transfer Protocol (HTTP)

DNS SERVERS

Since most people have trouble remembering the strings of numbers that make up IP addresses, and because IP addresses sometimes need to change, all servers on the Internet also have human-readable names, called **domain names**. For example, www.howstuffworks.com is a permanent, human-readable name. It is easier for most of us to remember www.howstuffworks.com than it is to remember 209.116.69.66.

The name www.howstuffworks.com actually has three parts:

1. The host name ("www")
2. The domain name ("howstuffworks")
3. The top-level domain name ("com")

Domain names are managed by a company called VeriSign. VeriSign creates the top-level domain names and guarantees that all names within a top-level domain are unique.

A set of servers called domain name servers (DNS) maps the human-readable names to the IP addresses. These servers are simple databases that map names to IP addresses, and they are distributed all over the Internet. Most individual companies, ISPs and universities maintain small name servers to map host names to IP addresses. There are also central name servers that use data supplied by VeriSign to map domain names to IP addresses.

PROXY SERVERS

Proxy servers sit between a client program (typically a web browser) and an external server (typically another server on the web) to filter requests, improve performance, and share connections.

INTERNET ADDRESSING

IP ADDRESS

An IP address is a unique numerical address assigned to every machine on the Internet. The IP address is a 32 bit binary number normally represented as 4 decimal values i.e. octets. Each octet represents 8 bits in range from 0 to 255 separated by decimal points. This method of notation is called **the dotted decimal notation** e.g. 216.27.61.137

To guarantee world-wide unique addresses, IP addresses are licensed from Network Information Center (NIC).

An IP address and its subnet mask perform the following functions:

- Enable the system to process the receipt and transmission of packets.
- They specify the device's local addresses.
- They specify a range of addresses that share the cable with the device.

Commands that can be used to verify IP configuration

Router interfaces must be configured with an IP address if it is to be routed to or from the interface.

PING

This uses ICMP protocol to verify the hardware connection and logical address at the network layer. This command is used to determine the response time of a host.

TRACERT

This command is used to determine the path of a packet.

IPCONFIG

This command is used to determine the IP address of your computer.

DOMAIN NAMES

This is a unique name that identifies an Internet site.

It is an alpha-numeric representation of the IP address. The characters are separated by dots and correspond to an IP address e.g. www.nation.co.ke

IP addresses are not user friendly and could cause typing errors; the domain name system (DNS) was created so people would not have to remember several confusing numbers. Domain names enable short, alphabetical names to be assigned to IP addresses.

They are easier to remember and to work with than an IP address and are informative and convenient to people.

A domain name is divided into two main parts:

(i) First level

It is an extension and is assigned according to what kind of domain it represents

E.g.

Domain name	**Type of domain**
Edu	Educational institution
Gov	Government organization
Mil	Military organization
Net	Network service provider
Com	Commercial organization
Org	Organizations
Au	Australian domain
Uk	United Kingdom domain
Ke	Kenyan domain
Za	South African domain

(ii) Second level

It is a name one chooses or the main host of the Internet.

E.g. www.mail.yahoo.com

Domain name contains the following information

- the name of the organization
- the organization itself
- the country
- the particular computer or network

FQDN
- A Fully Qualified Domain Name is a domain name that includes all higher level domains relevant to the entity named.
- It is usually selected to give a clear indication of the site's organization or sponsoring agent.

DOMAIN NAME SERVICE (DNS)
- This is a hierarchical, distributed method of organizing the name space of the Internet. It translates domain names to IP addresses and vice versa.
- It provides a centralized, distributed database which keeps track of computers names and their corresponding IP addresses.
- DNS servers are computers connected to the Internet host part of the DNS database and allow others to access it.
- DNS servers contains a subset of the entire databases. DNS uses a client/server model where the DNS servers contain information about a portion of the DNS database and makes information available to clients.

How DNS function
- Enter the domain name in the address location
- The browser software will ask Windows for the IP address it maps to
- Windows then sends a request to the local name server(local ISP)
- If the local ISP does not get the request then it forwards it to a higher name server until mapping is done.
- Translation then takes place ie domain name to IP address and vice versa

ROUTING
- A router is a machine that routes packets and keep information used to get data to its destination in routing tables.
- Each router knows about its sub-networks and which IP addresses they use.
- Routers form a tree-like structure on the Internet with Network Service Provider (NSP) backbone at the roots.
- When a packet (piece of data) arrives at a router, the router examines the IP address of its destination then checks its routing table. If the network containing the IP address is found the packet is sent to that network, if not, then the router sends the packet on a default route up the backbone to next router until it finds its destination. This process is called **package routing.**

TYPES OF DOCUMENTS

(1) **Static document**
- Documents are stored as a file on a server
- The same content is delivered every time that URL is accessed.
- Advantages: They are simple, reliable, fast and the documents can be cached locally at a client.
- Disadvantages: Inflexible as content can only be changed by updating the file.
  Information can become boring easily.

(2) **Dynamic documents**
- The documents are created by a program like CGI -script.
- Advantages: Information is timely and always reflect the latest information.
- Disadvantages: They are not reliable.
  Require high cost of executing and maintenance.
  Slow to access

(3) **Active documents**

- These are documents that contain executable elements that are executed by the client on arrival.
- Executable elements are in script language such as JavaScript, Active X, Java applets e.t.c
- Advantages: Documents reflect the latest information.
  Good performance
- Disadvantages: High cost of execution and maintenance. It is complex and poses a great security risks from servers and codes.

- ***Client***: most users of the web simply want to access content. they need an app that can receive incoming content and display it. this kind of app is called a client. (eg. strictly speaking, an email program is considered a client.) many kinds of possible web clients (real player, winamp), but most popular is a web browser (displays html, plus many other common media formats). *client cache*: the location data is temporarily stored while it is displayed in your browser.

- ***Server:*** so where does a web page come from? servers, the host computers that act as storage and distribution centres for web content waiting to be delivered to web clients. a web server is a 24-hour communication application that works something like an automated telephone switchboard. it listens for calls ("requests") placed by people using web browsers asking for web pages. once a request is made by a browser, the server checks to see if it can find the requested page. if it can find the page, the server sends it back to the browser and the browser displays it. if the server can't find the page, or there is some other problem, it sends back an error response in the form of a numeric code. some responses are: "404--the web server can't find the page you asked for", "403--you're not allowed to access the page you asked for without authorization".

  servers:

  - Are actual computers physically hooked up to the internet via ethernet, cable, telephone line, etc.
  - Run software that listens for requests for web content and returns messages and data.
  - Can perform tasks (run programs/scripts, query databases) before responding to clients.
  - Can cease functioning without affecting the stability of the rest of the web/internet.

- ***Protocol:*** the language used by the client and server to negotiate the transfer of data. web: http (hyper-text transfer protocol), file transfer: ftp (file transfer protocol), transmission control protocol/internet protocol (tcp/ip...tcp disassembles data into packets, ip handles addressing and routing of the packets).

Active Server Pages

ASP Introduction

ASP stands for **Active Server Pages**, an open, compile-free application environment for developing Web applications for Microsoft Internet Information Server (IIS) version 3.0 and later.

Active Server Pages (ASP) A web technology invented by Microsoft which enables program code and ACTIVEX objects to be embedded into web pages' to achieve effects, such as pull down lists or tables of database records, not possible in HTML alone. ASP works only with Microsoft's Internet Information Server, and permits scripts to be written in a variety of languages. When a request is made for the URL of their containing page, such scripts run on the server (rather than in the browser as Java applets do) and dynamically generate a page of ordinary HTML that is returned to

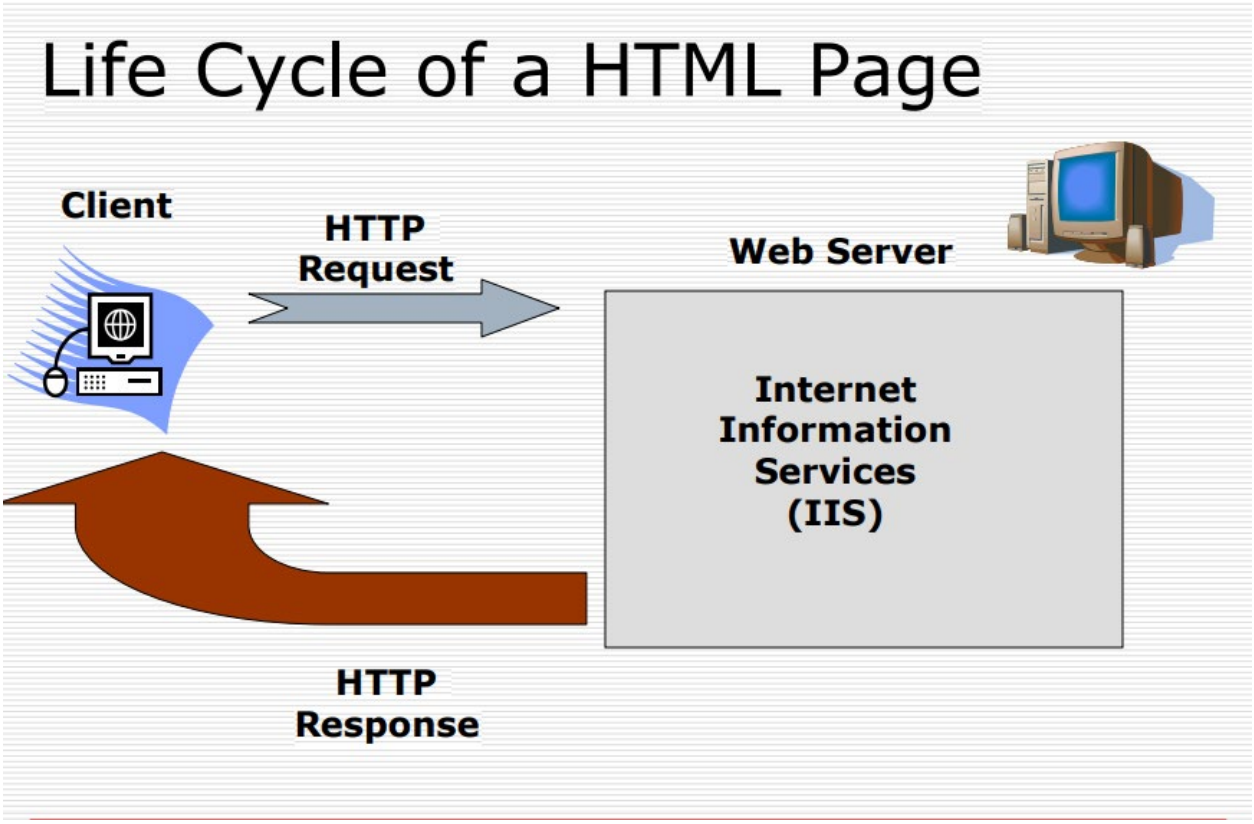
the requesting client.

An ASP file can contain text, HTML tags and scripts. Scripts in an ASP file are executed on the server.

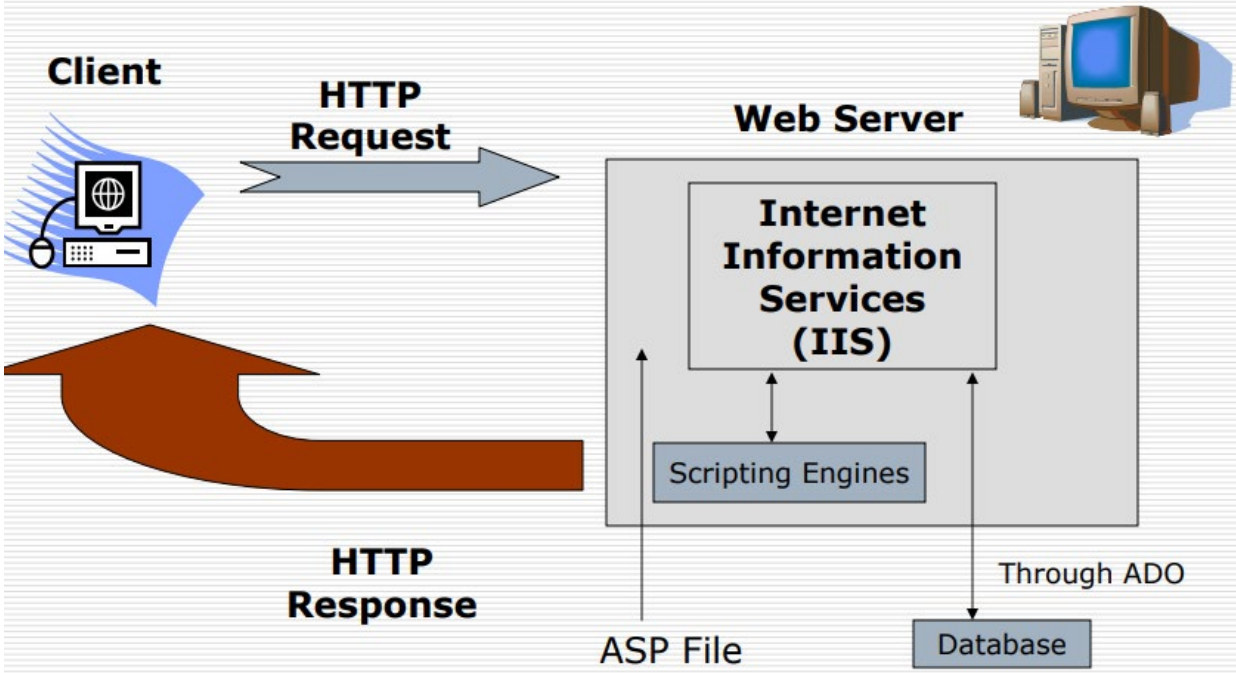
What you should already know

Before you continue you should have some basic understanding of the following:

- HTML / XHTML
- A scripting language like JavaScript or VBScript



Life Cycle of ASP Page



What is Active Server Pages (ASP)?

- As the name suggests, ASP represents pages that are executed on server side.
- ASP stands for Active Server Pages
- ASP is a program that runs inside IIS
- IIS stands for Internet Information Services
- IIS comes as a free component with Windows 2000 ,Windows XP and Windows 2000/2003 server.
- PWS is a smaller - but fully functional - version of IIS
- PWS can be found on your Windows 95/98 CD
- When a client machine requests an ASP page, request is being sent to server. Server processes the request with the help of C:\WINDOWS\system32\inetsrv\asp.dll file and sends back the response to the client machine.

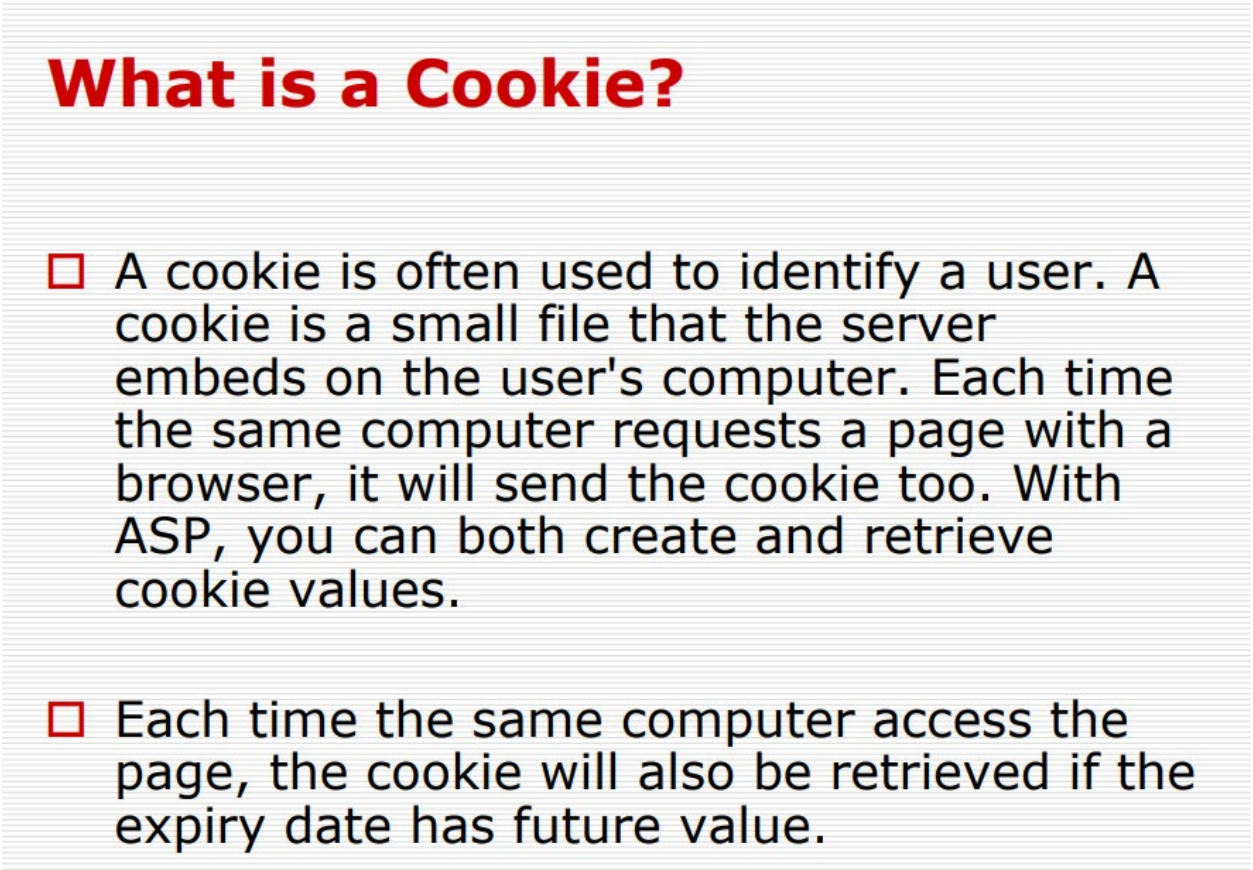
What is an ASP File?

- An ASP file is just the same as an HTML file
- An ASP file can contain text, HTML, XML, and scripts
- Scripts in an ASP file are executed on the server
- An ASP file has the file extension ".asp"

What you can do with ASP?

- Dynamically edit, change or add any content of a Web page
- Respond to user queries or data submitted from HTML forms
- Access any data or databases and return the results to a browser
- Customize a Web page to make it more useful for individual users
- The advantages of using ASP instead of CGI and Perl, are those of simplicity and speed
- Provide security since your ASP code can not be viewed from the browser
- Clever ASP programming can minimize the network traffic

Important: Because the scripts are executed on the server, the browser that displays the ASP file does not need to support scripting at all!



Internet Information Services (IIS)

IIS or Internet Information Services is the service integrated into the Microsoft Windows Server operating system platform that provides support for application-layer Internet protocols.

Internet Information Services general features

IIS is used, most frequently, to host ASP.NET web applications and static websites. It can also be used as an FTP server, host WCF services, and be extended to host web applications built on other platforms such as PHP.

There are built-in authentication options such as Basic, ASP.NET, and Windows auth. *Windows Auth* is useful if you have a Windows Active Directory environment – users can be automatically signed into web applications using their domain account. Other built-in security features include TLS certificate management and binding for enabling HTTPS and SFTP on your sites, request filtering for whitelisting or blacklisting traffic, authorization rules, request logging, and a rich set of FTP-specific security options.

Other features of Internet Information Services

- Fully integrated with Windows NT security and the version of NTFS used in Windows NT
- Full support for version 1.1 of Hypertext Transfer Protocol (HTTP)
- Support for File Transfer Protocol (FTP)
- Support for Simple Mail Transfer Protocol (SMTP)
- Support for Network News Transfer Protocol (NNTP)
- Support for advanced security using the Secure Sockets Layer (SSL) and related protocols

- Provides a platform for deploying scalable Web server applications using Active Server Pages (ASP); Internet Server API (ISAPI); Common Gateway Interface (CGI); Microsoft Visual Basic, Scripting Edition (VBScript); JScript; and other installable scripting languages such as Perl; ASP.NET and PHP (building a PHP site on IIS)
- Allows Web applications to be run as isolated processes in separate memory spaces to prevent one application crash from affecting other applications
- Integrates with Microsoft Transaction Server (MTS) and Microsoft Message Queue (MSMQ) Server for deploying transaction-based Web applications
- Can be managed using the Microsoft Management Console (MMC), through a standard Web browser such as Microsoft Internet Explorer, or by running administrative scripts using the Windows Scripting Host (WSH)
- Includes domain blocking for allowing/granting access on the basis of IP address or domain
- Allows IIS activity to be logged in various formats including IIS, World Wide Web Consortium (W3C), National Computer Security Association (NCSA), and open database connectivity (ODBC) logging
- Allows Web site operators to be assigned for limited administration of each Web site
- Bandwidth throttling to prevent one Web site from monopolizing a server's available bandwidth

More on ASP. Read Here>>http://collegejohelp.blogspot.com/p/aspnet.html

Java Server Pages-JSP

Java Server Pages (JSP) is a server-side programming technology that enables the creation of dynamic, platform-independent method for building Web-based applications. JSP have access to the entire family of Java APIs, including the JDBC API to access enterprise databases. This tutorial will teach you how to use Java Server Pages to develop your web applications in simple and easy steps.

Why to Learn JSP?

JavaServer Pages often serve the same purpose as programs implemented using the **Common Gateway Interface (CGI)**. But JSP offers several advantages in comparison with the CGI.

- Performance is significantly better because JSP allows embedding Dynamic Elements in HTML Pages itself instead of having separate CGI files.
- JSP are always compiled before they are processed by the server unlike CGI/Perl which requires the server to load an interpreter and the target script each time the page is requested.
- JavaServer Pages are built on top of the Java Servlets API, so like Servlets, JSP also has access to all the powerful Enterprise Java APIs, including **JDBC, JNDI, EJB, JAXP,** etc.
- JSP pages can be used in combination with servlets that handle the business logic, the model supported by Java servlet template engines.

Finally, JSP is an integral part of Java EE, a complete platform for enterprise class applications. This means that JSP can play a part in the simplest applications to the most complex and demanding.

Applications of JSP

As mentioned before, JSP is one of the most widely used language over the web. I'm going to list few of them here:

JSP vs. Active Server Pages (ASP)

The advantages of JSP are twofold. First, the dynamic part is written in Java, not Visual Basic or other MS specific language, so it is more powerful and easier to use. Second, it is portable to other operating systems and non-Microsoft Web servers.

JSP vs. Pure Servlets

It is more convenient to write (and to modify!) regular HTML than to have plenty of println statements that generate the HTML.

JSP vs. Server-Side Includes (SSI)

SSI is really only intended for simple inclusions, not for "real" programs that use form data, make database connections, and the like.

JSP vs. JavaScript

JavaScript can generate HTML dynamically on the client but can hardly interact with the web server to perform complex tasks like database access and image processing etc.

JSP vs. Static HTML

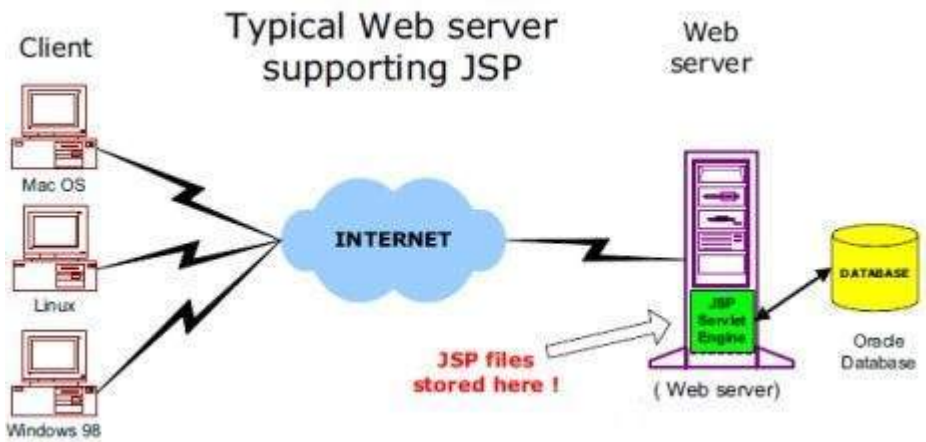
Regular HTML, of course, cannot contain dynamic information.

JSP - Architecture

The web server needs a JSP engine, i.e, a container to process JSP pages. The JSP container is responsible for intercepting requests for JSP pages. This tutorial makes use of Apache which has built-in JSP container to support JSP pages development.

A JSP container works with the Web server to provide the runtime environment and other services a JSP needs. It knows how to understand the special elements that are part of JSPs.

Following diagram shows the position of JSP container and JSP files in a Web application.

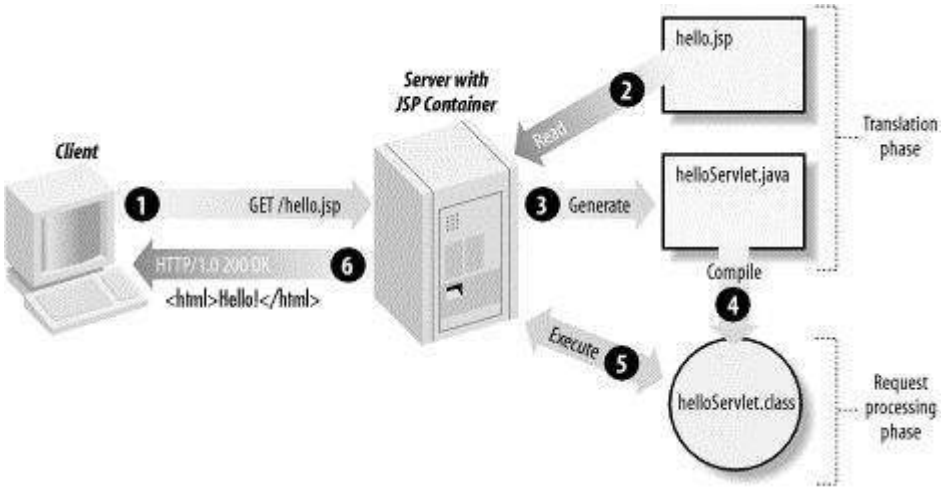


JSP Processing

The following steps explain how the web server creates the Webpage using JSP −

- As with a normal page, your browser sends an HTTP request to the web server.
- The web server recognizes that the HTTP request is for a JSP page and forwards it to a JSP engine. This is done by using the URL or JSP page which ends with **.jsp** instead of **.html**.
- The JSP engine loads the JSP page from disk and converts it into a servlet content. This conversion is very simple in which all template text is converted to println( ) statements and all JSP elements are converted to Java code. This code implements the corresponding dynamic behavior of the page.
- The JSP engine compiles the servlet into an executable class and forwards the original request to a servlet engine.
- A part of the web server called the servlet engine loads the Servlet class and executes it. During execution, the servlet produces an output in HTML format. The output is furthur passed on to the web server by the servlet engine inside an HTTP response.
- The web server forwards the HTTP response to your browser in terms of static HTML content.
- Finally, the web browser handles the dynamically-generated HTML page inside the HTTP response exactly as if it were a static page.

All the above mentioned steps can be seen in the following diagram −



Typically, the JSP engine checks to see whether a servlet for a JSP file already exists and whether the modification date on the JSP is older than the servlet. If the JSP is older than its generated servlet, the JSP container assumes that the JSP hasn't changed and that the generated servlet still matches the JSP's contents. This makes the process more efficient than with the other scripting languages (such as PHP) and therefore faster.

So in a way, a JSP page is really just another way to write a servlet without having to be a Java programming wiz. Except for the translation phase, a JSP page is handled exactly like a regular servlet.

JSP - Lifecycle

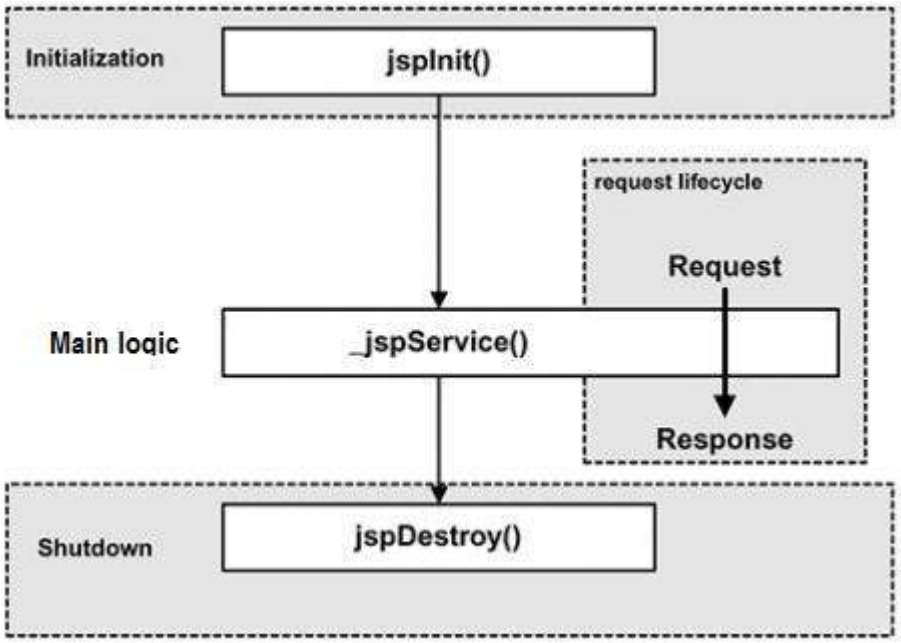
A JSP life cycle is defined as the process from its creation till the destruction. This is similar to a servlet life cycle with an additional step which is required to compile a JSP into servlet.

Paths Followed By JSP

The following are the paths followed by a JSP −

- Compilation
- Initialization
- Execution
- Cleanup

The four major phases of a JSP life cycle are very similar to the Servlet Life Cycle. The four phases have been described below −



JSP Compilation

When a browser asks for a JSP, the JSP engine first checks to see whether it needs to compile the page. If the page has never been compiled, or if the JSP has been modified since it was last compiled, the JSP engine compiles the page.

The compilation process involves three steps −

- Parsing the JSP.
- Turning the JSP into a servlet.
- Compiling the servlet.

JSP Initialization

When a container loads a JSP it invokes the **jspInit()** method before servicing any requests. If you need to perform JSP-specific initialization, override the **jspInit()** method −

```
public void jspInit(){
   // Initialization code...
}
```

Typically, initialization is performed only once and as with the servlet init method, you generally initialize database connections, open files, and create lookup tables in the jspInit method.

JSP Execution

This phase of the JSP life cycle represents all interactions with requests until the JSP is destroyed.

Whenever a browser requests a JSP and the page has been loaded and initialized, the JSP engine invokes the **_jspService()** method in the JSP.

The _jspService() method takes an **HttpServletRequest** and an **HttpServletResponse** as its parameters as follows −

```
void _jspService(HttpServletRequest request, HttpServletResponse
response) {
   // Service handling code...
}
```

The **_jspService()** method of a JSP is invoked on request basis. This is responsible for generating the response for that request and this method is also responsible for generating responses to all seven of the HTTP methods, i.e, **GET, POST, DELETE**, etc.

JSP Cleanup

The destruction phase of the JSP life cycle represents when a JSP is being removed from use by a container.

The **jspDestroy()** method is the JSP equivalent of the destroy method for servlets. Override jspDestroy when you need to perform any cleanup, such as releasing database connections or closing open files.

The jspDestroy() method has the following form −

```
public void jspDestroy() {
   // Your cleanup code goes here.
}
```

JSP - Directives

In this chapter, we will discuss Directives in JSP. These directives provide directions and instructions to the container, telling it how to handle certain aspects of the JSP processing.

A JSP directive affects the overall structure of the servlet class. It usually has the following form −

```
<%@ directive attribute = "value" %>
```

Directives can have a number of attributes which you can list down as key-value pairs and separated by commas.

The blanks between the @ symbol and the directive name, and between the last attribute and the closing %>, are optional.

There are three types of directive tag −

S.No.	Directive & Description
1	**<%@ page ... %>** Defines page-dependent attributes, such as scripting language, error page, and buffering requirements.
2	**<%@ include ... %>** Includes a file during the translation phase.
3	**<%@ taglib ... %>** Declares a tag library, containing custom actions, used in the page

JSP - The page Directive

The **page** directive is used to provide instructions to the container. These instructions pertain to the current JSP page. You may code page directives anywhere in your JSP page. By convention, page directives are coded at the top of the JSP page.

Following is the basic syntax of the page directive −

```
<%@ page attribute = "value" %>
```

You can write the XML equivalent of the above syntax as follows −

```
<jsp:directive.page attribute = "value" />
```

Attributes

Following table lists out the attributes associated with the page directive −

S.No.	Attribute & Purpose
1	**buffer** Specifies a buffering model for the output stream.
2	**autoFlush** Controls the behavior of the servlet output buffer.

3	**contentType** Defines the character encoding scheme.
4	**errorPage** Defines the URL of another JSP that reports on Java unchecked runtime exceptions.
5	**isErrorPage** Indicates if this JSP page is a URL specified by another JSP page's errorPage attribute.
6	**extends** Specifies a superclass that the generated servlet must extend.
7	**import** Specifies a list of packages or classes for use in the JSP as the Java import statement does for Java classes.
8	**info** Defines a string that can be accessed with the servlet's **getServletInfo()** method.
9	**isThreadSafe** Defines the threading model for the generated servlet.
10	**language** Defines the programming language used in the JSP page.
11	**session** Specifies whether or not the JSP page participates in HTTP sessions
12	**isELIgnored** Specifies whether or not the EL expression within the JSP page will be ignored.
13	**isScriptingEnabled**

	Determines if the scripting elements are allowed for use.

Check for more details related to all the above attributes at Page Directive.

The include Directive

The **include** directive is used to include a file during the translation phase. This directive tells the container to merge the content of other external files with the current JSP during the translation phase. You may code the ***include*** directives anywhere in your JSP page.

The general usage form of this directive is as follows −

```
<%@ include file = "relative url" >
```

The filename in the include directive is actually a relative URL. If you just specify a filename with no associated path, the JSP compiler assumes that the file is in the same directory as your JSP.

You can write the XML equivalent of the above syntax as follows −

```
<jsp:directive.include file = "relative url" />
```

For more details related to include directive, check the Include Directive.

The taglib Directive

The JavaServer Pages API allow you to define custom JSP tags that look like HTML or XML tags and a tag library is a set of user-defined tags that implement custom behavior.

The **taglib** directive declares that your JSP page uses a set of custom tags, identifies the location of the library, and provides means for identifying the custom tags in your JSP page.

The taglib directive follows the syntax given below −

```
<%@ taglib uri="uri" prefix = "prefixOfTag" >
```

Here, the **uri** attribute value resolves to a location the container understands and the **prefix** attribute informs a container what bits of markup are custom actions.

You can write the XML equivalent of the above syntax as follows −

```
<jsp:directive.taglib uri = "uri" prefix = "prefixOfTag" />
```

More on JSP. Read Here>>> https://www.tutorialspoint.com/jsp/index.htm

CHAPTER 3
OVERVIEW OF HTML

HTML CODING

- HTML stands for HyperText Markup Language.
- Hyper means active.
- It represents textual and image content.
- It is platform independent. This means that the text and the content are encoded in a way that they can be displayed on a wide range of computers.
- Pages are made of text, images and URL links.
- HTML is structured i.e. it has a beginning, body and end.
- HTML is composed of **tags** which are always enclosed in angle brackets <>
- Tags in HTML are not case sensitive.
- In HTML there are two types of tags, **container** tags, and **empty** tags.
- Container tags occur in pairs. An example of a container tag is the <title></title> tag. Whatever is contained within this tag is assigned to the title. Notice that the closing tag has a slash in it.
- Empty tags require no closing tag. An example of an empty tag is the break tag
. This forces the cursor to a new line.
- Tags should always be balanced hence containers should be nested within each other.

Advantages of HTML

1. It can be written in any editor.
2. It is universal and simple to learn and implement.
3. It gives an opportunity to further explore and add more features.

HTML document Structure

- HTML files usually have the extension of htm, html, or shtml.
- Document tags define the overall structure of an HTML document.
- HTML tags are used to mark elements of a file for your browser.
- An element is a fundamental component of the structure of a text document. Some examples of elements are heads, tables, paragraphs, and lists.
- Some elements may include an **attribute**, which is additional information that is included inside the start tag.
- There are four tags every HTML document should have. These tags define what type of document it is, and the major sections.
- These tags are <HTML>, <HEAD>, <TITLE>, and <BODY ...>.

e.g.

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0 Transitional//EN">

<HTML>

<HEAD>

<TITLE>This is my first web page</TITLE>

</HEAD>

<BODY>

<P>Hello world!</P>

</BODY>

</HTML>
```

- **COMMENT TAG**
  The first line of the code usually starts with !. It is usually for commenting and it is an empty tag. Comment tags do not show up in the browser window. One can tell your web browser what version of HTML being used. This needs to be written exactly as is. This, unlike the rest of the HTML language is case sensitive. You can write all the other tags in upper or lower case.

- **<HTML> </HTML>**
  This is now the beginning of the document. It tells the browser that the file contains HTML coded information. The file extension .html indicates that the file is a HTML document.
- **<HEAD> </HEAD>**
  This identifies the first part of the HTML - coded document that contains the title. The title is shown as part of the browser's window.
- **<TITLE> </TITLE>**
  The title element contains the document title and identifies its content in a global context. The title is typically displayed in the title bar at the top of the browser window, but not inside the window itself. The title is also what is used to identify the page for search engines and also what is displayed on a bookmark list. Titles should be descriptive, unique and relatively short.
- **<BODY> </BODY>**
  The body tag defines the look of the page as a whole – specifically global settings for the color of the text, the color of the background and the color of the links.
  This is the second and the largest part of the HTML document.
  The Body section of HTML contains other tags, which display text, images, links and multimedia.

  Body Tag Attributes

  BGCOLOR
  Defines the background color of the page. The color setting can be expressed in one or two ways, either by name e.g. " blue" or as a six digit hexadecimal number e.g. Blue – 0000ff, Green – 00ff00, Red – ff0000, White – ffffff, Black – 000000 e.t.c

  BACKGROUND
  Defines a background image. The images get tiled in the browser.

  TEXT
  Defines the color of the text of the page. NB/ Make sure your background contrasts with the text color.

  LINK
  Defines the color of hyperlinks which have yet to be selected.

  ALINK
  Defines the color of hyperlinks as they are being clicked.

  VLINK
  Defines the color of hyperlinks which have already been visited.

  BGPROPERTIES
  Only available in some modern browsers which "watermarks" the page, fixing any image specified with the BACKGROUND tag so it does not move if a user scrolls up and down a HTML page.

  e.g.

```
<HTML>
<HEAD>
<TITLE>Welcome to my page </TITLE>
</HEAD>

<BODY BGCOLOR="Blue" TEXT="White" LINK= "Green"
 ALINK= "Lightgreen"  VLINK= "DarkGreen">

The bulk of the page goes here
```

```
</BODY>
</HTML>
```

OTHER TAGS

HEADINGS

Html has six levels of headings numbered H1 to H6 with H1 being the largest. Headings are typically displayed in larger and/or bolder fonts than normal body text.

```
<HTML>
<HEAD>
<TITLE>Welcome to my page </TITLE>
</HEAD>

<BODY BGCOLOR=”Blue”  TEXT=”White” >
<H1> This is Heading 1 </H1>
<H2> This is heading 2 </H2>

The bulk of the page goes here

</BODY>
</HTML>
```

PARAGRAPH

The amount of spaces and carriage returns are automatically compressed into a single space when the HTML document is displayed in a browser.
Hence, the paragraph tag ,<P> </P> is used.

Attributes of Paragraph tag

<P ALIGN= CENTER></P>
<P ALIGN= RIGHT></P>
<P ALIGN=LEFT></P> is the default alignment i.e. if the align attribute is not included, the paragraph will be left aligned.

LINE BREAK

When your HTML document is viewed, normally the text will do a word-wrap at the end of a line. Using the
 tag forces a line break with no extra space between lines. This
 tag has no closing tag.

HORIZONTAL RULE

The <HR> tag produces a horizontal line the width of the browser window. A horizontal rule is useful to separate major sections of your document.

Attributes

WIDTH

The width of the rule can be expressed in two way: as a number or a as a percentage.
e.g <HR WIDTH=500> or <HR WIDTH = 75%>

SIZE

Allows the designer to specify how high, in pixels, the line will be.

NOSHADE

By default horizontal rules come with a 3D look. By using NOSHADE the line is displayed without the drop shadow that would normally accompany the basic line.

e.g.

<HR SIZE= 4 WIDTH="50%" NOSHADE>

CHAPTER 4
HTML TEXT

Html has two types of styles for individual words or sentences.
(i) Logical styles
(ii) Physical styles

Logical styles
These styles tag text according to its meaning.
These tags do not directly specify the type of highlighting they will employ.
- The advantage of this approach is that it reduces repetition of modification of text e.g. when you want to change the appearance of level one heading from 24-point times centered to 30 Helvetica right aligned, all one has to do is change the definition of level one in the web browser.
- Another advantage of logical styles is that they enforce consistency.

Examples of logical styles

<DFN>	For a word being defined, Typically displays the words in Italics.
<EM>	For emphasis. Typically displays the words in Italics.
<CITE>	For titles of books e.t.c. Typically displays words in Italics
<CODE>	For computer code. Displayed in a fixed width font.
<KBD>	For user keyboard entry. Displays words in plain text width font
<SAMP>	For a sequence of literal characters. Displayed in a fixed width font
<STRONG>	For strong emphasis. Typically displays words in bold.
<VAR>	For a variable, where you will replace the variable with specific information. Typically displays words in Italics.
<ADDRESS>	Displays a block of text in Italics and offsets it to a new line

Physical styles
They offer consistency in that something you tag a certain way will always be displayed that way for readers of your document.

Examples of physical styles

<B>	Bold text
<I>	Italic text
<TT>	Typewriter text e.g. fixed-width font
<U>	Underline text
<Strike>	Strikethrough text
<blink>	Causes text to blink
<Basefont>	Used to specify the overall font for your page. It is added immediately after the <body> tag. It has a face, size and color attributes. It has no closing tag. E.g. <body> <basefont face =”arial, verdana, courier” size=”5” color =”red”> Hello this is my page. This is text </body>
<Font>	It has face, size and color attributes. **Font size:** Used to set the size of the font from 1(smallest) to 7(largest) with size 3 being the normal text. Format- font size 6 The other method for using font tag is provided by relative size change from basefont size i.e. <basefont size=”5”>size five size six size four **Font face:** Used to specify the desired font typeface. The faces selected should be standard. The browser uses the first font in the list available on

	the visitor's computer. e.g. <body> Some text here **Attributes** ** ** This text is left aligned** ** **<div align=right">** This text is right aligned** **
Superscript **^{…..}**	The text is raised e.g. km³ would be km^3
Subscript **_{…..}**	The text is lowered e.g. H ₂O would be H_2O
Text justification/ Alignment	The text is justified according to the align attribute indicated to align the text in the page layout. The justification tag is …….
Blockquote <blockquote>……</ blockquote>	Forces a paragraph break above and below the text. Used to include lengthy quotations in separate block on the screen. Most browsers generally change the margins for the quotation to separate it from the surrounding text.
Preformatted text <PRE>….</PRE>	Used to generate text exactly as typed in including spaces, new lines and tabs. Useful for program listings. e.g. <pre> mangoes sh 5 Oranges sh 10 </pre>

Note: Most websites stick to fonts like Times Roman and Arial, because most computers have these fonts installed. If you were to use an elaborate font and your viewers didn't have that font, it will revert to Sans-Serif or Helvetica, as default.

SPECIAL CHARACTERS

The ASCII characters <,> and & have special meanings in HTML therefore, they cannot be used in text. The angle brackets are used to indicate the beginning and end of tags while the ampersand sign is used to indicate the beginning of an escape sequence.

& lt	Is used to display < less than
&gt	Is used to display > greater than
&amp	Is used to display & ampersand
&copy	Is used to display © copyright
&reg	Is used to display ® trademark
&quot	Is used to display " quote
&plusmn	Is used to display /- plus or minus
&not	Is used to display - negative
&deg	Is used to display o degree sign
&pound	Is used to display pound sign
&micro	Is used to display micron
&para	Is used to display paragraph mark
&yen	Is used to display yen sign
&sect	Is used to display § section
&cent	Is used to display cent
&raquo	Is used to display double greater that
&laquo	Is used to display double less than
&middot	Is used to display midline dot
	Is used to display space character It is used to add extra spaces.
&ouml	Is used to display a lower case o with an umlaut:

&ntilde	Is used to display a lower case n with a tilde:
&Egrave	Is used to display an uppercase E with a grave accent:

NB/ Unlike the rest of HTML, the escape sequences are case sensitive.

MARQUEE ELEMENT

This is a tag that creates a scrolling text.
i.e. <marquee>…….</marquee>
It can not be nested and it must have an ending tag.

Attributes

Marquee Align= top/middle/bottom
This align the marquee with the top, middle or bottom of neighboring text line.

Marquee behavior = scroll/ slide / alternate
This specifies the text behavior.

Marquee bgcolor = red
Sets the background color of marquee.

Marquee direction = left/right
This defines the direction in which the text scrolls

Marquee loop= number/infinitive
This specifies the number of loops as a number value or infinite.

Marquee scrollamount = number
Sets the number of pixels to move the content for each scroll movement

Marquee scrolldelay = number
Specifies the delay in milliseconds between successive movement of the marquee content.

Marquee Hspace = number
Sets the amount of space to clear left or right of the marquee.

Marquee Vspace= number
Sets the amount of space to clear above or below the marquee.
e.g. <marquee behavior= alternate bgcolor= "white" hspace= 2 vspace="4" >Some text here </marquee>

CHAPTER 5
HTML LISTS

Lists are often used to present information in an easy to read format. It can also be used to indent information. Lists can be numbered, unnumbered or definition lists. Lists tags can be nested.
There are three types of lists:

- Unordered/ Unumbered Lists (Bullets)
- Ordered/ Numbered Lists (Numbers)
- Definition Lists (no numbers or bullets)

Unordered Lists

These are bulleted lists i.e. a list of items is preceded by bullets or markers. It is a single item list. The list begins and ends with this tag.
To make a bulleted list:

- Start with an opening list<UL> (for unnumbered list) tag
- Enter the <LI> (list item) tag followed by the individual item; no closing</LI> tag is needed.
- End the entire list with a closing list </UL> tag.

The <LI> items can contain multiple paragraphs. Indicate the paragraphs with the <P> paragraph tags.
e.g.

```
<UL>
<LI>Monday
<LI>Tuesday
<LI>Wednesday
</UL>
```

Attributes

Type: This is used to set different kind of bullet character. The options are:

- disc <UL type = square>
- circle
- square

Ordered Lists

A numbered list is identical to the unnumbered list only you use <OL> to number the list

```
<OL>
<LI>Monday
<LI>Tuesday
<LI>Wednesday
</OL>
```

Attributes

Type: This is used to set different kind of numbered lists.

E.g.

Plain number	-	<OL type =1>
Capital Letter	-	<OL type = A>
Small Letter	-	<OL type = a>
Capital Roman numbers	-	<OL type = I>
Small Roman numbers	-	<OL type = I>

Value: This is used to change the number within a list and is used as part of LI command.

```
<OL>
<LI value ="7">Item 7
<LI>Item 2
</OL>
```

Definition Lists

It consists of alternating a definition term<DT> and a definition <DD>. Web browsers generally format the definition on a new line and indent it.
The <DT> and <DD> entries can contain multiple paragraphs.
e.g.
<DL>
<DT> Alligator
<DD>A large reptile with very sharp teeth.
<DT>Alliance
<DD>A union, relationship
</DL>

The **COMPACT** attribute can be used routinely in case your definition terms are very short.
e.g.
<DL COMPACT>
<DT>ctrl s
<DD>Short cut for saving in Windows
</DL>

Nested Lists
Lists can be nested. You can also have a number of paragraphs, each containing a nested list, in a single item.
e.g.
<UL>
<LI> A few provinces of Kenya:
    <UL>
    <LI>Nairobi
    <LI>Coast
    <LI>Western

<LI> Two towns in Central Province
    <UL>
    <LI>Nyeri
    <LI>Kiambu
    <LI>Muranga
</UL>

CHAPTER 6

HTML LINKS

The chief power of HTML comes from its ability to link text and/or an image to another document or section of a document. A browser highlights the identified text or image with color and/or underlines to indicate that it is hypertext link.
Links are used to:
- Jump from section to section within the same web page.(Internal)
- Link to different page within your web site. (Local)
- Link to another web page in another web site. (Global)
- Link to a graphical image

- Link to documents in other directories.

Relative pathnames
You can link to documents in other directories by specifying the relative path from the current document to the linked document. Relative links are useful because:
1. It is easier to move a group of documents to another location (because the relative path names will still be valid.
2. It is more efficient connecting to the server
3. There is less to type

Ways of providing links
- Clicking on a word, phrase r text
- Clicking on a button
- Clicking on an image

HTML uses <A> tag which stands for anchor.
To include an anchor in your document:
- Start the anchor with <A *(include a space after A)*
- Specify the document you're linking to by entering the parameter HREF="filename" followed by a closing right bracket >
- Enter the text that will serve as the hypertext link in the current document.
- Enter the ending anchor tag. </A>

e.g.
<A HREF ="C:/practice.html> This is practice document</A>

URLs
This is the address or location of the link.
The World Wide Web uses Uniform Resource Locators (URLs) to specify the location of files on other servers. A URL includes the type of resource being accessed (e.g. web, gopher, ftp), the address of the server and the location of the file. The syntax is:

Scheme://host.domain [:port]/path/filename

Where scheme is one of the following:

File	a file on your local system
ftp	a file on an anonymous FTP server
http	a file on a world wide web server
gopher	a file on a gopher server
WAIS	a file on a WAIS server
News	a Usenet newsgroup
telnet	a connection to a Telnet-based service.

NB/ Unless otherwise the port number can be omitted.

Attributes
Href
- Stands for hypertext reference.
- This is the location of the file or section of page that is referenced.

Color
The general color of the text links is specified in the <body> tag
i.e. <body **link**="ff00ff" **vlink**=#808080" **alink**="#ff0000"
where **link** is the color of the link that has not been visited, **vlink** is the color of the link that has been visited and **alink** is the color displayed when the mouse hovers over the text.

- One can also define colors for individual links on the page. There are two methods:
  (a) Placing font tags between <a href> and e.g.
  Click here to go to w3.
  This works for most browsers except IE 3.0
  (b) Using a style setting in the <a> tag e.g.
  click here to go to w3.
  This works for all browsers.

Target

The target is used if you want the link to open in another window or frame than where the link itself is placed.

The format is <a href="url" target =" ">

The predetermined targets are:

- Blank : loads the page into a new browser window
- Self : loads the page into the current widow
- Parent : loads the page into the frame that is superior to the frame the hyperlink is in.
- Top : Cancels all frames and loads in full browser window

E.g.

 here

Name attribute

- This is used to set up named anchors.
- You can invisibly mark certain points of a document as places that can be jumped directly instead of loading the document.
- The value of the Href attribute value of name attribute must be enclose with quotation marks.
- The anchor should have either name or Href attribute but not both.
- The anchor can not be nested.

Links to sections of a page

Links within a page

Anchors can be used to move a reader to a particular section in a document. To create the links, you require two items:

(a) HREF attribute
(b) NAME attribute

e.g.

 word

linkname is the name of the section that you are linking to

The # symbol instructs the browser to look through the HTML document for a named anchor.

A **named** anchor is a hidden reference marker for a particular section of the same page. It is also used to mark a section of another page.

e.g.

```
<body>
<a name ="top"></a>
.
.
<a href ="top">TOP</a>
</body>
```

or

```
<a href ="#linkname"> word</a>
.
.
<A name ="linkname">about some text here</a>
```

Links between sections of different documents

The HTML code for linking to a named anchor in another local HTML document is as follows:
Suppose you want to link from documentA.html to documentB.html
In documentA.html:

 Text to activate link

In documentB.html:

Text that responds to the link

Linking to another page anywhere in the Internet

This is used to create a link to a page in the Internet and the format is:
Yahoo page

Mailto

- Used to hyperlink email addresses, but this scheme is unique in that it uses only a colon (:) instead of :// between the scheme and the address.
- You can make it easy for a reader to send electronic mail to a specific person or mail alias by including the mailto attribute in a hyperlink. The format is:
  <A HREF ="mailto:emailinfo@host">Name
  e.g.
  <A HREF = " mailto:danil2000@yahoo.com"> Send me mail

- Mailto allows users to send emails by clicking on the hyperlink and a mail windo appears.
- One can also set the **subject,cc and bcc** to be part of the the mailto link:
  e.g. Greetings
  <a href "mailto:danil2000@yahoo.com?cc=person@hotmail.com>Greetings

NB/ It is important to note that UNIX is case-sensitive operating system where filenames are concerned.

CHAPTER 7

HTML IMAGES

Images are also called pictures, graphics, icons, cliparts.
Most web browsers support two inline image formats ie GIF and JPEG

GIF (Graphic Interchange Formats)

- All the graphical browsers use it for in line images.
- It compresses the picture information and translates it to binary codes that can be sent over the Internet. It is, however, not good in compressing photographs.
- It is excellent for banners, buttons and cliparts.
- It is limited to a maximum of 256 colors for any image.
- It has a feature of defining a color to be transparent.

JPEG or JPG (Joint Photographic Expert Group)

- It can support any number of colors.
- It takes significantly longer than GIF image to decompress and display. It has good image quality though it occupies larger file size.
- JPEG images do not have the ability to have transparency.

Factors to consider when using graphics in web pages.

- Large and numerous images may look great on high-end computers but frustrate users who might wait for images to load.
  Keep images no wider than 480 pixels and no higher than 300 pixels to avoid users to scroll or resize their web browser window.
  A single image can appear several times in a web page with little time delay.
- Image should add meaning to the page.
- Images can also be linked as external images. Using same image in several web pages will load them very fast.

HTML TAG FOR IMAGE

The basic HTML format for an inline image tag is:

```
<IMG SRC = “filename.gif”>
```

The first part of the tag tells the browser to expect an image, the second part specifies the location of the image
<IMG> is an empty tag hence has no closing tag.

Attributes

Size

- This scales the image and sets the appropriate space (in pixels) as it downloads the image.
  < IMG SRC =”filem\name.gif” HEIGHT=100 WIDTH = 100>

Hspace and Vspace

- This tag allows the designer to put space between the edge of the image and the surrounding text.
- They both take numeric values which specify the amount of horizontal and vertical space surrounding any image they are applied to. This value is specified in pixels.
  <IMG SRC = “filename.gif” VSPACE=”10” HSPACE =”20”>

Alignment

- By default an image is left aligned.
- This attributes controls text around the graphic. The align attribute can take the following values: CENTER, RIGHT, LEFT, TOP, and BOTTOM.

<IMG SRC =”filename.gif” ALIGN=RIGHT>

Border

- This is used to place or eliminate a border round the image.
- Border widths are measured in pixels.
  <IMG SRC =”filename.gif” BORDER=”0”>

Alt attribute

- This is used to define an alternate text for the image.
- It improves the display and usefulness for people using text only browsers or when image autoload is off.
- It is used to give the user more information about the image.
  <IMG SRC=”filename.gif” alt =”picture of a lion”>

Background Graphics

- Newer versions of web browsers can load an image and use it as a backgroundwhen displaying a page.
- Background images can be a texture or an image of a logo.
- Using a feature called tiling, a browser takes the image and repeats it across and down to fill your browser window.
  <BODY BACKGROUND = “filename.gif”>

Images as Hyperlinks

- Inline images can be used as hyperlinks just like plain text. Ie. They can be used as connectors to other web pages

  <A HREF =”index.html”><IMG SRC =”filename.gif BORDER=”50” ALT=”Picture of a cat”></A>

- Images can also link to other images forming hyperlinked images.
  <A HREF=”picture.gif”><IMG SRC=”hyper.gif” BORDER =”0”>
  hyper.gif acts s hyperlink to picture.gif

IMAGE MAPS

- This is an active click able image that sends visitors to different web pages depending on different parts of the image clicked. These are called hotspots.
- Using an image one can tell the browser that particular sections of the graphic, when clicked, will trigger a jump to a particular page.
- The image is divided into regions or areas with each area linked to a different URL.
- An image map has three components:
  - An image
  - A set of map data
  - A HTML host entry
- An image map is a normal web page typically stored in GIF or JPEG format.
- Map data is a set of description of the mapped regions within the image.
- There are two types of image map which can be used in a web page:
  - **Server-side image mapping**: It is the older technique and is compatible with all browsers. It relies on information being passed on back to the server when a visitor clicks on a map. The information is translated and the relevant link is passed back and acted upon by the browser.
    The process is interpreted by the web server when the visitor clicks on the image, the browser program transfer the coordinates of the click to a program running on the web server to be processed.
    This program then examines the map data and coordinates to determine the link then the URL or coordinates associated with those that well clicked on is sent back to the client’s browser and page is loaded.

How server side maps are stored depends on software installed on the web server.
This program is CGI script which examines the map data and determines the link. CGI is a standard method for a web server to interact with programs running on a client browsers.

Common Gateway Interface
The two most common server-side map standards ARE:

- NCSA
- CERN

- Both of these standards requires the map data to be stored in a text file separate from the HTML document usually with a .**map** extension to file name.

Merit
- It is supported by all browsers.

Demerits
- It takes a lot of time to load.
- It requires more system and programs e.g. CGI on your web server.
- Status bar shows only coordinates flying by along with the name of the image mp file include URL.

- **Client-side image mapping:**

A client side image map is made up of two elements: the graphical map and the code which tells the browser how to operate it.
The code itself requires two elements, the <IMG> tag which will display the map and the <MAP> tag which provides instructions on what it will do.
With a client-side map, all of the data required to operate the selection procedure is included with the HTML page, cutting out the need for streams of communications with the server.
Client-side maps data is stored in HTML files and embedded directly into a page containing other HTML elements and are interpreted by the web browser program.
It works independently of your web server hence interpreted by the visitors' web browser.
The web data are embedded in the host page and when the client clicks on the image, the browser processes without interaction with the server.

The <IMG> tag takes all the attributes including two extra ones:

- ISMAP which tells the browser this is an image map
- and USEMAP which contains an anchor link to the map definition data. USEMAP assigns a name to the image which must be unique. The value of the name is case sensitive.

e.g.
<IMG SRC ="mymap.gif" WIDTH =100 HEIGHT =100 ISMAP USEMAP ="#mapdata">
The map data sits inside the <MAP> container and looks like:

<MAP NAME ="mapdata">
<AREA ….>
<AREA ….>
<AREA ….>
</MAP>
The map definition contains a number of AREA definitions, each of which sets up areas of the graphic map which will trigger active links when the mouse pointer moves over them..

Merits
No interaction with web servers is required to determine what URL to use.
It is very fast to load and process.

The status bar at the bottom of browser's window tells you the URL or web page you would be loading.

Demerits
Older browsers do not support it.

The attributes of AREA tag
SHAPE
This defines the different shapes, regions or hotspots.
Can be a standard shape i.e RECT, CIRCLE, POLY or DEFAULT that allows the designer to specify an action for instances where none of the user-defined shapes have been selected.

RECT
Two pairs of coordinates are needed to specify a rectangle. The first pair specifies the top left corner of the rectangle while the second pair specifies the bottom right corner of the rectangle.
e.g.
<AREA SHAPE ="RECT" COORDS="100,100,200,200"
Href=http://www.apage.co.uk">

CIRCLE
This is defined by its center and its distance from the center to any point on the circle.
<AREA SHAPE ="CIRCLE" COORDS ="67,97,34">

67,97 is where the center is located and its radius is 34.

POLYGON
This is a figure that has got any number of sides. Polygon is built by a list of adjacent vertices.
E.g.
<AREA SHAPE ="POLY" COORDS =16,13,35,62,72,27,16,13>

COORDS
Specifies sets of X and Y coordinates within the map image that will act as hotspots. When the mouse moves over a hotspot the browser detects that a link has been activated. If a mouse button is pressed the browser will act in the same way as if a hyperlink has been selected.

HREF
Contains the URL the browser will jump to when the hotspot is clicked by the page visitor.

Advantages of using image maps
- Image maps provide an alternative to loading a page containing several linked pages.
- Image maps can provide an alternative to a cluttered page.
- Image map allows you to use a single image to provide links to a number of different URLs.

CHAPTER 8
HTML BACKGROUNDS
Modifying the background involves including attributes in the <BODY> tag:

BGCOLOR
It adds solid background color.
<BODY BGCOLOR= red>
TEXT
Is used to add color of normal body text.

LINK
Adds the color for hypertext links items.
VLINK - This attribute adds color for the recently visited link.(Visited link)
ALINK - This attribute adds color of the link that is currently being selected. (Active link)

BACKGROUND
This attribute overrides the BGCOLOR attribute since wallpaper goes on top of the background color.
Before including the background image you must consider the following point:

- File size
- Effect of texture
- Readability of the text

One can combine two or more attributes
e.g
<BODY BGCOLOR ="#FFFFFF" BACKGROUND ="bg.gif" TEXT = #000000 Link ="#0000FF" VLINK ="#660099">

BACKGROUND SOUND
Sound for the web comes in two main formats, digital audio or WAV and synthesized music of MIDI format. Both are compatible with all browsers.

WAV is a very versatile Microsoft Windows format, which allows anything from music samples to voice to be digitally recorded for later playback, either from a sound player or over the web.

The MIDI format cannot handle digital data and instead draws on collections of preset musical sounds which are usually stored on a chip on your PC sound card.

To add sound:
 Click here for my sound
When the user clicks the hyperlink, the browser will automatically launch a suitable player if one exists. If not, the browser will ask you if you wish to save the file to disk.
NB/ Care should be taken as these files are of a reasonable size.

BGSOUND TAG
It's an easy way of adding a background sound:
<BGSOUND SRC="mysound.wav" LOOP =50>
Attributes
SRC
The SRC attribute allows the designer to specify which sound file is to be played, and where to find it.

LOOP
This attribute takes one of two values. INFINITE which means the sound will continue to play forever and X, where X is any value which determines how many times the sound will play before ending.

NB/ Netscape does not support the <BGSOUND> tag.

MULTIMEDIA IN YOUR PAGE
<EMBED> tag is used to allow content and plug-in applications to be included in a web page like sound, video clip, motion etc.
<EMBED> tag supports many common file formats like: .wav, .mid, .au, .mov, etc

SRC
The SRC attribute allows the designer to specify which sound file should be played.

WIDTH
Allows the width of the sound playing control to be specified. Most sound controls have a small number of buttons including play, rewind and stop associated with them and by specifying the width and height it is possible to fix the size of the control and buttons.

HEIGHT
Specifies the height of the sound playing control.

HIDDEN
Takes the values "YES" and "NO". Hides the sound playing control from view- useful in conjunction with AUTOPLAY.

AUTOPLAY
Forces the file specified in SRC to play as soon as it is loaded.

LOOP
Takes the values TRUE or FALSE. Using TRUE forces the browser to endlessly repeat the sound file.

VOLUME
Entering a percentage value between 0 and 100 forces the sound tool to alter the playback volume of the sound to the specified level.
Placing it in a page:

<EMBED SRC =" mysound.wav" WIDTH="150" HEIGHT="250" CONTROLS ="TRUE" LOOP = "TRUE">

This would play back the the file mysound.wav, placing the sound player in an area 150 by 250 pixels. The controls for the player would be displayed and the sound will loop. As AUTOPLAY is not set the page visitor would have to use the player controls to start the sound.
<EMBED SRC = "moresound.wav" HIDDEN ="TRUE" LOOP = "FALSE" VOLUME ="50%" AUTOPLAY = "YES">
The above example will hide the sound player, automatically starting the sound as soon as it is loaded. The sound itself will play once(LOOP is set to FALSE) at half of the maximum possible volume .

NB/ You may use multiple embed tags within one page, but it is recommended to never set more than one on autoplay.

CHAPTER 9

HTML TABLES

Tables establish a way of presenting data in a highly controlled tabulated form giving greater control over positioning of elements.
Tables allow information to be in rows and columns.
Apart for allowing text to be arranged in columns tables can be used:

1. To divide the page into various sections
2. To create menus
3. To add interactive form fields
4. To create fast loading headers of page
5. For easy alignment of images

The <TABLE> tag is used to signify the start of a table definition.

CAPTION tag
This gives the caption for the title of the table. The default position of the title is centered at the top of the table.
It is only permitted after the TABLE tag.
A good caption should provide a short heading for the table.
It has the following align attribute values: ALIGN=BOTTOM, TOP, LEFT, RIGHT.

NOTE

- The content of a table is not shown until the entire table is loaded.
    - The table must be declared using the table tag

Attributes of <table> tag
WIDTH
Allows designers to set the width of the table using two methods, either as an explicit value or a percentage value. Using the first method it is possible to create a table to exact horizontal dimensions.
e.g.
<TABLE WIDTH=500> gives the table which is fixed to 500 pixels.

The second method, whilst not as exact, uses a percentage of the available space which allows the table to expand or contract along with the browser if it is resized at any point.
e.g.
<TABLE WIDTH="90%"> forces the table to take 90% of the available horizontal space.

HEIGHT
It is declared just like the width attribute, however, not all browsers accept this attribute.

BORDER
This attribute sets the thickness of the borders surrounding the table.
e.g.
<TABLE BORDER =2>
If no border is desired a value of 0 BORDER=0 is given.
Every table is a collection of rows and cells. Each row contains a number of cells, each of which contains the table information.
Rows must be set up before cells can be added in. In order, to create a table row the <TR> and </TR> container is used. Each row can contain a number of cells .
The color of the border can be set by declaring:
<TABLE BORDER COLOR = red>

Table rows
The <TR> tag has the following attributes:

ALIGN
Specifies the horizontal alignment of cell data for a row. ALIGN can be either LEFT, RIGHT, or CENTER.

VALIGN
Specifies the vertical alignment of cell data for a row. It takes one of the values TOP, MIDDLE, or BOTTOM.

BGCOLOR
A recent addition to the table attributes, BGCOLOR takes the same values as the BGCOLOR attribute of the BODY tag.
NB/ This tag is not supported by all browsers.

Table Cells
For every ROW in a table there must be a number of cells which contain the data to be displayed.

There are two elements for table cells ie <TH> and <TD>
<TD> tag that defines a table data cell. By default the text in this cell is aligned left and centered vertically. It specifies the start and end tag of a cell which may hold data.
<TH> tag that defines a table header cell. By default the text in this cell is bold and centered.
Every <TH> and <TD> tags have the following attributes:

CELLSPACING
This attribute sets the size of the "invisible" margin between individual cells in a table as well as the size of the gap between the cells and the border of the table overall.
e.g.
<TABLE BORDER = 1 BORDERCOLOR=#ff0000 CELLSPACING=20>
<TR><TD>Data1<TD>Data2<TD>Data 3
<TR><TD>Data 4<TD>Data 5<TD>Data 6
</TABLE>

CELLPADDING
Allows the setting of the gap between the left hand edge of table cells and the start of the cell contents. Cellpadding can be used to create areas of space within cells so that the cell contents don't appear to be pushed up close to the edge of that cell.
e.g.
<TABLE CELLSPACING = 20 or = 50%>
<TR><TD>D1<TD>D2<TD>D3
</TR><TD>D4<TD>D5<TD>D6
</Table>

ALIGN
This allows the horizontal position of the table overall to be set.
Giving values of RIGHT or LEFT allows the table to be pushed against the respective page edge, using CENTER will align the table so it sits neatly in the middle of the page.

VALIGN
This allows the designer to specify where the cell contents will physically appear.

BGCOLOR
Allows the background color of individual cells to be specified.

WIDTH
This attribute allows the width of individual table cell to be specified either as an explicit value or as percentage of the total table width.

NB/ Care should be taken when specifying cell widths as the browser will only allow one width to be used per column.
If multiple widths are specified the browser will resize all cells in that column to the width of the widest.

HEIGHT
Takes values explicitly or percentages.
NB/ It is not supported by all browsers.

NOWRAP
Tells the browser that any text in any cell which uses the NOWRAP attribute must appear as a single line, rather than over a number of lines.
NOWRAP can be useful if a specific sentence is required to fill just one line.
It turns off word wrapping.
e.g
```
<TABLE>
<CAPTION ALIGN =Bottom> Table Head Summary </CAPTION>
<TR>
<TD>Row 1 Col 1</TD>
<TD>Row 1 Col 2</TD>
</TR>
<TR>
<TD> Row 2 Col 1 </TD>
<TD> Row 2 Col 2 </TD>
</TR>
</TABLE>
```

Spanning Rows and Columns

- Using COLSPAN and ROWSPAN attributes it is possible to expand cells.
- The values of COLSPAN and ROWSPAN can be anything from 2 to the maximum number of cells in the width and height.
- Values greater than the total number of cells have no effect though care should be taken as this may cause problems later on when the table is extended.
- Spanned cells always extend to the right of their own position (in the case of row span) and into the space below their own position (if it's a colspan) It is not possible to tell a cell to span into space above or to the left of its own position.

COLSPAN = number
This is an attribute that specifies the number of columns spanned by the current cell. The value 0 means that cells span all columns from the current column to the last column of the column group. Cells may span several columns.
e.g
```
<TABLE BORDER =1>
<TR>
<TD>1
<TD>2
<TD>3
</TR>
```

```
<TR>
<TD>4
<TD>6
<TD Colspan="2">8
</TR>

<TR>
<TD>9
<TD>10
</TR>
</TABLE>
```

ROWSPAN = number
This attribute specifies the number of rows spanned by the current cell including the current row.
e.g.

```
<TABLE BORDER =1>
<TR>
<TD>A
<TD rowspan = 2>B
<TD>C

<TR>
<TD>D
<TD>E

<TR>
<TD>F
<TD>G
</TABLE>
```

e.g

```
<TABLE>
<CAPTION ALIGN =Bottom> Table Head Summary </CAPTION>
<TR>
<TD ROWSPAN=2>Row 1 Col 1</TD>
<TD COLSPAN=2>Row 1 Col 2</TD>
</TR>
<TR>
<TD> Row 2 Col 1 </TD>
<TD> Row 2 Col 2 </TD>
</TR>
</TABLE>
```

e.g.

```
<TABLE BORDER=1>
<TR><TD>Row 1  Cell 1</TD><TD>Row1 Cell 2</TD>
<TD>Row 1 Cell 3</TD>
</TR>
<TR><TD ROWSPAN = 2>Row 2 Cell 1</TD>
<TD COLSPAN =2>Row 2 Cell 2</TD>
</TR>
<TR><TD>Row 3 Cell 2</TD><TD>Row 3 Cell 3</TD>
```

</TR>
</TABLE>

ROW GROUP ELEMENTS

Table rows may be grouped into a table head, table foot and the table body sections. This division enables the browser to support scrolling of table bo….. independent of the table head and foot.
Table head and foot information can be repeated each page that contains table data.

THEAD
This contains the header information about the columns.
This element defines a group of header rows in a table. It follows caption, colspangroup elements and precedes the optional TFOOT element and TBODY.
TFOOT
This contains the footer information about the columns. This element defines a group of footer rows in a table. It follows the THEAD and precedes TBODY. It provides a summary row or footnotes that apply to the entire table or portions of it.

TBODY
This defines a group of data rows in a table. A table must have one or more TBODY element. It contains row groups

e.g.
<TABLE>
<THEAD>
<TR>
<TH>ABBR </TH>
<TH>Long Form </TH>
</TR>
</THEAD>
<TFOOT>
<TR>
<TD>Footer
</TR>
</TFOOT>
<TBODY>
<TR>

</TBODY>
</TABLE>

COL ELEMENT
This allows the designer to create structural divisions within a table.
It allows designers to share attributes among several columns without implying any structural grouping.

CHAPTER 10

HTML FRAMES

Frames allow author's to present or render documents in multiple views within sub windows of the main window. This help in keeping certain information stationary e.g. static banner, a small navigation bar.
Frames make it difficult to print paper copies of the web and not all web browsers support frames.
A web page consists of a master HTML document that define FRAMESETS or the arrangement of the framed areas on the page.
Frame is a way to divide the browser screen to allow easier navigation under some circumstances.
The tags used to create a frame document are:
<FRAMESET></FRAMESET>
<FRAME>
<NOFRAME>

General Format:
<HTML>
<HEAD>
<FRAMESET>
</HTML>

<FRAMESET>……..</FRAMESET>

This specifies the layout of the main user window in terms of rectangular subspaces. It is used to declare multiple frames.
This is a container element for dividing a window into rectangular subspaces called frames.
It takes the place of the body element and immediately follows the head.
It can be nested and contain one or more frames element.

NOFRAME element provide an alternate content for browsers that do not support frames.

Attributes

ROWS AND COLS

These define the dimensions of each frame in the set. I.e. row gives the height of each row and cols give the width of each column. Each attribute takes a comma separated by a list of lengths.
e.g.
<FRAMESET ROWS="30%, 25%, 45%" cols = "33%", 33%,34%">

If rows and columns are omitted the implied values for the attribute is 100%.
This element accepts ONLOAD and ONUNLOAD attributes to specify client-side scripting actions to perform when frames have all been loaded or removed.

<FRAME>
This element defines the contents and appearance of a single frame i.e. the rectangular subspace within a frameset documents. Each frame element must be contained within a frameset element that defines the dimensions of the frame.

Attributes of Frame

SRC Attribute

This provides the uniform resource identification of the frames content i.e. the source document which is a typical HTML document. It specifies the location of the initial contents to be contained in the frame.
e.g.
<HTML>
<HEAD><TITLE>A two row framed page</TITLE>
</HEAD>
<FRAMESET ROWS ="15%","85%" >

<FRAME SRC ="Frame-source1.html">
<FRAME SRC ="Frame-source2.html">
</FRAMESET>
</HTML>

Name Attribute
Name = data
This assigns a name to the current frame and may be used as the target of the subsequent links i.e. A, Link, Base element
The name attribute vale must begin with a character in the range A-Z or a-z
The name should be human-readable and based on the content of the frame.
e.g.
<FRAMESET COL = "40%", *">
<FRAME NAME = "Menu" SRC = "List.html" Title = "Menu">
<FRAME NAME = "Content" SRC = "add.html" Title = "Content">
</FRAMESET>

FRAMEBORDER ATTRIBUTE
This specifies whether or not the frame has a visible border.
e.g. FRAMEBORDER =1 tells the browser to draw a border between the frame and all adjoining frames. It is the default value.
FRAMEBORDER=0 tells the browser not to draw a border between this frame and every adjoining frame though borders from other frames will override this.

MARGINWIDTH
This specifies the amount of space to the left between the frame's contents in the left and right margins. The value must be greater than zero in pixels.

MARGINHEIGHT
This specifies the amount of space to be left between the frames content in its top and bottom margins.

NORESIZE
This is a boolean attribute that tells the browser that the frame cannot be resized.
SCROLLING
This specifies whether the scrollbar should be provided for the frame. It takes the values of AUTO, YES or NO.
The default value of AUTO generates scrollbars only when necessary.
The YES value generates scrollbars at all times.
The NO value suppress the scrollbar even if it is required hence should not be used.

LONGDESC
This specifies a link to a long description of the frame. It displays an alternate content for non-visual browsers. This description should supplement the short description provided using title attribute.
It is useful for non-visual browsers.

e.g.
<HTML>
<HEAD><TITLE>Some example of frames</TITLE>
<FRAMESET COLS ="33%", "33%", "33%">
<FRAMESET ROWS ="*, 200">
<FRAME SRC ="Source1.html" Scrolling ="yes">
<FRAME SRC ="Source2.html" NORESIZE MARGINWIDTH ="10">
</FRAMESET>
<FRAME SRC ="Source3.html" FRAMEBORDER ="0">

```
        <FRAME SRC ="Source4.html" FRAMEBORDER ="0">
</FRAMESET>
</HTML>
```

TARGET ATTRIBUTE

This specifies the name of a frame where a document is to be opened. By assigning a name to a frame via a name attribute, the author can refer to it as the target of links defined by other elements.
This attribute may be set for elements that creates links (a, link) image maps (area) and forms (form)
e.g.

```
<HTML>
<HEAD>
</HEAD>
<FRAMESET ROWS ="50%, 50%">
        <FRAME NAME ="Dynamic" SRC = "dy.html">
        <FRAME NAME ="Fixed" SRC = "fixed.html">
</FRAMESET>
</HTML>
```

E.G.

```
<HTML>
<HEAD>
</HEAD>
<BODY>
<P>Now you may have to advance to<A HREF ="doc.html TARGET ="Dynamic">Slide</A>
```

<NOFRAMES> ELEMENT

This contains text that should only be rendered when frames are not displaced or the browsers are configured not to display frames.
It is typically used in a frameset document to provide an alternate content for browsers that has to support the frames.
NOFRAMES element must not contain the BODY elemt.
A meaningful NOFRAMES element should always be provided in a frameset document and should at least contain links to the main frame.

e.g.

```
<HTML>
<HEAD>
<TITLE>This is an example</TITLE>
</HEAD>
<FAMESET COLS="50%, 50%>
        <FRAME SRC ="main.html">
<NOFRAMES>
<P>Here is the <A HREF ="main.html">NONFRAME</A?
</NOFRAMES>
</FRAMESET>
</HTML>
```

NB
It allows the explanation of the document's purpose in cases when it is used with browsers that do not support frames.

<|FRAME> ELEMENT

This defines an inline frame for the inclusion of external objects including other HTML documents.

It can act as a target for other links. It allows the designer to insert a frame within a block of text.
Inserting an inline frame is like inserting an object via the object element.
Object is more widely supported than inline frame.
The content of the Inline frame element should only be displayed by browsers that do not support frames or are configured not to display the frames.
Inline frames may not be resized. It can support the following attributes as the element.

- Frame border
- Marginwidth
- Marginheight
- Scrolling
- SRC
- Name
- Longdescr

Other attributes are:
ALIGN
This allows the content of Frame element: Left, right, bottom, and Center

WIDTH and HEIGHT
This specifies the width and the height of the inline frame.
e.g.

```
<FRAME SRC ="Source.html" WIDTH="400" HEIGHT ="500"  SCROLLING ="auto"
FRAMEBORDER ="1">
<A HREF ="source.html"> Related document </A>
</FRAME>

<FRAME SRC ="doc.html" TITLE ="Famous">
<H2>Frames </H2>
<H3>Gradient</H3>
</FRAME>
```

Note

- The NOFRAME element and LONGDES attributes displays an alternate content.
- Frameset definition never changes but the contents of one of its frames can.
- Frameset definition never changes but the contents of one of its frames can.
- Frameset may make navigation forward and backward through for user browsers.
- Frameset cols ="50,**">
  This means column of 50 pixels and * means the remain space left over.
- Authors should not use an image or similar objects as content of a frame for better accessibility and better indexing with search engines.

CHAPTER 11
HTML FORMS
An HTML form is a section of a document containing normal content, markup, special element called controls and labels on those controls (checkboxes, menus, radio buttons e.t.c)
Forms are used to:
- Add another level of interactivity to your web page.
- Allow communication between your viewers and your website.
- Gather information (take input from users)
- Offer different means of navigation.

Generally, a form is used in conjunction with a CGI, PHP, ASP script.
Forms gather different kinds of users input i.e. fields to type in text, menus to select, radio buttons to choose items.
The web browser takes this information wraps it up into a packaged format that can be sent directly to a web server where there is a customized program. The program can unpack the information, manipulate it, store data and send a feedback page back to the viewer.
Users usually complete a form by modifying its controls before submitting the form to an agent for processing.
Every form has three parts:
1. The FORM tag
2. The actual form elements where the visitor enters information.
3. The submit tag which creates the button that sends all collected information to the server.

FORM ELEMENT
<FORM>…….</FORM> defines an interactive form which is a web page form.
It contains the following form control element which are in the container for the form.
1. <INPUT>
2. <SELECT>……</SELECT>
3. <OPTION>
4. <TEXTAREA>…….</TEXTAREA>
5. <BUTTON>
6. <LABEL>
7. <OPTGROUP>
8. <LEGEND>
9. <FIELDSET>

Form element act as a container for controls:
It specifies:
- The layout of the form
- The program that will handle the completed and submitted form
- The receiving program must be able to place name/value pairs in order to make use of them.(action)
- The method by which user data will be sent to the server (method)
- The character encoding that will be accepted by the server in order to handle this form.

ATTRIBUTES OF THE FORM ELEMENT
1. METHOD
This specifies which HTTP method will be used to submit the form data set. The values are "GET" and "POST".
Format is:
<FORMAT METHOD ="GET">OR<FORM METHOD ="POST">

GET
Data from the form is posted by appendices the data to the end of script URL and send the form input in an URL to the processing agent server the amount of data that it can handle is limited by URL and what browser can process. It should be used when the form causes no side effects i.e. idempotent e.g database

It restricts form data set values to ASCII characters.
It allows form submission to be containers completely in a URL.

POST
****Data from the form is sent as a separate packet to the HTTP server, the form is sent in the body of the submission *******
It does not entail the character encoding and length restrictions imposed by GET.
The form data set is specified to cover the entire character set and is included in the body of the form.

2. ACTION=" "
This specifies the URL of the script that the form should be sent to.
The value of action attribute is the URL of the destination. It usually points to a CGI script or Java script that handles the form submission.
The value is the place where the form is to be sent to.
The URL identified does not have to be CGI script.
It may point to mailto: URL

<form action ="destination_url" method=GET/POST">

e.g
<form action ="mailto:daniel2003@yahoo.com" METHOD ="POST">

Note:
It is server side form handler and specifies the form processing agent.

3. ENCTYPE=" "
This specifies the content type used in submitting the form to the server when the value of method is post.
The default value is "application/x-www-form-url.encoded" the value "multipart/form -data" should be used in combination with input ELEMENT type="file".
e.g.
<form action =mailto:dan2003@yahoo.com METHOD="POST" ENCTYPE ="TEXT/PLAIN">
FORM CONTENTS
</FORM>
<FORM ACTION =http://amarco.co.ke/mg/aserver METHOD ="POST">
FORM CONTENT
</FORM>

4. ACCEPT =" "
It specifies a comma separated list of content types that a server processing this form will handle often it is used with INPUT element.

5. ACCEPT_CHARSET =" "
This specifies the list of character encoding for input data that is accepted by the server processing the form.
The default value is UNKNOWN considered as character encoding used to transmit the document containing form.

6. TARGET =
This is used with frames to specify in which frame the form response should be rendered. If no frame with such a name exist, response is rendered in a new window.
Th values are case sensitive e.g.
- TOP renders the response in the full unframed window.
- SELF renders the response in current frame

- BALNK renders the response in a new window
- PARENT renders the response in the immediate frameset parent.

The following attributes specify client side scripting actions for various events.

7. NAME
It names the element so that it may be referred to from style sheets or scripts.

8. ONSUBMIT
To specify when the form is submitted the action to be taken by the script.

9. ONRESET
To specify the action to be taken by script when the form is reset.

INPUT ELEMENT
This is one of the useful tags used in form's container which is an empty element.
It generates button, input fields and checkboxes.
It inputs areas in a form and defines a form control for the used to enter input.

The following attributes are used in INPUT tag.
1. NAME
This assigns the control name i.e. assign names to the input are.

2. VALUE
This specifies or sets the initial value of the area. It is optional except when type attribute has the value radio or checkbox.

3. SIZE=" "
This sets the horizontal space of the area i.e. the initial width of the control.

4. MAXLENGTH
This sets the maximum size of the characters allowed in the area.

5. CHECKED
This boolean attribute specifies that the button is on. I.e. it initially sets either a radio button or checkbox areas as checked.

6. SRC=" "
This specifies the source the source of an image to be used to decorate the graphical submit button.

7. ALT=
This specifies the alternative text for an image.

8. ALIGN=
This specifies the alignment of the graphical submit button to top, middle and bottom.

9. READONLY
This sets the value of the area to read-only.

10. DISABLED
This disables the use of an area.

11. ACCESSKEY=" "
This sets a key to access the area i.e. short cut key.

12. TABINDEX
This sets the order in which the area should receive focus i.e. position in tabbing order.

The following specify client-side scripting actions for various events.

13. ONCLICK
Used to define the action taken when button is activated.
e.g. <INPUT TYPE=button VALUE ="Hide non-strict attributes" ID = toggler ONCLICK ="TOGGLER()">

14. ONFOCUS
When the element receives focus.

15. ONBLUR
When the element loses focus.

16. ONSELECT
When text in an input of type text or password is selected.

17. ONCHANGE
When the element loses focus and its value has changed since it received focus.

TYPE=
This specifies the type of control to create the default value for this attribute is "text".
It defines the type of form control. It has the following value: text, password, checkbox, radio, submit, reset, file, hidden, image and button.
e.g.
<INPUT TYPE = "image src = "dol.gif">
<INPUT TYPE = text NAME = Username SIZE="8" MAXLENGTH ="8">

TYPE = text
This creates a single line text input control area. E.g. <INPUT TYPE = text NAME="text1" SIZE="30" VALUE ="H11">

TYPE = Password
Create a single line to input text but rendered as a series of asterisks. This contrtol type is used for sensitive inputs.
e.g. <INPUT TYPE = password name ="pass" SIZE="30" VALUE ="Pass">

TYPE = Checkbox
This produces a checkbox if checked. It will come up checked selected initially. For multiple checkboxes the name changes but value the same.
e.g.
<INPUT TYPE ="Checkbox" Name ="Rock" Value = "Yes" Checked>Hotdogs
<INPUT TYPE ="Checkbox" Value ="Yes" Name ="ED> Chips

Note
That every checkbox have a unique name and check boxes allows users to choose one or more options.

TYPE = Radio

This produces a radio button which always exist in a group. All members of this group have same name but different values hence allows users to choose only one of the several options i.e. only one button can be checked initially or latter.
e.g.
<INPUT TYPE ="radio" NAME = food VALUE =Dan>HOT
<INPUT TYPE ="radio" NAME = food VALUE =Yes Checked>COLD
<INPUT TYPE ="radio" NAME = food VALUE =Lock>WARM

TYPE = Submit
This produces a submit button which when pressed sends the content of the form to the servers.
For multiple submit buttons each should have a different name. It sends all the selection, values and entered text to defined action destination.
e.g.
<INPUT TYPE="Submit" VALUE ="Send" Name ="Okoth>
<INPUT TYPE ="Submit" VALUE ="Send" Name ="Dan">
TYPE = Reset
This produces a reset button which will restore the form to its original state if pressed. The value of VALUE attributes is used as text on the button.
e.g.
<INPUT TYPE ="reset" VALUE ="Clear">

TYPE = file
This creates a field through which users can upload files from their local computers or network.
It is typically presented as an input box with a button to start browsing the local hard disk and you can specify one or more file names to upload.
A form that include a file input must specify method =post and enctype = "multipart" form="data" in form tag.
e.g.
<FROM ACTION =http://serve.com/cgi/handle enctype = "multipart/ form - data" method ="post">
<P>Name<INPUT TYPE =text name ="Sender">
File sending?<INPUT TYPE ="file" NAME="files">
</P>
</FORM>

TYPE = Hidden
It allows author's to include form data without having it rendered to the user. It is useful if the document is generated by a script and author need to store state information.
User input can be carried from form to form by hidden inputs.
It sends the form data without appearing in the layout of the web page.
e.g.
<INPUT TYPE ="hidden" NAME ="Instructor" VALUE =Mulei@yahoo.com>

TYPE =Image
This creates a graphical submit button and provide alt attribute to act as an alternative text to image you can also create an image map.
e.g.
<INPUT TYPE =Image SRC = source.gif ALT ="Submit">

If only act as submit but not reset buttons.

TYPE = "text area"
Are text field that have more than one line and can scroll as the viewer enters more text.

Example

<INPUT TYPE ="Text" NAME ="Text" SIZE ="120">
<INPUT TYPE ="password" NAME ="pass" SIZE ="30">
<INPUT TYPE ="checkbox" NAME ="box" VALUE ="box" Checked>
<INPUT TYPE ="Radio" NAME ="food" VALUE ="Yes">
<INPUT TYPE ="Submit" VALUE ="SEND">
<INPUT TYPE ="Reset" VALUE ="RE">
<INPUT TYPE ="Textarea" COLS =30 ROWS = 40>

Quiz

What is the structure of a web form.
Describe the content of a web from data submission.
What is the difference between radio button and checkboxes, submit and reset forms buttons.

TEXTAREA ELEMENT

This is a container element which defines a text input window, <textarea> is used for large textual areas.
It allows users to enter multiple lines of text or information.
When the form is submitted the current value of any textarea within the form is sent to the server as name/ value pair.
The name attribute provides the name used and cols and rows attributes specify the number of visible rows and columns in visual browser.
Readonly attribute prevent the user from editing the content of the textarea.
Tabindex attribute specifies a number between 0 and 3276 to indicate te tabbing order of the element.
Accessing attribute specified a single unicode character as a shortcut key for giving focus to the textarea.

WRAP attribute
This is automatically wrap the text according the values.

Wrap = Virtual
Means that the text in box wraps but it is sent as one long continuous string.

Wrap = Physical
Means that the text in the box wraps and is sent that way too.

Wrap = Off
Means that the text in the box does not wrap but it is sent exactly the way it was typed in.

e.g.
<TEXTAREA NAME="Comments" ROWS ="10" Cols ="45" WRAP ="Virtual">
</Textarea>

DROP DOWN SELCT MENU
ELEMENT <SELECT>
This is a container element used to create a menu that will drop down when clicked on and allow the viewer to choose one from a list of choices.
It defines a form control for the selection of options.
It contains one or more OPTION or OPTGROUP elements to provide a menu of choices for the user.
By default users can only select one option but multiple attribute of select allow multiple selection.
The NAME attribute provides the control to sent to the server with the value of the selected option.
The SIZE attribute specifies the number of rows in the list that should be visible at the same time.
e.g.
<SELECT NAME = Section>
<OPTION>Otieno
<OPTION>Kamau
<OPTION>Nyambane
</Select>

<Select name ="Section" Multiple>

STRUCTURAL DEFINITION

	Heading	<H?></H?>	(the spec. defines 6 levels)
	Align Heading	<H? ALIGN=LEFT|CENTER|RIGHT></H?>	
	Division	<DIV></DIV>	
	Align Division	<DIV ALIGN=LEFT|RIGHT|CENTER|JUSTIFY></DIV>	
4.0	**Defined Content**	<SPAN></SPAN>	
	Block Quote	<BLOCKQUOTE></BLOCKQUOTE>	(usually indented)
4.0	**Quote**	<Q></Q>	(for short quotations)
4.0	**Citation**	<Q CITE="URL"></Q>	
	Emphasis	<EM></EM>	(usually displayed as italic)
	Strong Emphasis	<STRONG></STRONG>	(usually displayed as bold)
	Citation	<CITE></CITE>	(usually italics)
	Code	<CODE></CODE>	(for source code listings)
	Sample Output	<SAMP></SAMP>	
	Keyboard Input	<KBD></KBD>	
	Variable	<VAR></VAR>	
	Definition	<DFN></DFN>	(not widely implemented)
	Author's Address	<ADDRESS></ADDRESS>	
	Large Font Size	<BIG></BIG>	
	Small Font Size	<SMALL></SMALL>	
4.0	**Insert**	<INS></INS>	(marks additions in a new version)
4.0	**Time of Change**	<INS DATETIME=":::"></INS>	
4.0	**Comments**	<INS CITE="URL"></INS>	
4.0	**Delete**	<DEL></DEL>	(marks deletions in a new version)
4.0	**Time of Change**	<DEL DATETIME=":::"></DEL>	
4.0	**Comments**	<DEL CITE="URL"></DEL>	
4.0	**Acronym**	<ACRONYM></ACRONYM>	
4.0	**Abbreviation**	<ABBR></ABBR>	

PRESENTATION FORMATTING

	Bold	<B></B>	
	Italic	<I></I>	
4.0*	**Underline**	<U></U>	(not widely implemented)
	Strikeout	<STRIKE></STRIKE>	(not widely implemented)
4.0*	**Strikeout**	<S></S>	(not widely implemented)
	Subscript		
	Superscript		
	Typewriter	<TT></TT>	(displays in a monospaced font)
	Preformatted	<PRE></PRE>	(display text spacing as-is)
	Width	<PRE WIDTH=?></PRE>	(in characters)

	Center	<CENTER></CENTER>	(for both text and images)
N1	**Blinking**	<BLINK></BLINK>	(the most derided tag ever)
	Font Size	<FONT SIZE=?></FONT>	(ranges from 1-7)
	Change Font Size	<FONT SIZE="+|-?"></FONT>	
	Font Color	<FONT COLOR="#$$$$$$"></FONT>	
4.0*	**Select Font**	<FONT FACE="***"></FONT>	
N4	**Point size**	<FONT POINT-SIZE=?></FONT>	
N4	**Weight**	<FONT WEIGHT=?></FONT>	
4.0*	**Base Font Size**	<BASEFONT SIZE=?>	(from 1-7; default is 3)
MS	**Marquee**	<MARQUEE></MARQUEE>	

CHAPTER 12

INTRODUCTION TO COMMON GATEWAY INTERFACE(CGI)

WHAT IS CGI Scripts?

A CGI script is a program that is stored on the remote web server and executed on the web server in response to a request from a user.

- The Common Gateway Interface *(CGI)* defining how a program interacts with a Hyper Text Transfer Protocol (HTTP) server.

The Common Gateway Interface (CGI) provides the middleware between WWW servers and external databases and information sources. CGI applications *perform specific* information processing, retrieval, and formatting tasks on behalf of WWW servers.

What is a CGI program?

A CGI program is a computer program that is started by the web server in response to an HTTP request.
A CGI program is generally used to process HTML form input from a Web browser. The HTML <*form*> tag's *action* attribute specifies the name of the CGI program (including the server where the program resides).
For example, the following <form> tag specifies a program named *query.cgi* in the */cgi-bin* directory on the Web server *digital. com*
<FORM method = GET action = http://www.digital.com/cgi-bin/query.cgi>

A CGI script file is written in a programming language, which can be either:

- Compiled to run on the server.
- Interpreted by an interpreter on the server.

Examples, of languages used to write CGI scripts include:

- Compiled languages - C, C++, Ada
- Interpreted languages - Perl, JCL (e.g. The unix sh command language)

Why is CGI used?

An interesting aspect of a CGI enabled Web server is that computer programs can be created and deployed that can accept user input and create a Web page on the fly. Unlike static Web pages that display some preset information, these *interactive* web pages enable a client, to send information to the Web server and get back a response that depends on the input.

A Web search engine is a good example of an interactive web page. The client enters one or more keywords, and the Web index returns a list of Web pages that satisfy the search criteria entered. The Web page returned by the Web index is also *dynamic,* because the content of that page depends on what the client types in as search words - it's not a predefined static document.
To create an interactive Web page, HTML elements are used to display a form that accepts a client's input and passes this to special computer programs on the Web server. This computer programs process a client's input and return requested information, usually in the form of a web page constructed on the fly by the computer program. These programs are known as gateways because they typically act as a conduit between the Web server and an external source of information, such as a database. Gateway programs exchange information with the Web server using a standard known as *The Common Gateway Interface.* This is the reason *CGI programming* is used to describe the task of writing computer programs that handle clientrequests for information.

The term *gateway* describes the relationship between the WWW server and external applications

that handle data access and manipulation chores on its behalf. A gateway interface handles information requests in an orderly fashion, and then returns an appropriate response. For example, an HTML document generated on *the fly, which contains the results of a query, applied against an external database.*
In other words, CGI allows a WWW server to provide information to WWW clients that would otherwise not be available to those clients. This could, for example, allow a WWW client to issue a query to an Oracle database and receive an appropriate response in the form of a custom built Web document.
Some common uses of CGI include

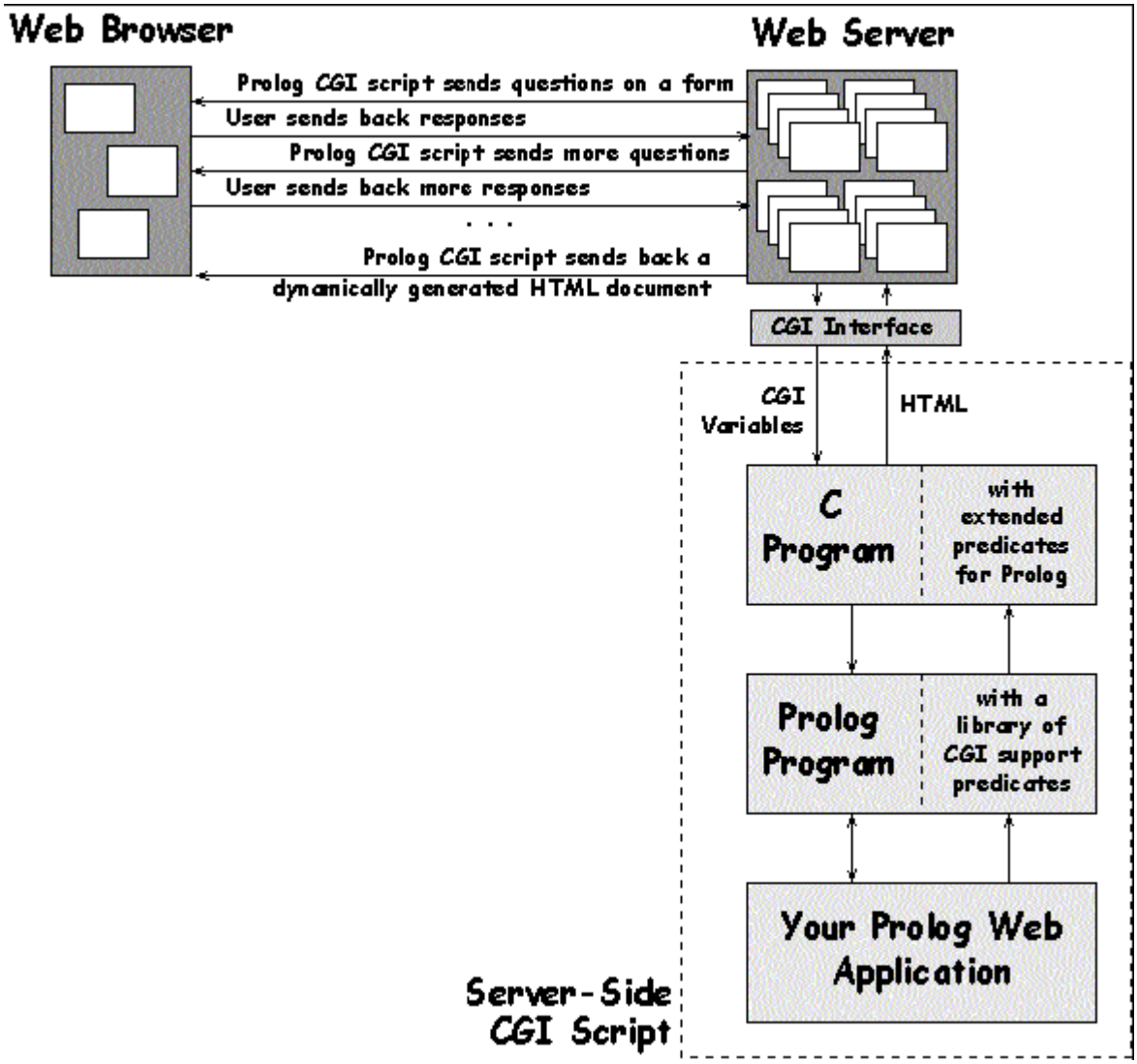
- Gathering user feedback about a product line through an HTML form.
- Querying an Oracle database and rendering the result as an HTML document.

CGI - HOW IT WORKS

A Web browser running on a client machine exchanges information with a Web server using the HyperText Transfer Protocol or HTTP. The Web server and the CGI program normally run on the same system, on which the web server resides. *Depending* on the *type* of request from the browser, the web server either provides a document from its own document directory or executes a CGI program.

The sequence of events for creating a dynamic HTML document on the fly through CGI scripting is as follows:

- A client makes an HTTP request by means of a URL. This URL could be typed into *the 'Location'* window of a browser, be a hyperlink or be specified in the 'Action' attribute of an HTML <*form*> tag.
- From the URL, the' Web server determines that it should activate the gateway program listed
- in the URL and send any parameters passed via the URL to that program.
- The gateway program processes the information and returns HTML text to the Web server.
- The server, in turn, adds a MIME header and returns the HTML text to the Web browser.
- The Web browser displays the document received from the Web server.



The CGI script is executed when an anchor tag <A ... > or an image tag <IMG ...> refers to the CGI script file rather than a normal file.

The determination of whether this is a CGI script file or just an HTML file is made on the physical placement of the file on the server. Remember, the script file is placed on the same machine on which the web server runs and not on your local machine.

It is of course possible to run a local web server on the machine on which you run the web browser. However, for other people to access the files delivered by your web server you will need a permanent connection to the internet.

Usually this placement is in the remote web server's **cgi-bin** directory. However the exact location of this directory on the server machine is determined by the web administrator for that machine. This placement and control of the cgi-bin directory is determined by the web administrator to prevent security problems, that could occur if arbitrary programs where allowed to be executed by anybody accessing the machine.
Some web administrators may not allow you to create CGI scripts on their machine.

Call Of A CGI Script File

An anchor tag to execute the CGI script dynamic_page on the server www.mc.com is:

<A HREF="http://www.mc.com/cgi-bin/dynamic_page">Dynamic page</A>

When the web server process a request to fetch a file, if the requested file is in the servers nominated cgi-bin directory then as long as this file is marked as being executable the script will be run on the server. If the file is not executable then an error will be reported.

The script eventually returns an HTML page or image to be displayed as the result of its execution. When a CGI script file executes it may access environment variables to discover additional information about the process that it is to perform. The first line of the returned data **must** be:

How information is transferred from the Web Browser to a CGI program
The Web browser uses the *method* attribute of the <*form*> tag to determine how to send the form's data to the Web server. There are two submission methods:

GET
The Web browser submits the form's data as a part of a U RL

POST:
The Web browser sends the form's data separately from the URL as a stream of bits.

The GET Method

Tile GET method is so called because the browser uses the HTTP GET command to submit the data. The GET command sends a URL to the Web server. If the form's data is sent to the Web server using the GET command, the browser must include all data in the URL. The key features of the GET method of data submission are as follows:
The values of all the fields are concatenated and passed to the U RL specified in the *lie/ion* attribute of the <*form*> tag. Each field's values appear in the *name-value* format.
Any character with a special meaning in the form's data is encoded using a special encoding scheme commonly referred to as URL encoding. In this encoding scheme a space is replaced by a plus sign (+), fields are separated by an ampersand (&), and any non-alphanumeric character is replaced by a %xx code (where xx is a hexadecimal representation of the character).

The POST Method
In the POST method of data submission, the Web browser uses the POST command to submit the data to the server and includes the form's data in the body of that command. The POST method can handle any amount of data, because the browser sends the data as a separate stream. The POST method should be used to send potentially large amounts of data to a web server.
How a CGI URL is interpreted by the Web Server
The Web server must be configured to recognize an HTTP request for a CGI program, In short, configuring the web server involves informing it of the directory where the CGI programs reside. The URL specifying a CGI program looks like any other URL, but the Web server can examine the directory name and determine whether the URL is a normal document or a CGI program. The Web server expects the CGI program's name to appear immediately following the CGI directory (e.g. /cgi-bin/).
A URL can also include *additional path information,* which can be used by the CGI program. This path information needs to be included in the URL immediately following the CGI program name.

How a CGI Program returns Information to the Server

Regardless of how the Web server passes information to -the CGI program, the CGI program always returns information to the server by writing to the standard output. In other words, if a CGI program wants to return an HTML document, the program must write that document to standard output. The Web server then processes that output and sends the data back to the browser that had originally submitted the request. The CGI program adds appropriate header information to its output and sends this to the web server.

Processing Form Information in a CGI Program

CGI program needs to be able to access the form data and process it some way before generating any meaningful output. When a form submits data via the GET method, the CGI program

297

receives information through the QUERY_STRING environment variable. If the form submits data via the POST method, the CGI program receives information through standard input.

The basic steps for a CGI program designed to handle URL-encoded data sent through both GET and POST methods are as follows:

Check the REQUEST_METHOD environment variable to determine whether the request is GET or POST.

- If the method is GET, use the value of the QUERY STRING environment variable as the
- input. Also, check for any path information in the PATH_INFO environment variable.
- If the method is POST, get the length of the input (in number of bytes) from the
- CONTENT_LENGTH environment variable. Then read that many bytes from the standard
- input.
- Extract the *name-value* pairs for various fields by splitting the input data at the ampersand
- (&) character, which separates the values of fields.
- In each *name-value* pair convert all + signs to spaces.
- In each *name-value* pair, convert all %xx sequences to ASCII characters (xx, denotes a pair
- of hexadecimal digits).
- Save the *name-value* pairs denoting values of specific fields for later use.

Why Perl for CGI ?

A CGI program can be written in any of the above-mentioned languages, but Perl is especially suited for this because Perl programs are easy to learn and write. Perl has great text-processing capabilities (CGI programs have to process the URL-enclosed text data and print HTML text to standard output), which is why it was a natural choice for some of the first CGI sample programs provided by NCSA. As it can accomplish the same task as a C or *C++* program with far fewer lines of code, it has become the most widely used option for custom CGI scripts.

Besides this Perl is a scripting language, which means it does not have to be compiled. Instead, an interpreter executes the Perl script, this makes it easy to write and test Perl scripts, because they do not have to go through the typical edit-compile-link cycles.

CHAPTER 13

JAVA SCRIPT

INTRODUCTION TO JAVASCRIPT

1.1: JavaScript and Java

JavaScript is a compact, object-based scripting language for developing client and server Internet applications. Netscape Navigator 2.0 interprets JavaScript statements embedded directly in an HTML page, enables you to create server-based applications similar to common gateway interface (CGI) programs.

In a client application for Navigator, JavaScript statements embedded in an HTML page can recognise and respond to user events such as mouse clicks, form input, and page navigation.

For example, you can write a JavaScript function to verify that users enter valid information into a form requesting a telephone number. Without any network transmission, an HTML page with embedded JavaScript can interpret the entered text and alert the user with a message dialog if the input is invalid. Or you can use JavaScript to perform an action (such as play an audio file, execute an applet, or communicate with a plug-in) in response to the user opening or exiting a page.

The JavaScript language resembles Java, but without Java's static typing and strong type checking. JavaScript supports most of Java's expression syntax and basic control flow constructs. In contrast to Java's compile-time system of classes built by declarations, JavaScript supports a run-time system based on a small number of data types representing numeric, Boolean, and string values. JavaScript has a simple instance-based object model that still provides significant capabilities.

JavaScript also supports functions, again without any special declarative requirements. Functions can be properties of objects, executing as loosely typed methods.

JavaScript complements Java by exposing useful properties of Java applets to script authors. JavaScript statements can get and set exposed properties to query the state or alter the performance of an applet or plug-in.

By comparison, JavaScript is used to produce scripts designed to react to user and environment events as well as in future being the glue to hook Java applets more seamlessly into WEB pages. The following are some example:

- An interactive colour picker for WEB developers to test different background and text colours in their documents.
- Calculators: Examples on the WEB include a unit conversion calculator and loan interest calculator.
- Dynamic output based on the current environment and the user’s previous surfing history.
- Forms verification: JavaScript can be used to ensure that from data is entered properly before sending it to the server, rather then relying than relying on the server to verify from content after is submitted.
- Building URLs: JavaScript is used to build custom URLs based on the user choices in forms.
- JavaScript can be used to replace many CGI scripts for clients-side processing, easing bandwidth demands, and decreasing server load for busy WEB severs.
- Newsgroup-like discussion groups can incorporate sophisticated features such as threading.

1.2 Strengths of JavaScript

The following compares and contrasts JavaScript and Java.

JavaScript	**Java**
Interpreted (not compiled) by client.	Compiled on server before execution on client.
Object-based. Code uses built-in, extensible objects, but no classes or inheritance.	Object-oriented. Applets consist of object classes with inheritance.
Code integrated with, and embedded in, HTML.	Applets distinct from HTML (accessed from HTML pages).
Variable data types not declared (loose typing).	Variable data types must be declared (strong typing).
Dynamic binding. Object references checked at run-time.	Static binding. Object references must exist at compile-time.
Secure. Cannot write to hard disk.	Secure. Cannot write to hard disk.

1.2.1 Quick Development

Because JavaScript doses not require time-consuming compilation, scripts can be developed in relatively short period of time. This is enhanced by the fact that most of the interface features, such as dialog boxes, forms, and other GUI elements, are handled by the browser and HTML code. JavaScript programmers don't have to worry about creating or handling these elements of their application.

1.2.2 Easy to Learn

While JavaScript may share many similarities with Java. By learning just a few command and simple rules of syntax, along with understanding the way objects are used in JavaScript, it is possible to begin creating fairly sophisticated programs.

1.2.3 Platform Independence

Because the World Wide Web, by its very nature, is platform-independent, JavaScript programs created for Netscape Navigator are not tied to any specific hardware platform or operating system. The same program code can be used on any platform for which Navigator 2.0 is available.

1.2.4 Small Overhead

JavaScript programs tend to be fairly compact and are quite small, compared to the binary applets produced by Java. This minimises storage requirements on the server and download times for the user. In addition, because JavaScript programs usually are included in the same file as the HTML code for a page, they require fewer separate network accesses.

1.3 Weaknesses of JavaScript

As would be expected, JavaScript also has its own unique weaknesses. These include a limited set of built-in methods, the inability to protect source code from prying eyes, and the fact that JavaScript still doesn't have a mature development and debugging environment.

1.3.1 Limited Range of Built-in Methods

Early version of the Navigator 2.0 beta included a version of JavaScript that was rather limited. In the final release of Navigator 2, the number of built-in methods had significantly increased, but still didn't include a complete set of methods to work with documents and the client windows. With the release of Navigator 3, things have taken a further step forward with the addition of numerous methods, properties, and event handlers.

1.3.2 No Code Hiding

Because the source code of JavaScript presently must be included as part of the HTML source code for a document, there is no way to protect the code from being copied and reused by people who view your WEB pages.

This raises concerns in the software industry about protection of intellectual property. The consensus is that JavaScript scripts are basically freeware at this point of time.

1.3.3 Lack of Debugging and Development Tools

Most well-developed programming environment include a suite of tools that make development easier and simplify and speed up the debugging process.

Currently, there are some HTML editors that provide JavaScript support. In addition, there are some on-line tools for debugging and testing JavaScript script. However, there are really no integrated development environments such as those available for Java, C, or C++. Live wire from Netscape provides some development features but is not a complete development environment for client side JavaScript.

1.4 JavaScript Development

A script author is not required to extend, instantiated, or know about classes. Instead, the author acquires finished components exposing high-level properties such as "visible" and "colour", then gets and sets the properties to cause desired effects.

As an example, suppose you want to design an HTML page that contains some catalogue text, a picture of a shirt available in several colours, a form for ordering the shirt, and a colour selector tool that's visually integrated with the form. You could write a Java applet that draws the whole page, but you'd face complicated source encoding and forgo the simplicity of HTML page authoring.

A better route would use Java's strengths by implementing only the shirt viewer and colour picker as applets, and using HTML for the framework and order form. A script that runs when a colour is picked could set the shirt applet's colour property to the picked colour. With the availability of general-purpose components like a colour picker or image viewer, a page author would not be required to learn or write Java. Components used by the script would be reusable by other scripts on pages throughout the catalogue.

1.5 Preparing to Begin

In order to take full advantage of this book, you will need several tools. A copy of the latest version of Netscape Navigator or Microsoft Internet Explorer is essential to develop and test program code. In addition, a good editing program that you will feel comfortable using will make program development process easier.

1.6 Editing and Development Tools

In addition to a copy of Navigator or Internet Explorer, a strong editor or development tool will make the task of entering, developing, and debugging JavaScript much easier.

If you have been doing a lot of HTML authoring or programming, you probably have you own tools that should also well suited to JavaScript development. As long as you editor produces plain ASCII text file, you should be fine.

In considering editors, it would be worth looking at tools that can help you in identifying the current line number for debugging scripts.

1.7 WEB Browser

Although Navigator or Internet Explorer started out as a basic WEB Browser, it has grown increasingly popular. Unlike earlier Browsers and today's basic web applications, both WEB Browsers now provide authors with numerous tools to step beyond the traditional constraints of HTML. Instead of simply combing text, pictures, sound and video, WEB designer now have finer control over document layout fonts, colour, they are able to extent the functionality of the Browser using plug-ins and Java, and they can produce interactive applications using JavaScript.

2.1 Incorporating JavaScript into HTML

All JavaScript scripts need to be included as integral part of an HTML document. To have this, Netscape has implemented an extension to standard HTML: the <SCRIPT> tag.

WEB SCRIPTING USING HYPERTEXT MARKUP LANGUAGE (HTML)

Hypertext Markup Language consists of *tags* that are interpreted by the web browser to display text when the HTML file is opened on the screen by Web browser software.

A **Tag** is a special word enclosed between the less than and greater than (**< >**) symbols, and the browser can interpret is as a command.
E.g., to start a HTML page, type; <HTML> at the top of the document.

The ***Markup tags*** tell the internet Browser that the file contains HTML-coded information. They also define the various components of a WWW document such as tables, paragraphs, and lists.

Web pages are written using HTML in a text format that provides many features such as the capability for in-line graphics, font control, and Hypertext links. A ***Hypertext link*** is a word or phrase in an HTML-document that enables the user to jump to another Web page, or connect to another Web Server.

HTML is simple, doesn't have the declaration part and control structures. In addition, it has many limitations; hence, cannot be used alone when developing functional websites.
To add functionality to the HTML pages, some special blocks of code known as ***scripts*** are inserted in HTML pages using scripting languages like *JavaScript*, *VBScript* and *Hypertext Preprocessor*.

A **Script** is a small program fragment, written in a different language other than HTML, but inserted into the HTML program.

Most HTML tags have an opening tag and a closing tag. An opening tag is enclosed between < >, while a closing tag is enclosed between </ >. The text that is to be displayed on the screen is enclosed between an opening and closing tag.

HTML Tag	**Meaning**
<HTML> </HTML>	Marks the beginning and end of a HTML document.

	All other tags and text fall between these two tags.
<HEAD></HEAD>	Marks the header part of the document
<TITLE></TITLE>	Gives the title of the web page. The text between these tags will appear in the title bar when the HTML document page is running or browsed.
<BODY></BODY>	Marks the body part of the document
<CENTER></CENTER>	Centers text and objects on the web page
<B></B>	Bolds the text on the web page, e.g., the statement <B> Hello </B> will display the word "Hello" in bold on the screen
<I></I>	Italicize the text
<H1></H1>	Sets the size of the text on the web page with H6 displaying the smallest and H1 the largest size

Creating a script using JavaScript

Before writing your HTML program with a script inserted, make sure you have the latest browser software installed on your computer.
e.g., if you're using Internet Explorer, it should be version 5.0 and above.

1. Open Notepad, and type the following program.
2. Save your file on the desktop as *Example.html*, and then close the notepad. Notice that the icon to your file on the desktop looks like that of the default web browser in your computer.
3. To view the web page, double-click the icon of the file Example.html on the desktop.

	Explanation
<HTML>	The tag <HTML> marks the beginning of the HTML document
<HEAD> <TITLE> Kiongwani Secondary School </TITLE> </HEAD>	Displays the text Kiongwani Secondary School in the title bar of the running HTML document. The title is written in the header section, i.e., between<HEAD> and </HEAD> tags.
<BODY>	It marks the beginning of the body section. Anything between<BODY> and </BODY> will be executed and displayed when the web page starts running.
<HI> <CENTER. <B> We are the world </B> </CENTER> </HI>	This line will display the text We are the world on the screen. The text will be large, i.e, size H1 and it will be centered on the screen. The text will also be bolded.
<SCRIPT LANGUAGE = 'JavaScript'>	Marks the start point of the script. The line LANGUAGE = 'JavaScript' tells the browser that the script will be written in JavaScript language.
Document.Write ('My name is strongman');	The statement Document. Write tells the browser to write whatever is in the brackets using JavaScript. **NB**. In JavaScript, the end of a statement is marked by a semicolon (;).
Alert ('congratulations for succeeding to run this script');	The word alert displays a message box on the screen with an OK button. Clicking the button makes the message box to disappear. The text in the brackets appears in the dialog box.
</SCRIPT>	Closes the script

</BODY> </HTML>	Marks the end of the body, and the HTML code.

Practical activity

1. Open a text editor program on your computer like Notepad or WordPad, and type the following program.

```
<HTML>
<HEAD>
<TITLE> Hello! This is my first webpage </TITLE>
</HEAD>
<BODY bgcolour = “Blue”>
<H3><CENTER><B>Hello World</B></CENTER></H3>
</BODY>
</HTML>
```

2. Save your file on the desktop as *Webpage.html*, and make sure that the *Save As Type* box reads “All Files” before clicking the **Save** button.
3. Close your text editor.
4. Double-click the icon of the file on the desktop to view the web page.
5. Change your background to blue, and save your program..

2.1.1 The SCRIPT Tag

To include the script tag in an HTML document is very simple. Every scripting statement must be contained inside a *SCRIPT* container tag. Just like HTML, when you are opening <BODY> tag, it starts the body of your HTML document and a closing </BODY> tag ends it. In JavaScript, all that you have to do is open the script tag by <SCRIPT> and close it by have this tag at the end of you script </SCRIPT>.

- <SCRIPT>
- Scripting statements
- </SCRIPT>

2.1.2 Including JavaScript in an HTML File

The first and easiest, way is to include the actual source code in the HTML file, by using the following syntax:

```
<SCRIPT LANGUAGE = “JavaScript”>
```

```
JavaScript Program
</SCRIPT>
```

2.1.3 Specifying the JavaScript Version

The optional LANGUAGE attribute specifies the scripting language and JavaScript version:

```
<SCRIPT LANGUAGE="JavaScriptVersion">
JavaScript statements...
</SCRIPT>
```

JavaScriptVersion specifies one of the following to indicate which version of JavaScript your code is written for:

- <SCRIPT LANGUAGE="JavaScript"> specifies JavaScript for Navigator 2.0.
- <SCRIPT LANGUAGE="JavaScript1.1"> specifies JavaScript for Navigator 3.0.
- <SCRIPT LANGUAGE="JavaScript1.2"> specifies JavaScript for Navigator 4.0.

Statements within a <SCRIPT> tag are ignored if the user's browser does not have the level of JavaScript support specified in the LANGUAGE attribute; for example:

- Navigator 2.0 executes code within the <SCRIPT LANGUAGE="JavaScript"> tag; it ignores code within the <SCRIPT LANGUAGE="JavaScript1.1"> tag and <SCRIPT LANGUAGE="JavaScript1.2"> tag.
- Navigator 3.0 executes JavaScript code within either the <SCRIPT LANGUAGE="JavaScript"> or <SCRIPT LANGUAGE="JavaScript1.1"> tags but ignores code within the <SCRIPT LANGUAGE="JavaScript1.2"> tag.
- Navigator 4.0 executes JavaScript code within either the <SCRIPT LANGUAGE="JavaScript"> or <SCRIPT LANGUAGE="JavaScript1.1"> tags and <SCRIPT LANGUAGE="JavaScript1.2"> tag.

If the LANGUAGE attribute is omitted, Navigator 2.0 assumes LANGUAGE="JavaScript". Navigator 3.0 assumes LANGUAGE="JavaScript1.1". Navigator 4.0 assumes LANGUAGE="JavaScript1.2"

example

This example shows how to define functions twice, once for JavaScript 1.0, and once using JavaScript 1.1 features.

```
<SCRIPT LANGUAGE="JavaScript">
// Define 1.0-compatible functions such as doClick() here
</SCRIPT>
<SCRIPT LANGUAGE="JavaScript1.1">
// Redefine those functions using 1.1 features
// Also define 1.1-only functions
</SCRIPT>
<FORM ...>
<INPUT TYPE="button" onClick="doClick(this)" ...>
. . . </FORM>
```

2.2 Hiding Scripts from Other Browsers

One major problem, that we are going to face with this <SCRIPT> tag is browses that do not support JavaScript will attempt to display the content of the script. To avoid this problem, Netscape recommends the following approach using HTML comments:

```
<HTML>
<HEAD>
<SCRIPT LANGUAGE = “JavaScript”>
<!-- HIDE THE SCRIPT FORM OTHER BROWSERS

JavaScript Program

// STOP HIDING FORM OTHER BROWSERS -->
</SCRIPT>
</HEAD>
</HTML>
```

These Comments (<! -- HIDE THE SCRIPT FORM OTHER BROWSERS and // STOP HIDING FORM OTHER BROWSERS -- >) ensure that Web browsers that do not support JavaScript will ignore the entire script and not display it in the window, because everything between <! - - and - - > should be ignored by a standard Web browser.

Look out: Unlike HTML, comments are created with these tags <! - - Comments Here - - >, JavaScript comments start with a double-slash (//) anywhere on the line and continue to the end of that line.

Starting with Netscape Navigator3, they introduced the <NOSCRIPT> tag, which enable you to display alternative text for non-JavaScript browsers. Texts that are between the <NOSCRIPT> tag will be display by non-JavaScript browsers, but for Netscape Navigator v3.0 and above will ignore those texts.

```
<HTML>
<HEAD>
<SCRIPT LANGUAGE = "JavaScript">
<!-- HIDE THE SCRIPT FORM OTHER BROWSERS
//JavaScript Program
// STOP HIDING FORM OTHER BROWSERS -->
</SCRIPT>
</HEAD>
<NOSCRIPT>
<CENTRE>
JavaScript Script Appears Here<BR>
<H3>Best View in one of the following Browsers</H3>
Download Netscape Navigator v3.0 and above or<BR>
Internet Explorer v3.0 and above to use it. <BR>
</CENTRE>
<NOSCRIPT>
</HTML>
```

2.3 Where to Put Your JavaScript Code

An HTML file is able to have more than one <SCRIPT> tag. JavaScript script and programs can be included anywhere in the header or the body of an HTML file. However, major sites like Netscape's Web site, and many others have their SCRIPT container in the header of the HTML file, which is the preferred format.

INPUT: 2.1 Coding exercise – Include a JavaScript program in an HTML file.

```
<HTML>
<HEAD>
<TITLE> JavaScript Exercise 1 </TITLE>
</HEAD>
<BODY>
Welcome to Informatics Computer School.<BR>
<SCRIPT LANGUAGE = "JavaScript">
<!-- HIDE THE SCRIPT FORM OTHER BROWSERS

//Output "Cool, I am learning JavaScript and it works!!!"
document.write("Cool, I am learning JavaScript and it works!!!");

// STOP HIDING FORM OTHER BROWSERS -->
</SCRIPT>
</BODY>
</HTML>
```

how JavaScript comments work. The line that begins with two slashes,

```
//Output "Cool, I am learning JavaScript and it works!!!"
```

that how JavaScript has a single line comment similar to those used in C++ or C programming Language syntax. Everything after the double-slash until the end of that line is a comment. JavaScript also support C style of multiple line comments, which starts with /* and end with */.

```
<HTML>
<HEAD>
<TITLE> JavaScript Exercise 1 </TITLE>
</HEAD>
<BODY>
Welcome to Informatics Computer School.<BR>
<SCRIPT LANGUAGE = "JavaScript">
<!-- HIDE THE SCRIPT FORM OTHER BROWSERS

/* ---------- Multiple line comments starts here ----------
Output the sentence below without the quotes
"Cool, I am learning JavaScript and it works!!!"
if you do not see these words it means that
you browser do not support JavaScript.
---------- And multiple line comments ends here. ---------- */

document.write("Cool, I am learning JavaScript and it works!!!");
```

```
// STOP HIDING FORM OTHER BROWSERS -->
</SCRIPT>
</BODY>
</HTML>
```

2.4 Basic Command Syntax

The basic command unit is a one-line command or expression following by a optional semicolon ; for example:

```
document.write("Cool, I am learning JavaScript and it works!!!");
```

This command line invokes the write() method, which is part of the document object. The semicolon indicates the end of the command.

A single JavaScript command can be written in multiple lines, as well as multiple command can also written in a single line, as long as there is a semicolon to mark the end of each command. For example:

```
document.write("Cool, I am learning JavaScript"); document.write(" and it works!!!");
```

2.5 Command Blocks

Multiple commands can be combined into a single command blocks by using curly brace ({ and }). Command blocks are used to group sets of JavaScript statements or commands into a single unit, which can be used for various purpose, like defining function.

```
    {
    document.write("JavaScript testing zone");
    document.write("Cool, I am learning JavaScript and it
    works!!!");
}
```

Command blocks can also be embedded, as the following lines;

```
{
    document.write("JavaScript Command Blocks");
    {
       document.write("JavaScript testing zone");
       document.write("Cool, I am learning JavaScript and it
       works!!!");
    }
}
```

when you are embedding command blocks like the above, it is important that you remember that all open curly braces must be closed and that the first closing braces will close the last open braces. The opening and closing of braces are mark by |:

```
{
|       document.write("JavaScript Command Blocks");
```

```
|     {
|     |     document.write("JavaScript testing zone");
|     |     document.write("Cool, I am learning JavaScript and it
              works!!!");
|     }
|
}
```

Look out: when embedding command blocks inside each other, it is common practice to indent each successive command block so that it's is easier to identify where the block start and end when reading program code. Spaces and tabs do not have any effect on JavaScript programs.

2.6 Outputting Text

In JavaScript, output can be directed to several places including the current document window and pop-up dialog box.

In JavaScript, programmers can output text to the your browser in sequence with the HTML file. In the next section you will learn how to output text to the browser window.

2.6.1 The document.write() and document.writeln() Methods

The document object found in JavaScript includes two methods constructed for output text to the client windows: write() and writeln(). To use theses two method, they are called by combining the object name with the method name:

```
object.name.property.name
```

Data that these methods needs to perform this job is provided as an argument in the parentheses, for example:

```
document.write("JavaScript testing zone");
document.writeln("Cool, I am learning JavaScript and it          works!!!");
```

Look out:

1. *Arguments* are data provided to a function or a method for use in calculations and processing. They passed information to the function or method by listing them in the parentheses following the function or method name. Multiple arguments can be passed at once by having commas to separating them.

2. In the example, it also show you that the output text are surrounded by double quotes and both of the method are invoked in the same manner. Open and close quotes must be the same type, you cannot open with a double and close with a single quote.

INPUT: 2.2 Coding exercise – Outputting HTML tags from JavaScript .

```
<HTML>
<HEAD>
<TITLE> JavaScript Exercise 2 </TITLE>
</HEAD>
<BODY>
These are normal plain text.<BR>
<SCRIPT LANGUAGE = "JavaScript">
<!-- HIDE THE SCRIPT FORM OTHER BROWSERS

document.write("<B>This is how you can use HTML tags in JavaScript these text are bold</B>");

// STOP HIDING FORM OTHER BROWSERS -->
</SCRIPT>
</BODY>
</HTML>
```

Output

The result from the HTML file should produce a similar result to those in ***Fig 2.3***

Fig 2.3: HTML tags affect the output of JavaScript script

Analysis

The Script in the listing 2.2 demonstrates that HTML tags can also be use as part of JavaScript script.

INPUT: 2.3 Coding exercise – Outputting HTML tags from JavaScript .

```
<HTML>
```

```
<HEAD>
<TITLE> JavaScript Exercise 3 </TITLE>
<PRE>
<SCRIPT LANGUAGE = "JavaScript">
<!-- HIDE THE SCRIPT FORM OTHER BROWSERS

document.writeln("<CENTER><H1>Welcome to</H1>");
document.writeln("<H2>Informatics Computer School</H2><HR>");
document.writeln("JavaScript is fun!!!!!!!!!!!");
document.write("I am learning JavaScript, ");
document.write("and this is the third exercises.</CENTER>");
// STOP HIDING FORM OTHER BROWSERS -->
</SCRIPT>
</HEAD>
</HTML>
```

Output

This example produces an output like those in ***Fig 2.4***

Fig 2.4: Different output using the writeln() and write() method

Analysis

In JavaScript, strings of text, such as those used to produce output with the write() and writeln() methods, can include special keystrokes to represent characters that cannot be typed, such as new line, tabs, and carriage returns. The special characters are reviewed in following:

Characters	**Description**

\n	new line
\t	tab
\r	carriage return
\f	form feed
\b	backspace

In another words, to have a new line after a sentence instead of using HTML ”
” tags, you can use the ”\n” special characters to have a new line. For example to have a new after the line “JavaScript is fun”, you will have to:

```
document.write(“JavaScript if fun!!!\n”);
```

All special characters have to start with a backslash “\”. The backslash is called the escaping characters. Escaping characters are used in most programming languages to represents characters that cannot be typed. Like having a new line after a sentence.

Look out: In order to use the special characters you have to use an HTML tags, the <PRE></PRE> tags. You must enclose the entire special characters in the <PRE> tags container.

INPUT: 2.4 Coding exercise – Special Characters from JavaScript.

```
<HTML>
<HEAD>
<TITLE> JavaScript Exercise 4 </TITLE>
</HEAD>
<BODY>
<PRE>
<SCRIPT LANGUAGE = "JavaScript">
<!-- HIDE THE SCRIPT FORM OTHER BROWSERS

document.write('<CENTER><IMG SRC = "ICS.JPEG">');
document.write("<H2>Special Characters</H2></CENTER>");
document.write("<CENTER><H3>in JavaScript</H3></CENTER>");
document.write("<HR>");
document.write("In JavaScript \\n\n is to have a new line.\n");
document.write("<HR>");
document.write("In JavaScript \\t\t is to have a tap spacing.\n");
document.write("<HR>");
document.write("In JavaScript \\r\r is to have the cursors jump to next line");
document.write("<HR>");
document.write("In JavaScript \\f\f is to form feed the printer.\n");
document.write("<HR>");
document.write("In JavaScript \\b\b is to have a backspace.\n");
document.write("<HR>");
document.write("\"In JavaScript \\\" is to have the sentence quoted\"\n");

// STOP HIDING FORM OTHER BROWSERS -->
```

```
</SCRIPT>
</PRE>
</BODY>
</HTML>
```

Output

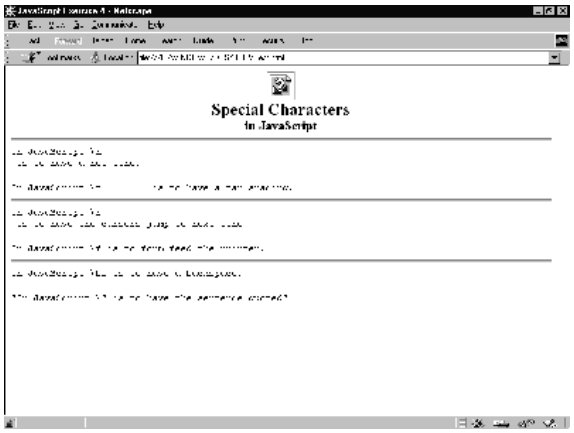


Fig 2.5: Special Characters that affect the output of JavaScript script

Analysis

In exercise 2.4, we have learned how to use special characters in JavaScript, and learned how to insert a image in JavaScript. It's just like normal HTML but it has to be enclose in the document.write() method.

document.write('<CENTER><IMG SRC = "ICS.JPEG">');

2.6 Outputting Text Beyond the Document Window

As a WEB site developer, you will realise that HTML has some limitation. One of the main restrictions of HTML is you will only have single client window for display result. Even with the frames, WEB designers are still limited, HTML files can only be display in complete browsers windows and unable to be more interactive like having dialog box. In JavaScript, you are able to direct messages to the user and get users input as well. In the next section you will see how you can output message via dialog box.

2.7.1 Working with Dialog Boxes

JavaScript provides the ability for programmers to generate output in small dialog boxes. The simplest way to direct output to a dialog box is to use the alert() method. To use the alert() method, all you need is to provide the message as what you did with document.write() and document.writeln() in the above sections:

INPUT: 2.5 Coding exercise – JavaScript alert.

```
<HTML>
<HEAD>
<TITLE> JavaScript Exercise 5 </TITLE>
</HEAD>
<BODY>
<PRE>
<SCRIPT LANGUAGE = "JavaScript">
<!-- HIDE THE SCRIPT FORM OTHER BROWSERS

alert("Welcome Informtics Computer School!!\n\n This is JavaScript Alert");
document.write('<CENTER><IMG SRC = "ICS.JPEG"></CENTER>');

// STOP HIDING FORM OTHER BROWSERS -->
</SCRIPT>
</PRE>
</BODY>
</HTML>
```

Output

The alert() command generates output similar to ***Fig 2.6***. The dialog box displays the message passed to it in parentheses, and an OK button. The script and HTML holding the script will not continue evaluating or executing until the user clicks the OK button.

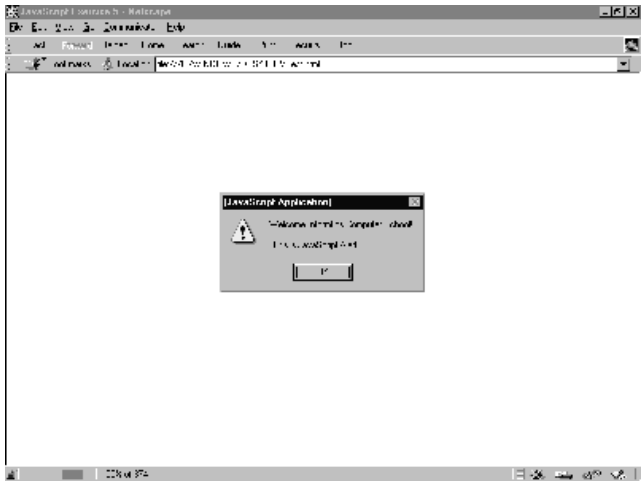


Fig 2.6: Show alert method

Generally, the alert() method is used for warn the users or to alert them to something. Examples are as follow:

1. Incorrect information input to a form
2. Invalid result for a calculation

3. Users clicks on the wrong button

Nevertheless, the alert() method can also be used for friendly messages.

2.8 Interacting with the User

The alert() method still does not allow you to interact with the user. The addition of the OK button only allow you to have some control over the timing of an event, but still cannot be used to generate any dynamic output or customise output based on the user input.

The simplest way to interact with the user is using the prompt() method. Like the alert(), prompt() method will also create a dialog box with the message you wanted, and it also provides a single line entry field. The user may fill in the fields and click OK. An example of prompt() method is as follow:

```
document.write(prompt("Please enter you name:","NAME"));
```

Have you notice the difference the prompt() with the way you used the alert() method: you are providing two strings to the method in the parentheses. The prompt() method requires two pieces of information: first is text displayed and second is the default data in the entry field.

Look out: The information provided in parentheses to a method or a function are know as arguments. In JavaScript, when a method requires more than one arguments, they are separated by commas.

INPUT: 2.6 Coding exercise – JavaScript prompt method.

```
<HTML>
<HEAD>
<TITLE> JavaScript Exercise 6 </TITLE>
</HEAD>
<BODY>
<PRE>
<SCRIPT LANGUAGE = "JavaScript">
<!-- HIDE THE SCRIPT FORM OTHER BROWSERS

document.write('<CENTER ><IMG SRC = "ICS.JPEG">\n');
document.write("<H1>Welcome to Informatics Computer School, ");
document.write(prompt("Please enter you name:","NAME"));
document.write("</H1><\CENTER>");
    // STOP HIDING FORM OTHER BROWSERS -->
</SCRIPT>
</PRE>
</BODY>
</HTML>
```

Output

The script will display the GIF in the Navigator window, follow by the sentence, *"Welcome to Informatics Computer School,"*. Then, a prompt dialog wills asks for the user's name and once it is entered, the sentence is completed and display with you name at the end.

Fig 2.7: Display the prompt dialog

Analysis

In exercise 2.6, we have learned how to used the prompt() method to construct a personalised welcome greeting. However, you should that the process was somewhat cumbersome, required four document.write() commands to display just a picture and a line of words. This can be done easier by combining multiple strings of text into single string of text using what is know as concatenation.

Look out: Concatenation will be discussed more in depth in the Chapter. Using concatenation means multiple strings are combined into a single string and are treated as a single string by JavaScript. To use concatenation in this script you will have to use a single document.write() command and a simple plus sign (+). For example:

INPUT: 2.7 Coding exercise – Using concatenation in JavaScript.

```
<HTML>
<HEAD>
<TITLE> JavaScript Exercise 7 </TITLE>
</HEAD>
<BODY>
<PRE>
<SCRIPT LANGUAGE = "JavaScript">
<!-- HIDE THE SCRIPT FORM OTHER BROWSERS

document.write('<CENTER ><IMG SRC = "ICS.JPEG">\n' +  "<H1>Welcome to Informatics Computer School, " +
prompt("Please enter you name:","NAME") + "</H1><\CENTER>");

// STOP HIDING FORM OTHER BROWSERS -->
```

```
</SCRIPT>
</PRE>
</BODY>
</HTML>
```

The result from this script will be show below.

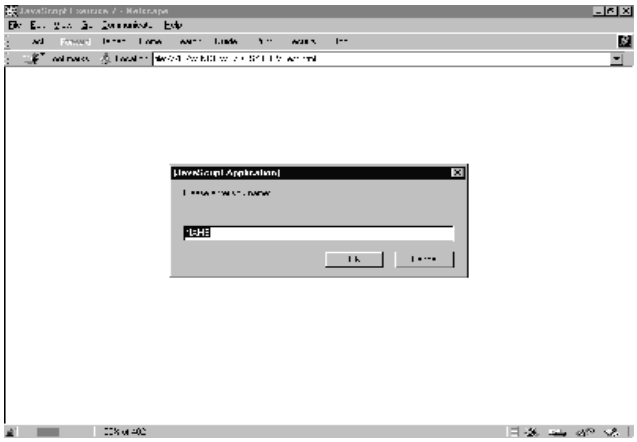


Fig 2.8: Display the prompt dialog

WORKING WITH DATA AND INFORMATION

At the end of this chapter, trainees will be able to:

- ❖ Learn about data types in JavaScript
- ❖ Understand the using and declaring variables
- ❖ Understand assignment expressions
- ❖ Learn about operators
- ❖ Learn about applying comparison: if – else constructs.
- ❖ Learn about extending user interaction with confirm() method

3.1 Data Types in JavaScript

JavaScript use four data types – numbers, strings, Boolean values, and a null value – to represent all information the language can handle. Compare to most other programming languages, this is a small number of data types, but it is sufficient to handle most of the data used in most programs except those very complex programs.

The four data types in JavaScript are discuss in the following:

JavaScript recognises the following types of values:

- Numbers, any number between 2.22E-308 … 1.79E+308, such as 15, 3.142, or 54e7.
- Logical (Boolean) values, either true or false.
- Strings, such as "Informatics Computer School Welcomes You!".
- null, a special keyword denoting a null value (it means nothing).

This relatively small set of types of values, or data types, enables you to perform useful functions with your applications. There is no explicit distinction between integer and real-valued numbers. Nor is there an explicit date data type in Navigator. However, you can use the Date object and its methods to handle dates.

Objects and functions are the other fundamental elements in the language. You can think of objects as named containers for values, and functions as procedures that your application can perform.

3.1.1 Numbers

The JavaScript number type encompasses what would be several types in languages such as Java. Using only the numbers data types, it is possible to declare both integers and floating point values.

3.1.1.1 Integers

Integers are numbers without any portion following the decimal point like whole numbers. Integers can be positive or negative numbers. The maximum integers size dependent on the platform running the JavaScript applications.

In JavaScript, integers can be expressed in decimal (base 10), hexadecimal (base 16), and octal (base 8).

A decimal integer literal consists of a sequence of digits without a leading 0 (zero). A leading 0 (zero) on an integer literal indicates it is in octal; a leading 0x (or 0X) indicates hexadecimal. Hexadecimal integers can include digits (0-9) and the letters a-f and A-F. Octal integers can include only the digits 0-7.

Some examples of integer literal are, for example, bb, 143, 33, 0xFFF, and -345.

3.1.1.2 Floating Point Values

Floating point values can include a factional component or a decimal point. Floating point can be positive or negative numbers.

In JavaScript, a floating-point literal can have the following parts: a decimal integer, a decimal point ("."), a fraction or decimal number, an exponent, and a type suffix. The exponent part is an "e" or "E" followed by an integer, which can be signed (preceded by "+" or "-"). A floating-point literal must have at least one digit, plus either a decimal point or "e" (or "E").

Some examples of floating-point literals are 1.470, -134.33e69, 0.1e12, and 2E3

Lookout: In JavaScript, when handling of floating point units will introduce inaccuracy in some calculations. You kept this in mind for your programs.

3.1.1.3 Boolean Literals

The Boolean type has two literal values: true and false. This type of literal comes in handy when computing data and making decisions, as we will see later in the book.

Look out: Unlike C, C++, Java and many others, in JavaScript Boolean values can only be represented with true and false. Values of 1and 0 are not recognises as Boolean values.

3.1.1.4 String Literals

A string literal is zero or more characters enclosed in double (") or single (') quotation marks. A string must be delimited by quotation marks of the same type; that is, either both single quotation marks or double quotation marks. The following are examples of string literals:

- "Welcome to Informatics Computer School"
- 'JavaScript is fun'
- "13433"
- " "
- "Cool, I am learning JavaScript and it works!!!"

In addition to ordinary characters, you can also include special characters in strings, as shown in the last chapter.

```
<HTML>
<HEAD>
<SCRIPT LANGUAGE="JavaScript">
var quote = "We are leaning \"JavaScript\" at Informatics Computer School."
document.write(quote)
</SCRIPT>
</HEAD>
</HTML>
```

The result of this would be

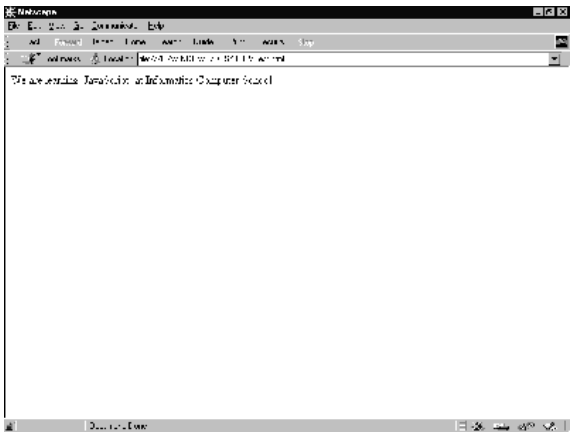


Fig 3.1: Showing String literals

To include a literal backslash inside a string, you must escape the backslash character. For example, to assign the file path c:\temp to a string, use the following:

```
var temp = "c:\\temp"
```

3.2 Data Type Conversion

JavaScript is a loosely typed language i. e. you do not have to specify the data type of a variable when you declare it, and data types are converted automatically as needed during script execution. So, for example, you could define a variable as follows:

```
var answer = 33
```

And later, you could assign the same variable a string value, for example,

```
answer = "Thanks you for taking courses with us..."
```

Because JavaScript is loosely typed, this assignment does not cause an error message.

In expressions involving numeric and string values, JavaScript converts the numeric values to strings. For example, consider the following statements:

```
x = "The answer to the question is " + 33
y = 33 + " is the answer the question."
```

The first statement returns the string "The answer to the question is 33." The second statement returns the string "33 is the answer the question."

For more information on these functions, see Chapter 4 & 5, "Built-in objects and functions."

JavaScript provides several special functions for manipulating string and numeric values:

- eval attempts to evaluate a string representing any JavaScript literals or variables, converting it to a number.
- parseInt converts a string to an integer of the specified radix (base), if possible.
- parseFloat converts a string to a floating-point number, if possible.

3.3 Variables

You use variables as symbolic names for values in your application. You give variables names by which you refer to them and which must conform to certain rules.

A JavaScript identifier, or name, must start with a letter or underscore ("_"); subsequent characters can also be digits (0-9). Because JavaScript is case sensitive, letters include the characters "A" through "Z" (uppercase) and the characters "a" through "z" (lowercase).

var varname [= value] [..., varname [= value]]

Some examples of legal names are:

4. "Funan_Centre",
5. "info99",
6. and "_world".

3.3.1 Declaring Variables

In order, to use a variable, you must declare a variable. Declaring a variable tells JavaScript that a variable of the given name exists so that the JavaScript interpreter can understand reference to that variable name throughout the rest of the script. A variable can be declared in two ways:

- By simply assigning it a value; for example, x = 143;
- With the keyword var; for example, var x = 143;

NB

When you set a variable identifier by assignment outside of a function, it is called a global variable, because it is available everywhere in the current document.

When you declare a variable within a function, it is called a local variable, because it is available only within the function. Using var is optional, but you need to use it if you want to declare a local variable inside a function that has already been declared as a global variable.

You can access global variables declared in one window or frame from another window or frame by specifying the window or frame name. For example, if a variable called phoneNumber is declared in a FRAMESET document, you can refer to this variable from a child frame as parent.phoneNumber.

3.3.2 Incorporating Variables in a Script

Using variables in a welcome program.

```
<HTML>
<HEAD>
<TITLE> Incorporating Variables in a Script </TITLE>
<SCRIPT LANGUAGE="JavaScript">
var name = prompt ("Enter your Name:", "name");
</SCRIPT>
</HEAD>
<BODY>
<SCRIPT LANGUAGE="JavaScript">
document.write("<H1>Greetings," + name + ". Welcome to Informatics Computer School.</H1>")
</SCRIPT>
</BODY>
</HTML>
```

The result of this would be

Fig 3.2: Incorporating Variables in a Script

3.4 Expressions

An expression is any valid set of literals, variables, operators, and expressions that evaluates to a single value. The value may be a number, a string, or a logical value. Conceptually, there are two types of

expressions: those that assign a value to a variable, and those that simply have a value. For example, the expression

```
x = 31
```

is an expression that assigns x the value 31. This expression itself evaluates to 31. Such expressions use assignment operators. On the other hand, the expression

```
30 + 1
```

simply evaluates to 31; it does not perform an assignment. The operators used in such expressions are referred to simply as operators.

JavaScript has the following kinds of expressions:

- Arithmetic: evaluates to a number, for example, 30+1
- String: evaluates to a character string, for example, "Easy" or "134"
- Logical: evaluates to true or false

The special keyword null denotes a null value. In contrast, variables that have not been assigned a value are undefined, and cannot be used without a run-time error.

3.5 Conditional Expressions

A conditional expression can have one of two values based on a condition.

The syntax is

```
(condition) ? val1 : val2
```

If condition is true, the expression has the value of val1, Otherwise it has the value of val2. You can use a conditional expression anywhere you would use a standard expression.

For example,

```
status = (100 >= 1000 ) ? "true" : "false"
```

This statement assigns the value "true" to the variable status if 100 is 1000 or greater. Otherwise, it assigns the value "false" to status.

*3.6 Assignment Operators (=, +=, -=, *=, /=)*

An assignment operator assigns a value to its left operand based on the value of its right operand. The basic assignment operator is equal (=), which assigns the value of its right operand to its left operand. That is, x = y assigns the value of y to x.

3.7 Operators

JavaScript has arithmetic, string, and logical operators. There are both binary and unary operators. A binary operator requires two operands, one before the operator and one after the operator:

- Operand1 operator operand2
  - For example, 3 + 3 or x * y

A unary operator requires a single operand, either before or after the operator:

- Operator operand or operand operator
  - For example x++ or ++x.

3.8 Arithmetic Operators

Arithmetic operators take numerical values (either literals or variables) as their operands and return a single numerical value.

3.8.1 Standard Arithmetic Operators

The standard arithmetic operators are addition (+), subtraction (-), multiplication (*), and division (/). These operators work in the standard way.

3.8.2 Modulus (%)

The modulus operator is used as follows:

```
var1 % var2
```

The modulus operator returns the first operand modulo the second operand, that is, var1 modulo var2, in the statement above, where var1 and var2 are variables. The modulo function is the remainder of integrally dividing var1 by var2. For example, 13 % 3 returns 1.

3.8.3 Increment (++)

The increment operator is used as follows:

```
var++ or ++var
```

This operator increments (adds one to) its operand and returns a value. If used postfix, with operator after operand (for example x++), then it returns the value before incrementing. If used prefix with operator before operand (for example, ++x), then it returns the value after incrementing.

For example, if x is 2, then the statement

```
y = x++
```

increments x to 3 and sets y to 2.

If x is 3, then the statement

```
y = ++x
```

increments x to 3 and sets y to 3.

3.8.4 Decrement (--)

The decrement operator is used as follows:

```
var-- or --var
```

This operator decrements (subtracts one from) its operand and returns a value. If used postfix (for example x--) then it returns the value before decrementing. If used prefix (for example, --x), then it returns the value after decrementing.

For example, if x is 3, then the statement

```
y = x--
```

decrements x to 2 and sets y to 3.

If x is 3, then the statement

```
y = --x
```

decrements x to 2 and sets y to 2.

Testing Using IF for Repetetion

Using the if statement, you are going to extend the “Testing User’s Response” exercise. You are going to enable the user to indicate if he or she wants a second chance to answer the question correctly.

What you want to do is ask the question and check the result. If the result is wrong, you will ask the user if he or she wishes to try again. If the user does, you ask one more time.

In order to make this easier, you will use the confirm() method, which is similar to the alert() and prompt() methods that you already know how to use. The confirm() method takes a single string as an argument. It display the string in a dialog box with OK and CANCEL buttons and returns a value of true if the user select OK or false if CANCEL is selected.

```
<HTML>
<HEAD>
```

```
<TITLE> Using if for repetition </TITLE>
<SCRIPT LANGUAGE="JavaScript">
<!-- HIDE FROM OTHER BROWSERS
// DEFINE VARIABLES FOR REST OF SCRIPT
var question="What is 20+20?";
var answer=40;
var right='<IMG SRC="right.gif">';
var wrong='<IMG SRC="wrong.gif">';
// ASK THE QUESTION
var response = prompt(question,"0");
// CHECK THE ANSWER THE FIRST TIME
if (response != answer) {
  // THE ANSWER WAS WRONG: OFFER A SECOND CHANCE
  if (confirm("Wrong! Press OK for a second chance."))
    response = prompt(question,"0");
}
// CHECK THE ANSWER
var output = (response == answer) ? right : wrong;
// STOP HIDING FROM OTHER BROWSERS -->
</SCRIPT>
</HEAD>
<BODY>
<SCRIPT LANGUAGE="JavaScript">
<!-- HIDE FROM OTHER BROWSERS
// OUTPUT RESULT
document.write(output);
// STOP HIDING FROM OTHER BROWSERS -->
</SCRIPT>
</BODY>
</HTML>
```

The result would look like Fig 3.6 prompting for user to input the answer. If the user enter, the right answer Fig 3.7 will be show. If the user enter the wrong answer Fig 3.8 will show and prompt the user for a second chance. If the user click on OK he or she will be given the second and the if the user enter the wrong answer Fig 3.9 will be display or if the right answer is input, Fig 3.7 will be displayed.

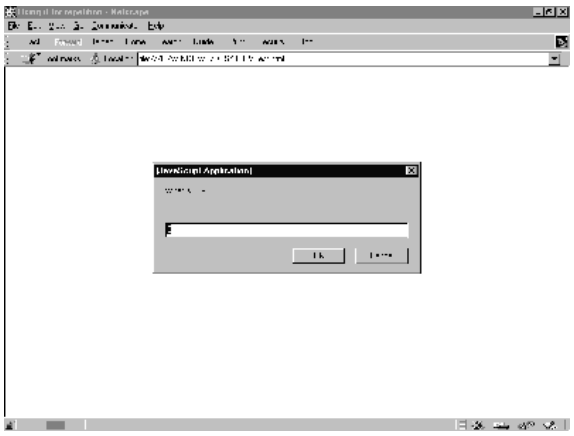


Fig 3.6: Prompt user to enter answer for question

Fig 3.7: Will be display if user enter the right answer for the question

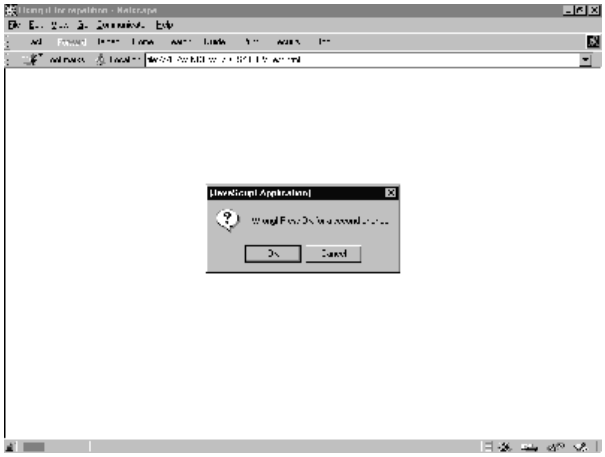


Fig 3.8: Will be display if user enter the wrong answer for the question

Fig 3.9: Will be display if user enter the wrong answer for the question

In order to add the second chance, you have to add only two if statements. In order to grasp how this works, lets look at the program line by line staring with the first prompt() method.

```
var response = prompt(question,"0");
```

In this line, you declare the variable response, ask the user to answer the question and assign the user's answer to the variable response.

```
if (response != answer)
```

In this line, we compare the user's respond to the variable answer that we assign our correct answer to. If the answer is incorrect, then the next line is executed. If the answer is correct, the program will skip down to output the result.

```
if (confirm("Wrong! Press OK for a second chance."))
```

If the user has made an incorrect response, then you check whether the user wants a second chance with confirm() method, which returns a Boolean values, which is evaluated by the if statement.

```
response = prompt(question,"0");
```

If the user selects OK in the confirm dialogs box, the confirm() method returns true, and this line executes. With this command, the user is again asked the question, and the second respond is stored in the respond variable, replace is stored in the response variable, replacing the previous answer.

FUNTIONS AND OBJECTS

4.2 Defining Functions

Functions are define using the functions statement. The function statement requires a name for the function, a list of parameters or arguments that will be passed to the function, and a command block that define what the function does:

Syntax

function *name*([*param*] [, *param*] [..., *param*]) {
    statements }

Examples

This function returns the total dollar amount of sales, when given the number of units sold of products x, y, and z.

```
function total_sales(units_x, units_y, units_z) {
    return units_x*16 + units_y*143 + units_z*33 }
```

It is important to realize that defining a function does not execute the commands that make up function. It is only when the function is called by name somewhere else in the script that the function is executed.

In the above function, you can see that total_sales accepts three argument called; units_x, units_y, units_z, Within the function, references to name refer to the value passed to the function.

4.3 RETURN

As mentioned in the previous section, functions can return result. Results are return using the return statements. The return statement can be used to return any valid expression that evaluates to single value.

Syntax

return expression

Examples

The following function returns the square of its argument, x, where x is a number.

```
function square( x ) {
    return x * x }
```

4.4 Putting Function to Work

To demonstrate the use of functions, you are going to rewrite the simple test question example you used in “3.17 Testing User’s Response”. In order to do this, you are going to create a function that receives a

question as an argument, poses the question, check the answer, and returns an output string based on accuracy of the user's response as shown in the following example.

```
<HTML>
<HEAD>
<TITLE>Putting Function to work</TITLE>

<SCRIPT LANGUAGE="JavaScript">
<!-- HIDE FROM OTHER BROWSERS
//DEFINE FUNCTION testQuestion()
function testQuestion(question) {
        //DEFINE LOCAL VARIABLES FOR THE FUNCTION
        var answer=eval(question);
        var output="What is " + question + "?";
        var correct='<IMG SRC="correct.gif">';
        var incorrect='<IMG SRC="incorrec.gif">';

        //ASK THE QUESTION
        var response=prompt(output,"0");

        //CHECK THE RESULT
        return (response == answer) ? correct : incorrect;
}

// STOP HIDING FROM OTHER BROWSERS -->
</SCRIPT>

</HEAD<

<BODY>

<SCRIPT LANGUAGE="JavaScript">
<!-- HIDE FROM OTHER BROWSERS

//ASK QUESTION AND OUTPUT RESULTS
var result=testQuestion("10 + 10");
document.write(result);

//STOP HIDING FROM OTHER BROWSERS -->
</SCRIPT>

</BODY>

</HTML>
```

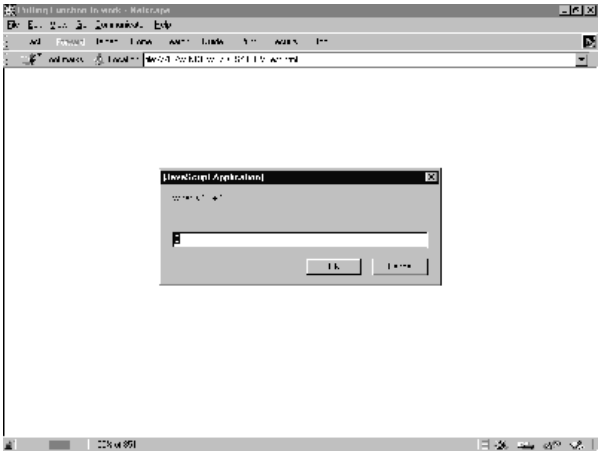


Fig 4.1: Putting Function to Work example

4.5 Recursive Function

Now you have seen an example of how functions work, let's take a look at an application of functions call recursion. For instance, the following is an example of a recursive function that calculates a factorial:

```
function factorial (number){
    if (number > 1)
        return number * factorial (number –1);
        } else {
            return number;
        }
    }
```

Example using recursive function in a program.

```
<HTML>

<HEAD>
<TITLE> recursive function </TITLE>

<SCRIPT LANGUAGE="JavaScript">
<!-- HIDE FROM OTHER BROWSERS
//DEFINE FUNCTION testQuestion()
function testQuestion(question) {
    //DEFINE LOCAL VARIABLES FOR THE FUNCTION
    var answer=eval(question);
    var output="What is " + question + "?";
    var right='<IMG SRC="right.gif">';
    var wrong='<IMG SRC="wrong.gif">';

    //ASK THE QUESTION
```

```
        var response=prompt(output,"0");

        //CHECK THE RESULT
        return (response == answer) ? correct :
           testQuestion(question);
}

// STOP HIDING FROM OTHER BROWSERS -->
</SCRIPT>

</HEAD<

<BODY>

<SCRIPT LANGUAGE="JavaScript">
<!-- HIDE FROM OTHER BROWSERS

//ASK QUESTION AND OUTPUT RESULTS
var result=testQuestion("100 + 100");
document.write(result);

//STOP HIDING FROM OTHER BROWSERS -->
</SCRIPT>

</BODY>

</HTML>
```

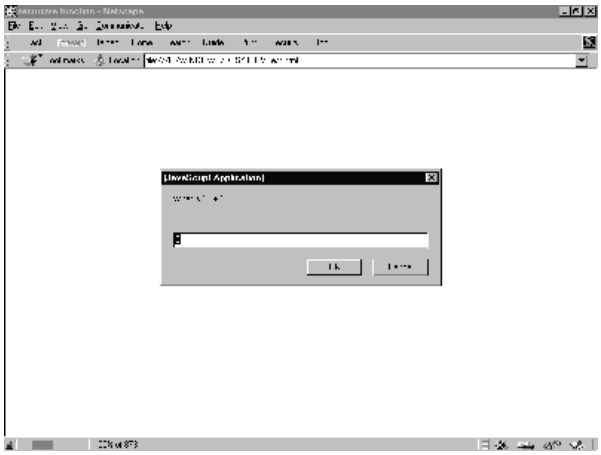


Fig 4-2: Recursive Function example

4.6 Building Object in JavaScript

It is possible to use function to build custom objects in JavaScript, to do this, you must be able to define an object's properties, to create new instances of objects, and to add methods to objects.

4.6.1 Defining an Object's Properties

Before creating a new object, it is necessary to define that object by outlining its properties. This is done by using a function that defines the name and properties of the function. This type of function is know as a constructor function. If you want to object type for student in class, you could create an object named student with properties for name, age, and grade. This could be done with the function:

```
function student ( name, age, grade ){
    this.name = name;
    this.age = age;
    this.grade = grade;
    }
```

4.6.2

This

Syntax

this[.propertyName]

Examples

JavaScript has a special keyword, this, that you can use to refer to the current object. For example, suppose you have a function called Check that validates an object's value property, given the object, and the max and min values:

```
function Check(obj, minvalue, maxvalue) {
    if ((obj.value < minvalue) || (obj.value > maxvalue))
    alert("Invalid Value!, <Br> Please Re-Enter The Value")
    }
```

Then, you could call Check in each form element's onChange event handler, using this to pass it the form element, as in the following example:

```
<INPUT TYPE = "text" NAME = "age" SIZE = 3 onChange= "Check(this, 18, 99)">
```

In general, in a method this refers to the calling object.

4.7 New

Is an operator that lets you create an instance of a user-defined object type.

Creating an object type requires two steps:

7. Define the object type by writing a function.

8. Create an instance of the object with new.

To define an object type, create a function for the object type that specifies its name, properties, and methods. An object can have a property that is itself another object. See the examples below.

You can always add a property to a previously defined object. For example, the statement car1.color = "black" adds a property color to car1, and assigns it a value of "black". However, this does not affect any other objects. To add the new property to all objects of the same type, you must add the property to the definition of the car object type.

Syntax

```
objectName = new objectType ( param1 [,param2] ...[,paramN] )
```

objectName is the name of the new object instance.

objectType is the object type. It must be a function that defines an object type.

param1...paramN are the property values for the object. These properties are parameters defined for the objectType function.

Examples

Example 1: object type and object instance.Suppose you want to create an object type for cars. You want this type of object to be called car, and you want it to have properties for make, model, year, and color. To do this, you would write the following function:

```
function car(make, model, year) {
    this.make = make
    this.model = model
    this.year = year
}
```

Now you can create an object called mycar as follows:

```
mycar = new car("Ford", "Mondeo", 1998)
```

This statement creates mycar and assigns it the specified values for its properties. Then the value of mycar.make is the string "Ford", mycar.year is the integer 1998, and so on.

You can create any number of car objects by calls to new. For example,

```
boycar = new car("Lotus", "GTO", 1995)
```

Example 2: object property that is itself another object. Suppose you define an object called person as follows:

```
function person(name, age, sex) {
    this.name = name
    this.age = age
    this.sex = sex
}
```

And then instantiate two new person objects as follows:

```
Boy = new person("Boy Bond", 27, "M")
Rich = new person("Richmond Wee", 26, "M")
```

Then you can rewrite the definition of car to include an owner property that takes a person object, as follows:

```
function car(make, model, year, owner) {
        this.make = make;
        this.model = model;
        this.year = year;
        this.owner = owner;
}
```

To instantiate the new objects, you then use the following:

```
car1 = new car("Ford", "Mondeo", 1998, Rich);
car2 = new car("Lotus", "GTO", 1995, Boy)
```

Instead of passing a literal string or integer value when creating the new objects, the above statements pass the objects Rich and Boy as the parameters for the owners. To find out the name of the owner of car2, you can access the following property:

```
car2.owner.name
```

4.8 Exercise for Object

```
<HTML>
<Script = JavaScript>

function grade(math, english, science){
        this.math = math;
        this.english = english;
        this.science = science;
        }

function student(name, age, grade){
        this.name = name;
        this.age = age;
        this.grade = grade;
        //this.mother = mother;
        this.displayProfile = displayProfile;
        }

function displayProfile(){
        document.write("Name : " + this.name + "<BR>");
        document.write("Age : " + this.age + "<BR>");
```

```
        document.write("Mother's Name : " + this.mother +
           "<BR>");
        document.write("Maths Grade : " + this.grade.math +
           "<BR>");
        document.write("English Grade : " + this.grade.english +
           "<BR>");
        document.write("Science Grade : " + this.grade.science +
           "<BR>");
        document.write("<BR>");
        }


    bobGrade = new grade(75, 80, 77);
    janeGrade = new grade(82, 88, 75);
    student1 = new student ("Bob", 10, bobGrade);
    student2 = new student ("Jane", 9, janeGrade);

    student1.mother = "Susan";

    student1.displayProfile();
    student2.displayProfile();
    </Script>
    </HTML>
```

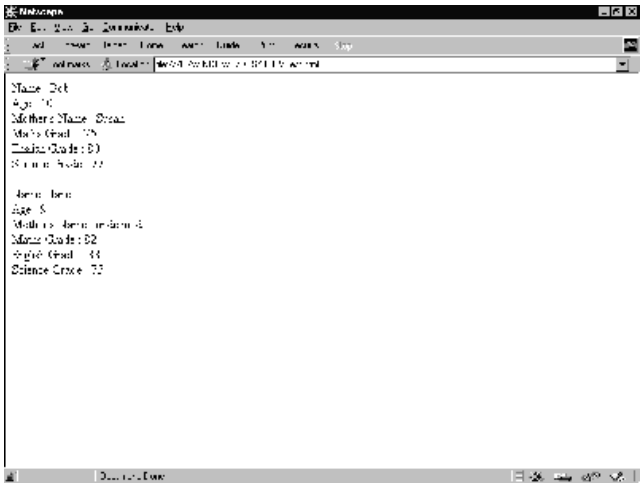


Fig 4-3: JavaScript object example

4.9 Object as Properties of Object

You can also use objects as properties of other object. For instance, if you were to create an object called grade

```
function grade(math, english, science){
        this.math = math;
        this.english = english;
        this.science = science;
        }
```

you could then create two instance of the grade object for the two students:

```
bobGrade = new grade(75, 80, 77);
janeGrade = new grade(82, 88, 75);
```

Using these objects, you could then create the student objects like this:

```
student1 = new student ("Bob", 10, bobGrade);
student2 = new student ("Jane", 9, janeGrade);
```

4.10 Adding Method to Object

Because methods are essentially functions associated with an object, first you need to create a function that defines the method you want to add to your object definition. For example if you want to print the student particular you will need to add a method to the student object. for example is show below.

```
function displayProfile(){
    document.write("Name : " + this.name + "<BR>");
    document.write("Age : " + this.age + "<BR>");
    document.write("Mother's Name : " + this.mother +
      "<BR>");
    document.write("Maths Grade : " + this.grade.math +
      "<BR>");
    document.write("English Grade : " + this.grade.english +
      "<BR>");
    document.write("Science Grade : " + this.grade.science +
      "<BR>");
    document.write("<BR>");
    }
```

Having define the method, you now need to change the object definition to include the method:

```
function student(name, age, grade){
    this.name = name;
    this.age = age;
    this.grade = grade;
    //this.mother = mother;
    this.displayProfile = displayProfile;
    }
```

then, you could output Bob's student profile by using the command:

```
student1.displayProfile();
```

USING OF BUILD-IN OBJECT AND FUNCTION

The JavaScript Language contains the following built-in objects and functions:

- String Object
- Math Object
- Date Object
- Build-in Functions

These objects and their properties and methods are built into the language. You can use these objects in both client applications with Netscape Navigator and server applications with LiveWire.

5.1 Using the String Object

Whenever you assign a string value to a variable or property, you create a string object. String literals are also string objects. For example, the statement

```
mystring = "Hello, World!"
```

creates a string object called mystring. The literal "Hello, World!" is also a string object.

The string object has methods that return:

- a variation on the string itself, such as *substring* and *toUpperCase*.
- an HTML formatted version of the string, such as *bold* and *link*.

For example, given the above object, mystring.toUpperCase() returns "HELLO, WORLD!", and so does "hello, world!".toUpperCase(). The following execise show how the string object is used.

```
<HTML>
<HEAD>
<TITLE>String Object Example</TITLE>
</HEAD>
<BODY>
<SCRIPT LANGUAGE="JavaScript">
<!-- HIDE FROM OTHER BROWSERS
var sample = "Informatics";
document.write("<BR>");
document.write(sample.italics());
document.write("<BR>");
document.write(sample. toUpperCase());
document.write("<BR>");
document.write(sample.link("http://www.ics-sin.com.sg"));
document.write("<BR>");
document.write(sample.fontsize(7));
document.write("<BR>");
document.write(sample.bold().strike());
```

```
document.write("<BR>");
document.write(sample.fontcolor("iceblue").big().sup());
// STOP HIDING FROM OTHER BROWSERS -->
</SCRIPT>
</BODY>
</HTML>
```

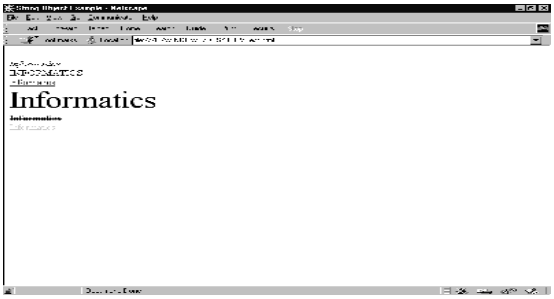


Fig 5-1: Result of the string object example

5.2 Using the Math Object

The built-in Math object has properties and methods for mathematical constants and functions. For example, the Math object's PI property has the value of pi, which you would use in an application as

- Math.PI

  Similarly, standard mathematical functions are methods of Math. These include trigonometric, logarithmic, exponential, and other functions. For example, if you want to use the trigonometric function sine, you would write

- Math.sin(1.56)

  Note that all trigonometric methods of math take arguments in radians.

  It is often convenient to use the with statement when a section of code uses several math constants and methods, so you don't have to type "Math" repeatedly. For example,

```
with (Math) {
    a = PI * r*r;
    y = r*sin(theta)
    x = r*cos(theta)
}
```

Using the Date Object

JavaScript does not have a date data type. However, the date object and its methods enable you to work with dates and times in your applications. The date object has a large number of methods for setting, getting, and manipulating dates. It does not have any properties.

To create a date object:

```
varName = new Date(parameters)
```

where *varName* is a JavaScript variable name for the date object being created; it can be a new object or a property of an existing object.

The *parameters* for the Date constructor can be any of the following:

- Nothing: creates today's date and time. For example, `today = new Date()`
- A string representing a date in the following form: "Month day, year hours:minutes:seconds". For example, `Xmas95= new Date("December 25, 1995 13:30:00")` If you omit hours, minutes, or seconds, the value will be set to zero.
- A set of integer values for year, month, and day. For example, `Xmas95 = new Date(95,11,25)`
- A set of values for year, month, day, hour, minute, and seconds For example, `Xmas95 = new Date(95,11,25,9,30,0)`

The Date object has a large number of methods for handling dates and times. The methods fall into these broad categories:

- "set" methods, for setting date and time values in date objects
- "get" methods, for getting date and time values from date objects
- "to" methods, for returning string values from date objects.
- parse and UTC methods, for parsing date strings.

The "get" and "set" methods enable you to get and set seconds, minutes, hours, day of the month, day of the week, months, and years separately. There is a getDay method that returns the day of the week, but no corresponding setDay method, because the day of the week is set automatically. These methods use integers to represent these values as follows:

- seconds and minutes: 0 to 59
- hours: 0 to 23
- day: 0 to 6 (day of the week)
- date: 1 to 31 (day of the month)
- months: 0 (January) to 11 (December)
- year: years since 1900

For example, suppose you define the following date:

```
Xmas95 = new Date("December 25, 1995")
```

Then `Xmas95.getMonth()` returns 11, and `Xmas95.getYear()` returns 95.

The getTime and setTime methods are useful for comparing dates. The getTime method returns the number of milliseconds since the epoch for a date object.

For example, the following code displays the number of shopping days left until Christmas:

```
<HTML>
<HEAD>
<TITLE>Date Object Example</TITLE>
<SCRIPT LANGUAGE="JavaScript">
<!-- HIDE FROM OTHER BROWSERS
today = new Date()
nextXmas = new Date("December 25, 1990")
nextXmas.setYear(today.getYear())
msPerDay = 24 * 60 * 60 * 1000 ; // Number of milliseconds per day
daysLeft = (nextXmas.getTime() - today.getTime()) / msPerDay;
daysLeft = Math.round(daysLeft);
document.write("Number of Shopping Days until Christmas: " + daysLeft);
// STOP HIDING FROM OTHER BROWSERS -->
</SCRIPT>
</HEAD>
</HTML>
```

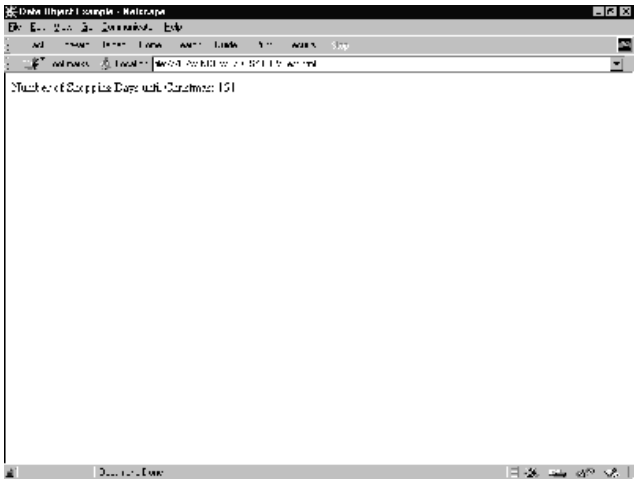


Fig 5-2: Result of the date object example

This example creates a date object named today that contains today's date. It then creates a date object named nextXmas, and sets the year to the current year. Then, using the number of milliseconds per day, it computes the number of days between today and nextXmas, using getTime, and rounding to a whole number of days.

The parse method is useful for assigning values from date strings to existing date objects. For example, the following code uses parse and setTime to assign a date to the IPOdate object.

```
IPOdate = new Date()
IPOdate.setTime(Date.parse("Aug 9, 1995"))
```

Example 1. The following example displays an alert message 5 seconds (5,000 milliseconds) after the user clicks a button. If the user clicks the second button before the alert message is displayed, the timeout is cancelled and the alert does not display.

```
<SCRIPT LANGUAGE="JavaScript">
function displayAlert() {
        alert("5 seconds have elapsed since the button was
           clicked.")
}
</SCRIPT>
<BODY>
<FORM>
Click the button on the left for a reminder in 5 seconds;
click the button on the right to cancel the reminder before
it is displayed.
<P>
<INPUT TYPE="button" VALUE="5-second reminder"
        NAME="remind_button"
        onClick="timerID=setTimeout('displayAlert()',5000)">
<INPUT TYPE="button" VALUE="Clear the 5-second reminder"
        NAME="remind_disable_button"
        onClick="clearTimeout(timerID)">
</FORM>
</BODY>
```

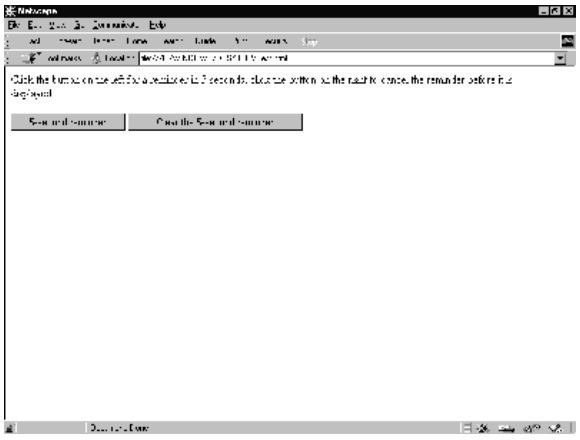


Fig 5-3: Result of the timer control example

Example 2. The following example displays the current time in a text object. The showtime() function, which is called recursively, uses the setTimeout method update the time every second.

```
<HEAD>
<SCRIPT LANGUAGE="JavaScript">
<!--
var timerID = null
var timerRunning = false
function stopclock(){
        if(timerRunning)
```

```
            clearTimeout(timerID)
        timerRunning = false
}
function startclock(){
        // Make sure the clock is stopped
        stopclock()
        showtime()
}
function showtime(){
        var now = new Date()
        var hours = now.getHours()
        var minutes = now.getMinutes()
        var seconds = now.getSeconds()
        var timeValue = "" + ((hours > 12) ? hours - 12 : hours)
        timeValue  += ((minutes < 10) ? ":0" : ":") + minutes
        timeValue  += ((seconds < 10) ? ":0" : ":") + seconds
        timeValue  += (hours >= 12) ? " P.M." : " A.M."
        document.clock.face.value = timeValue
timerID = setTimeout("showtime()",1000)
        timerRunning = true
}
//-->
</SCRIPT>
</HEAD>

<BODY onLoad="startclock()">
<FORM NAME="clock" onSubmit="0">
        <INPUT TYPE="text" NAME="face" SIZE=12 VALUE ="">
</FORM>
</BODY>
```

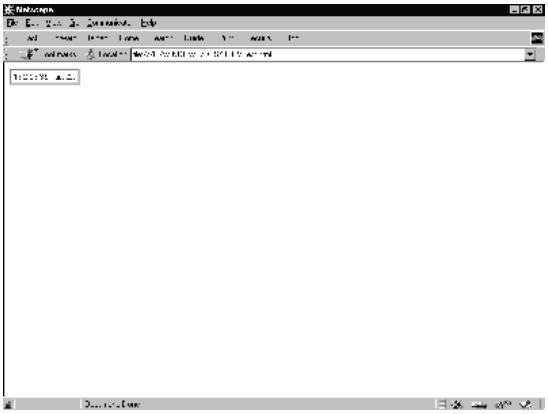


Fig 5-4: Result of the online Clock example

5.3 Using Built-in Functions

JavaScript has several "top-level" functions built-in to the language. They are:

- eval
- parseInt
- parseFloat

5.4.1 The eval Function

The built-in function *eval* takes a string as its argument. The string can be any string representing a JavaScript expression, statement, or sequence of statements. The expression can include variables and properties of existing objects.

If the argument represents an expression, eval evaluates the expression. If the argument represents one or more JavaScript statements, eval performs the statements.

This function is useful for evaluating a string representing an arithmetic expression. For example, input from a form element is always a string, but you often want to convert it to a numerical value.

The following example takes input in a text field, applies the eval function and displays the result in another text field. If you type a numerical expression in the first field, and click on the button, the expression will be evaluted. For example, enter "(666 * 777) / 3", and click on the button to see the result.

```
<SCRIPT>
function compute(obj) {
        obj.result.value = eval(obj.expr.value)
}
</SCRIPT>
<FORM NAME="evalform">
Enter an expression: <INPUT TYPE=text NAME="expr" SIZE=20 >
<BR>
Result: <INPUT TYPE=text NAME="result" SIZE=20 >
<BR>
<INPUT TYPE="button" VALUE="Click Me" onClick="compute(this.form)">
</FORM>
```

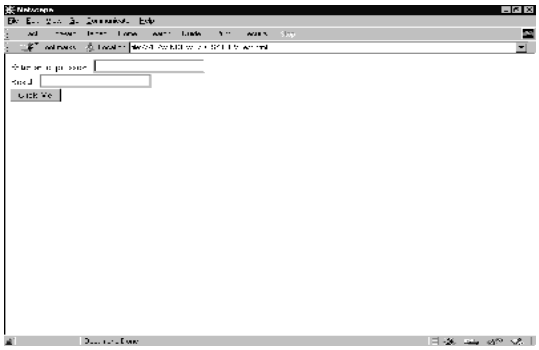


Fig 5-5: Result of the eval Function example

The eval function is not limited to evaluating numerical expressions, however. Its argument can include object references or even JavaScript statements. For example, you could define a function called setValue that would take two arguments: and object and a value, as follows:

```
function setValue (myobj, myvalue) {
        eval ("document.forms[0]." + myobj + ".value") = myvalue;
}
```

Then, for example, you could call this function to set the value of a form element "text1" as follows:

```
setValue(text1, 42)
```

5.4.2 The parseInt and parseFloat Functions

These two built-in functions return a numeric value when given a string as an argument.

ParseFloat parses its argument, a string, and attempts to return a floating point number. If it encounters a character other than a sign (+ or -), numeral (0-9), a decimal point, or an exponent, then it returns the value up to that point and ignores that character and all succeeding characters. If the first character cannot be converted to a number, it returns *NaN.*

The parseInt function parses its first argument, a string, and attempts to return an integer of the specified radix (base). For example, a radix of 10 indicates to convert to a decimal number, 8 octal, 16 hexadecimal, and so on. For radixes above 10, the letters of the alphabet indicate numerals greater than 9. For example, for hexadecimal numbers (base 16), A through F are used.
If parseInt encounters a character that is not a numeral in the specified radix, it ignores it and all succeeding characters and returns the integer value parsed up to that point. If the first character cannot be converted to a number in the specified radix, it returns *NaN.* ParseInt truncates numbers to integer values.

Question that Probably Ponder in your Mind

9. Using the date object example. It show that the number of days left to this year Xmas. Can you enhance the example to display day left to year 2000?

10. What would the output of the following code segment look like assuming there were no HTML tags elsewhere in the file affecting the output?

```
var sample = “test.”;
sample.big();
sample.blod();
sample.strike();
sample.fonesize(7);
document.write(sample.italics());
```

EVENTS HANDLERS IN JAVA SCRIPT

The following event handlers are available in JavaScript:

6.1 onBlur Event Handler

A blur event occurs when a select, text, or textarea field on a form loses focus. The onBlur event handler executes JavaScript code when a blur event occurs.

Event Handler of

- select, text, textarea

Examples

In the following example, *userName* is a required text field. When a user attempts to leave the field, the onBlur event handler calls the required() function to confirm that *userName* has a legal value.

Syntax

```
<INPUT TYPE="text" VALUE="" NAME="userName" onBlur="required(this.value)">
```

6.2 onChange Event Handler

A change event occurs when a select, text, or textarea field loses focus and its value has been modified. The onChange event handler executes JavaScript code when a change event occurs.

Use the onChange event handler to validate data after it is modified by a user.

Event Handler of

- select, text, textarea

Examples

In the following example, *userName* is a text field. When a user attempts to leave the field, the onBlur event handler calls the checkValue() function to confirm that *userName* has a legal value.

onChange syntax.

```
<INPUT TYPE="text" VALUE="" NAME="userName" onBlur="checkValue(this.value)">
```

6.3 onClick Event Handler

A click event occurs when an object on a form is clicked. The onClick event handler executes JavaScript code when a click event occurs.

Event Handler of

- button, checkbox, radio, link, rest, submit

Examples

For example, suppose you have created a JavaScript function called compute(). You can execute the compute() function when the user clicks a button by calling the function in the onClick event handler, as follows:

onClick syntax.

```
<INPUT TYPE="button" VALUE="Calculate" onClick="compute(this.form)">
```

In the above example, the keyword *this* refers to the current object; in this case, the Calculate button. The construct *this.form* refers to the form containing the button.

example,

Suppose you have created a JavaScript function called pickRandomURL() that lets you select a URL at random. You can use the onClick event handler of a link to specify a value for the HREF attribute of the <A> tag dynamically, as shown in the following example: **Error! Hyperlink reference not valid.**

In the above example, the onMouseOver event handler specifies a custom message for the Navigator status bar when the user places the mouse pointer over the Go! anchor. As this example shows, you must return true to set the window.status property in the onMouseOver event handler.

6.4 onFocus Event Handler

A focus event occurs when a field receives input focus by tabbing with the keyboard or clicking with the mouse. Selecting within a field results in a select event, not a focus event. The onFocus event handler executes JavaScript code when a focus event occurs.

See the relevant objects for the onFocus syntax.

Event Handler of

- select, text, textarea

Examples

The following example uses an onFocus handler in the *valueField* textarea object to call the valueCheck() function.

onClick syntax.

```
<INPUT TYPE="textarea" VALUE="" NAME="valueField" onFocus="valueCheck()">
```

6.5 onLoad Event Handler

A load event occurs when Navigator finishes loading a window or all frames within a <FRAMESET>. The onLoad event handler executes JavaScript code when a load event occurs.

Use the onLoad event handler within either the <BODY> or the <FRAMESET> tag, for example, <BODY onLoad="...">.

In a <FRAMESET> and <FRAME> relationship, an onLoad event within a frame (placed in the <BODY> tag) occurs before an onLoad event within the <FRAMESET> (placed in the <FRAMESET> tag).

Event Handler of

- window

Examples

In the following example, the onLoad event handler displays a greeting message after a web page is loaded.

<BODY onLoad="window.alert("Welcome to the Brave New World home page!")>

6.5.1 onMouseOver Event Handler

A mouseOver event occurs once each time the mouse pointer moves over an object from outside that object. The onMouseOver event handler executes JavaScript code when a mouseOver event occurs.

You must return true within the event handler if you want to set the status or defaultStatus properties with the onMouseOver event handler.

See the relevant objects for the onMouseOver syntax.

Event Handler of

- link

Examples

By default, the HREF value of an anchor displays in the status bar at the bottom of the Navigator when a user places the mouse pointer over the anchor. In the following example, the onMouseOver event handler provides the custom message "Click this if you dare."Click me.

See onClick for an example of using onMouseOver when the <A> tag's HREF attribute is set dynamically.

6.7 onSelect Event Handler

A select event occurs when a user selects some of the text within a text or textarea field. The onSelect event handler executes JavaScript code when a select event occurs.

Event Handler of

- text, textarea

Examples

The following example uses an onSelect handler in the *valueField* text object to call the selectState() function.

onSelect syntax.

```
<INPUT TYPE="text" VALUE="" NAME="valueField" onSelect="selectState()">
```

6.8 onSubmit Event Handler

A submit event occurs when a user submits a form. The onSubmit event handler executes JavaScript code when a submit event occurs.

You can use the onSubmit event handler to prevent a form from being submitted; to do so, put a **return** statement that returns false in the event handler. Any other returned value lets the form submit. If you omit the **return** statement, the form is submitted.

Event Handler of

- form

Examples

In the following example, the onSubmit event handler calls the formData() function to evaluate the data being submitted. If the data is valid, the form is submitted; otherwise, the form is not submitted.

onSubmit syntax.

```
form.onSubmit="return formData(this)"
```

6.9 onUnload Event Handler

An unload event occurs when you exit a document. The onUnload event handler executes JavaScript code when an unload event occurs.

Use the onUnload event handler within either the <BODY> or the <FRAMESET> tag, for example, <BODY onUnload="...">.

In a <FRAMESET> and <FRAME> relationship, an onUnload event within a frame (placed in the <BODY> tag) occurs before an onUnload event within the <FRAMESET> (placed in the <FRAMESET> tag).

Event Handler of

window

Examples

In the following example, the onUnload event handler calls the cleanUp() function to perform some shut down processing when the user exits a web page:

```
<BODY onUnload="cleanUp()">
```

6.10 xample on Event Handlers

6.10.1 Example for onLoad and onUnload

```
<HTML>
<HEAD>
<TITLE> onLoad and onUnload </TITLE>
<SCRIPT LANGUAGE="JavaScript">

<!-- HIDE FROM OTHER BROWSERS

// DEFINE GLOBAL VARIABLE

var name = "";

function hello() {
        name = prompt('Enter Your Name:','Name');
        alert('Greetings ' + name + ', welcome to my page!');
}

function goodbye() {
        alert('Goodbye ' + name + ', sorry to see you go!');
}

// STOP HIDING FROM OTHER BROWSERS -->
</SCRIPT>
</HEAD>
<BODY onLoad="hello();"  onUnload="goodbye();">
</BODY>
</HTML>
```

Result from the above script,

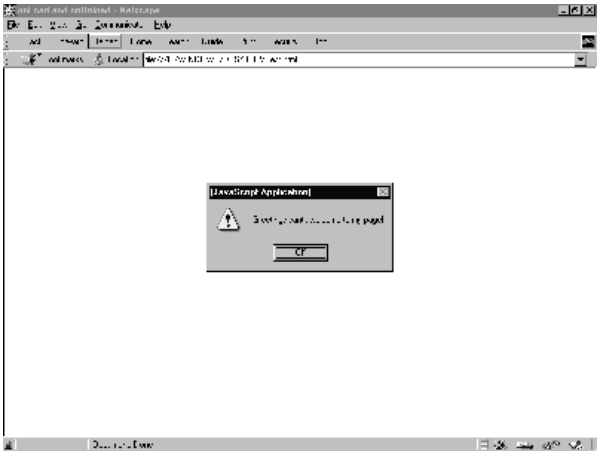


Fig 6-1: onLoad and onUnload example

more Example for onLoad and onUnload

```
<HTML>
<HEAD>
<TITLE> more  on onLoad and onUnload </TITLE>
<SCRIPT LANGUAGE="JavaScript">
<!-- HIDE FROM OTHER BROWSERS
function urlList(a,b,c,d,e) {
        // DEFINE FIVE-ELEMENT OBJECT
        this[0] = a;
        this[1] = b;
        this[2] = c;
        this[3] = d;
        this[4] = e;
}
function selectPage(list) {
        // SELECT RANDOM PAGE
        var today = new Date();
        var page = today.getSeconds() % 5;
        // OPEN PAGE
        window.open(list[page],"Random_Page");
}
// DEFINE SELECTION LIST
choices = new urlList("http://www.yahoo.com",
                "http://www.cnn.com",
                "http://www.dataphile.com.hk",
                "http://home.netscape.com",
                "http://www.landegg.org/landegg");

// STOP HIDING FROM OTHER BROWSERS -->
</SCRIPT>
```

```
</HEAD>
<BODY onLoad = "selectPage(choices);">
<H1>
<HR>
Please Wait ... Selecting Page.
<HR>
</H1>
</BODY>

</HTML>
```

Result from the above script,

Fig 6-2: More onLoad and onUnload example

Example for blur event

```
<HTML>

<HEAD>
<TITLE> blur event </TITLE>

<SCRIPT LANGUAGE="JavaScript">
<!-- HIDE FROM OTHER BROWSERS

function calculate(form) {
        form.results.value = eval(form.entry.value);

}

function getExpression(form) {
        form.entry.blur();
        form.entry.value = prompt("Please enter a JavaScript
           mathematical expression","");
        calculate(form);
```

```
}


// STOP HIDING FROM OTHER BROWSERS -->
</SCRIPT>

</HEAD>

<BODY>
<FORM METHOD=POST>
Enter a JavaScript mathematical expression:
<INPUT TYPE=text NAME="entry" VALUE=""
            onFocus="getExpression(this.form);">
<BR>
The result of this expression is:
<INPUT TYPE=text NAME="results" VALUE=""
      onFocus="this.blur();">

</FORM>
</BODY>
</HTML>
```

Result from the above script,

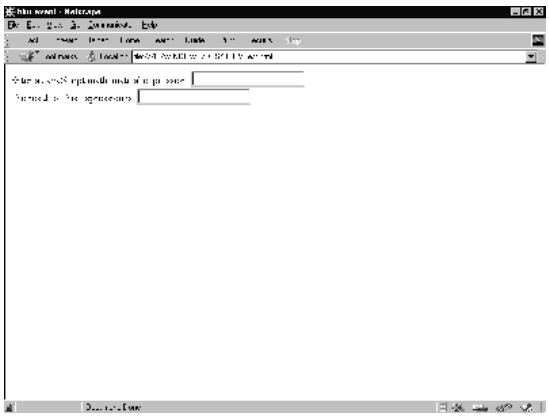


Fig 6-3: Blur event

Example for Mouse Over Text - Alert

Want to annoy someone? Move your mouse over this to see how. This is how it's done. Add the follow script to you page.

```
<a href="" onMouseOver="alert('YOUR MESSAGE');return true;">The highlighted text goes here</a>
```

Example for Mouse Over Image - Alert

Want to annoy someone using an image? Move your mouse over the picture. Just copy the purple stuff

```
<a href="" onMouseOver="alert('YOUR MESSAGE');return true;"><IMG SRC="YOUR IMAGE FILE" border="0"></a>
```

Example for Mouse Over Text - New Window

Put your mouse over THIS to make a new window pop up. Put these lines in your head tag.

```
<SCRIPT Language='JavaScript'>
function winopen () {
msg=open("","NewWindow","toolbar=no,location=no,directories=no,status=no,menubar=no,scrollbars=no,resizable=no,copyhistory=yes,width=400,height=400");
msg.location = "YOUR LINK LOCATION"
}
</SCRIPT>
```

This is the text that gets highlighted. put the following in the body.

```
<a href="" onMouseOver="winopen(); return true;">TEXT GOES HERE</a>
```

Question that Probably Ponder in your Mind

1.0 Create an HTML pages and JavaScript script that include a form with three input field. the relationship of the value of the filed is that the second field is twice the value of the of the first field and the third field is square of the first field.

If the user enters a value in the second or third field, the script should calculate the appropriate value in the fields.

7.0 INTRODUCTION

The form object is one of the most heavily used objects in JavaScript written for Navigator. As a programmer, by using the form object.

7.1 Form Object (forms array)

Lets users input text and make choices from form objects such as checkboxes, radio buttons, and selection lists. You can also use a form to post data to a server.

Syntax

To define a form, use standard HTML syntax with the addition of the onSubmit event handler:

```
<FORM
    NAME="formName"
    TARGET="windowName"
    ACTION="serverURL"
    METHOD=GET | POST
    ENCTYPE="encodingType"
     [onSubmit="handlerText"]>
</FORM>
```

NAME="formName" specifies the name of the form object.

TARGET="windowName" specifies the window that form responses go to. When you submit a form with a TARGET attribute, server responses are displayed in the specified window instead of the window that contains the form. *windowName* can be an existing window; it can be a frame name specified in a <FRAMESET> tag; or it can be one of the literal frame names _top, _parent, _self, or _blank; it cannot be a JavaScript expression (for example, it cannot be parent.*frameName* or *windowName.frameName*). Some values for this attribute may require specific values for other attributes. You can access this value using the target property.

ACTION="serverURL" specifies the URL of the server to which form field input information is sent. This attribute can specify a CGI or LiveWire application on the server; it can also be a mailto: URL if the form is to be mailed. Some values for this attribute may require specific values for other attributes. You can access this value using the action property.

METHOD=GET | POST specifies how information is sent to the server specified by *ACTION*. GET (the default) appends the input information to the URL which on most receiving systems becomes the value of the environment variable *QUERY_STRING*. POST sends the input information in a data body which is available on *stdin* with the data length set in the environment variable

CONTENT_LENGTH. Some values for this attribute may require specific values for other attributes. You can access this value using the method property.

ENCTYPE="encodingType" specifies the MIME encoding of the data sent: "application/x-www-form-urlencoded" (the default) or "multipart/form-data". Some values for this attribute may require specific values for other attributes. You can access this value using the encoding property.

To use a form object's properties and methods:

11. formName.propertyName

12. formName.methodName(parameters)

13. forms[index].propertyName

14. forms[index].methodName(parameters)

formName is the value of the NAME attribute of a form object.
propertyName is one of the properties listed below.
methodName is one of the methods listed below.
index is an integer representing a form object.

Description

Each form in a document is a distinct object.

You can reference a form's elements in your code by using the element's name (from the NAME attribute) or the elements array. The *elements* array contains an entry for each element (such as a checkbox, radio, or text object) in a form.

7.2 The Forms Array

You can reference the forms in your code by using the forms array (you can also use the form name). This array contains an entry for each form object (<FORM> tag) in a document in source order. For example, if a document contains three forms, these forms are reflected as document.forms[0], document.forms[1], and document.forms[2].

To use the *forms* array:

15. document.forms[*index*]

16. document.forms.length

index is an integer representing a form in a document.

To obtain the number of forms in a document, use the length property: document.forms.length.

You can also refer to a form's elements by using the forms array. For example, you would refer to a text object named *quantity* in the second form as document.forms[1].quantity.

You would refer to the value property of this text object as

document.forms[1].quantity.value.

Elements in the forms array are read-only. For example, the statement

document.forms[0]="music" has no effect.

The value of each element in the *forms* array is <object *nameAttribute*>, where *nameAttribute* is the NAME attribute of the form.

Properties

The form object has the following properties: action reflects the ACTION attribute elements is an array reflecting all the elements in a form encoding reflects the ENCTYPE attribute length reflects the number of elements on a form method reflects the METHOD attribute target reflects the TARGET attribute.

The *forms* array has the following properties: length reflects the number of forms in a document

Methods

- submit

Event handlers

- onSubmit

Property of

- document

Examples

Example 1: named form. The following example creates a form called form1 that contains text fields for first name and last name. The form also contains two buttons that change the names to all upper case or all lower case. The function setCase shows how to refer to the form by its name.

```
function setCase (caseSpec){ if (caseSpec == "upper") {
      document.form1.firstName.value=document.form1.firstNa
        me.value.toUpperCase()    document.form1.lastName.value=document.form1.lastName.
        value.toUpperCase()}
else {
      document.form1.firstName.value=document.form1.firstNa
        me.value.toLowerCase()    document.form1.lastName.value=document.form1.lastName.
        value.toLowerCase()} }
```

Example 2: onSubmit event handler. The following example shows an onSubmit event handler that determines whether to submit a form. The form contains one text object where the user enters three characters. The onSubmit event handler calls a function, *checkData*, that returns true if the number of characters is three; otherwise, it returns false. Notice that the form's onSubmit event handler, not the submit button's onClick event handler, calls the *checkData* function. Also, the onSubmit event handler contains a return statement that returns the value obtained with the function call.

```
var dataOK=false

function checkData (){
    if (document.form1.threeChar.value.length == 3)
        {
            return true}
            else {
          alert("Enter exactly three characters. " +    document.form1.threeChar.value + " is not valid.")
          return false
                    }
    }
```

Example 3: submit method. The following example is similar to the previous one, except it submits the form using the submit method instead of a submit object. The form's onSubmit event handler does not prevent the form from being submitted. The form uses a button's onClick event handler to call the *checkData* function. If the value is valid, the *checkData* function submits the form by calling the form's submit method.

```
var dataOK=false

function checkData (){
    if (document.form1.threeChar.value.length == 3)
        {
        document.form1.submit()
        }
        else {
          alert("Enter exactly three characters. " +    document.form1.threeChar.value + " is not valid.")
          return false
    }
}
```

7.3 Radio Button Alert

Click on one of the radio buttons to see a message

Here's the code for this.

```
<FORM>
<p>
1:
<INPUT TYPE="radio" NAME="radio" value="YOUR MESSAGE GOES HERE"
onClick="alert(value)">
2:
<INPUT TYPE="radio" NAME="radio" value="YOUR MESSAGE GOES HERE"
onClick="alert(value)">
```

3:

```
<INPUT TYPE="radio" NAME="radio" value="YOUR MESSAGE GOES HERE"
onClick="alert(value)">
</form>
```

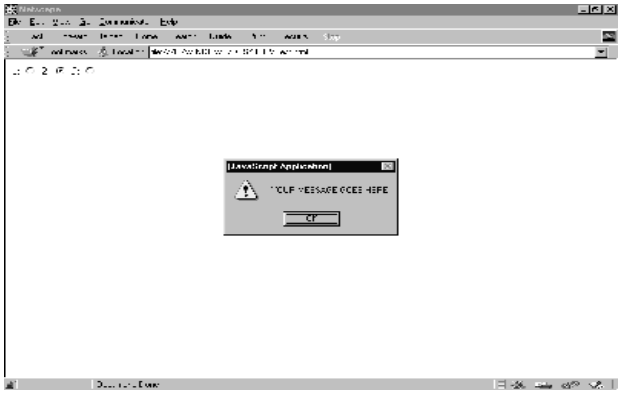


Fig 7-1: Radio Button Alert example

7.4 Button Alert

Push this button to make a message appear.

Here's the code.

```
<form><input type="button" value="message" onClick="alert('this is the message'); return true"></form>
```

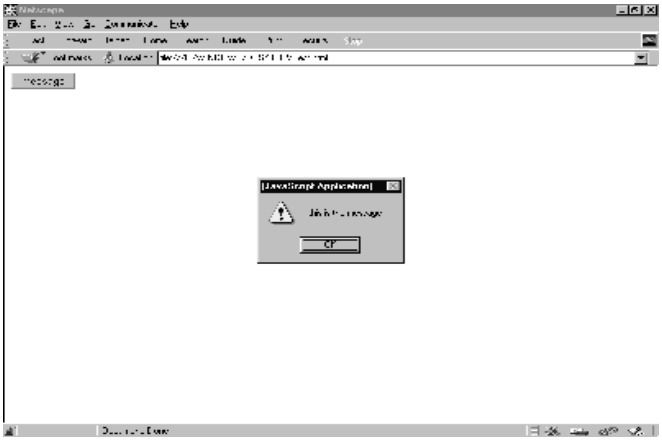


Fig 7-2: Button Alert example

7.5 Link Alert

When you click on a link like this, an alert will pop up and then you will be taken to the link destination. Give it a shot!

To make use of this script, copy the following lines of code.

<A HREF="LINK GOES HERE" onClick="alert('YOUR MESSAGE GOES HERE')">YOUR LINK DESCRIPTION ***GOES HERE</A>***

Fig 7-3: Link Alert example

7.6 Link Confirm/ Cancel Alert

Click on the link to see this example. CLICK ME!!

Here how it works. Just put this part in your HEAD tag and then change YOUR CONFIRM MESSAGE and YOUR LINK GOES HERE.

```
<script>
function rusure(){
        question = confirm("YOUR CONFIRM MESSAGE")
        if (question !="0"){
              top.location = "YOUR LINK GOES HERE"
        }
}
</script>
```

Now put this anywhere in your page and change YOUR LINK DESCRIPTION

```
<a href="" onClick="rusure(); return false;">YOUR LINK DESCRIPTION</a>
```

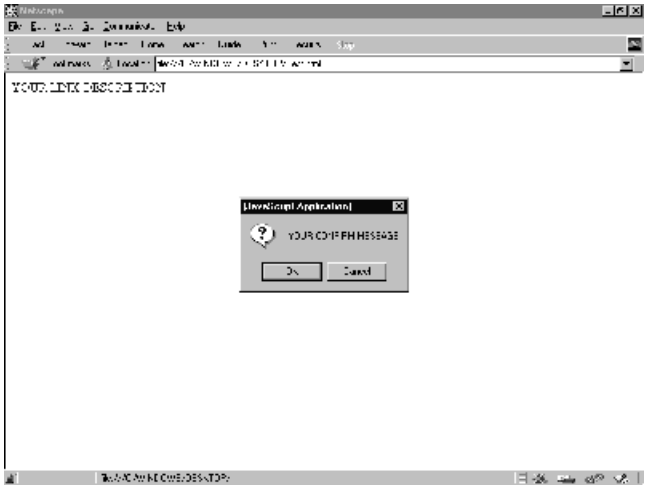


Fig 7-4: Link Confirm/Cancel Alert example

7.7 Pull Down Surfing

Put this wherever you want in your document.

```
<script language="JavaScript">
<!-- Hide the script from old browsers --
 function surfto(form) {
                    var myindex=form.dest.selectedIndex
                    location=form.dest.options[myindex].value;
}
//-->
</SCRIPT>
<FORM NAME="myform">
      <SELECT NAME="dest" SIZE=1>
           <OPTION SELECTED VALUE="http://URL#1">URL #1
                DESCRIPTION
           <OPTION VALUE="http://URL#2">URL #2
                DESCRIPTION
           <OPTION VALUE="http://URL#3">URL #3
                DESCRIPTION
           <OPTION VALUE="http://URL#4">URL#4
                DESCRIPTION
           </SELECT>
<INPUT TYPE="BUTTON" VALUE="GO NOW!" onClick="surfto(this.form)">
</FORM>
```

Don't forget to change the URLs and the descriptions!!!

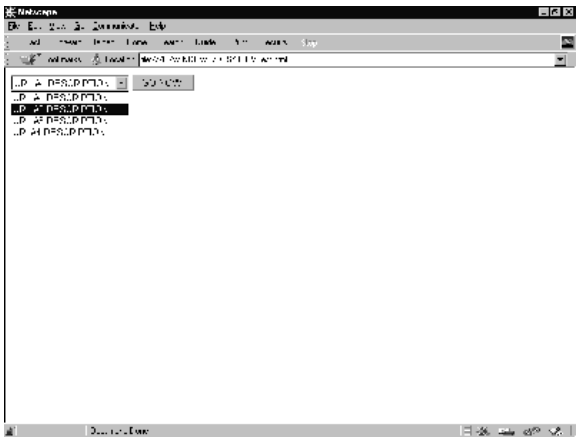


Fig 7-5: Pull Down Surfing example

Question that Probably Ponder in your Mind

17. Which of these HTML tags are valid?

    a. <FORM METHOD=POST onClick= ”go() ;” >

    b. <BODY onLOAD= ”go() ;” >

    c. <INPUT TYPE=text onChange= ”go() ;” >

    d. <INPUT TYPE=checkbox onChange= ”go() ;” >

    e. <BODY onUnload= ”go() ; “ >

    f. <INPUT TYPE=text onClick= ”go() ; “ >

LOOP

JavaScript statements consist of keywords used with the appropriate syntax. A single statement may span multiple lines. Multiple statements may occur on a single line if each statement is separated by a semi-colon.

Syntax conventions: All keywords in syntax statements are in bold. Words in italics represent user-defined names or statements. Any portions enclosed in square brackets, [ ], are optional. {*statements*} indicates a block of statements, which can consist of a single statement or multiple statements delimited by a curly braces {}.

The following statements are available in JavaScript:

- basic concept of loops
- the for and for …….. in loop
- the while loop
- the break and continue statement

8.1 BREAK

A statement that terminates the current **while** or **for** loop and transfers program control to the statement following the terminated loop.

Syntax

break

Examples

The following function has a **break** statement that terminates the **while** loop when *i* is 3, and then returns the value 3 * *x*. function func(x) { var i = 0 while (i < 6) { if (i == 3) break i++ } return i*x }

8.2 CONTINUE

A statement that terminates execution of the block of statements in a **while** or **for** loop, and continues execution of the loop with the next iteration. In contrast to the **break** statement, **continue** does not terminate the execution of the loop entirely: instead,

- In a **while** loop, it jumps back to the *condition*.
- In a **for** loop, it jumps to the *update* expression.

Syntax

continue

Examples

The following example shows a **while** loop that has a **continue** statement that executes when the value of *i* is 3. Thus, *n* takes on the values 1, 3, 7, and 12.

i = 0 n = 0 while (i < 5) { i++ if (i == 3) continue n += i }

8.3 FOR

A statement that creates a loop that consists of three optional expressions, enclosed in parentheses and separated by semicolons, followed by a block of statements executed in the loop. The parts of the **for** statement are:

- The *initial expression*, generally used to initialize a counter variable. This statement may optionally declare new variables with the **var** keyword. This expression is optional.
- The *condition* that is evaluated on each pass through the loop. If this condition is true, the statements in the succeeding block are performed. This conditional test is optional. If omitted, then the condition always evaluates to true.
- An *update expression* generally used to update or increment the counter variable. This expression is optional.
- A block of statements that are executed as long as the *condition* is true. This can be a single statement or multiple statements. Although not required, it is good practice to indent these statements from the beginning of the **for** statement.

Syntax

```
for ([initial expression]; [condition]; [update expression]) {
        statements
}
initial expression = statement | variable declaration
```

Examples

This **for** statement starts by declaring the variable *i* and initializing it to zero. It checks that *i* is less than nine, and performs the two succeeding statements, and increments *i* by one after each pass through the loop. for (var i = 0; i < 9; i++) { n += i myfunc(n) }

The following is an for loop example.

```
<HTML>
<HEAD>
<TITLE>for Loop Example</TITLE>
</HEAD>
<BODY>
<SCRIPT LANGUAGE="JavaScript">
<!-- HIDE FROM OTHER BROWSERS
```

```
var name = prompt("What is your name?","name");
var query = "";
document.write("<H1>" + name + "'s 10 favorite foods</H1>");
for (var i=1; i<=10; i++) {
        document.write(i + ". " + prompt('Enter food number ' +
          i,'food') + '<BR>');
}
// STOP HIDING FROM OTHER BROWSERS -->
</SCRIPT>
</BODY>
</HTML>
```

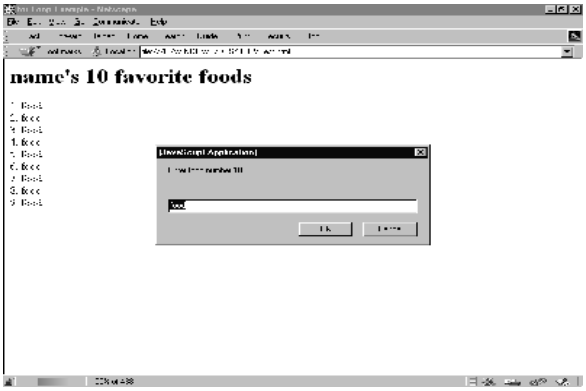


Fig 8-1: Result of the above script

8.4 FOR…..IN

A statement that iterates a variable *var* over all the properties of object *obj*. For each distinct property, it executes the statements in *statements*.

Syntax

for (var **in** obj) {
    statements }

Examples

The following function takes as its argument an object and the object's name. It then iterates over all the object's properties and returns a string that lists the property names and their values. function dump_props(obj, obj_name) { var result = "" for (var i in obj) { result += obj_name + "." + i + " = " + obj[i] + " " } result += " " return result }

8.5 IF…..ELSE

A conditional statement that executes the statements in *statements* if *condition* is true. In the optional **else** clause, it executes the statements in *else statements* if *condition* is false. These may be any JavaScript statements, including further nested **if** statements.

Syntax

if (condition) {
    statements
} [**else** {
    else statements
}]

Examples

if (cipher_char == from_char) { result = result + to_char x++ } else result = result + clear_char

8.6 WHILE LOOP

A statement that creates a loop that evaluates the expression *condition*, and if it is true, executes *statements*. It then repeats this process, as long as *condition* is true. When *condition* evaluates to false, execution continues with the statement following *statements*.

Although not required, it is good practice to indent the statements a **while** loop from the beginning of a **for** statement.

Syntax

while (condition) {
    statements
}

Examples

The following **while** loop iterates as long as *n* is less than three. Each iteration, it increments *n* and adds it to *x*. Therefore, *x* and *n* take on the following values:

- After the first pass: x = 1 and n = 1
- After the second pass: x = 2 and n = 3
- After the third pass: x = 3 and n = 6

After completing the third pass, the condition n < 3 is no longer true, so the loop terminates. n = 0 x = 0 while(n < 3) { n ++; x += n }

Example of statement use in JavaScript

<HTML>
<HEAD>
<SCRIPT LANGUAGE="JavaScript">

function wkdy (day)

```
{
        if (day =0)
                day="sunday";
        else if (day ==1)
                day="Monday";
        else if (day ==2)
                day="Tuesday";
        else if (day ==3)
                day="Wednesday";
        else if (day ==4)
                day="Thursday";
        else if (day ==5)
                day="Friday";
        else day="Saturday";
return day;
}
function month (mth)
{
        if (mth==0)
                mth="January";
        else if (mth==1)
                mth="February";
        else if (mth==2)
                mth="March";
        else if (mth==3)
                mth="April";
        else if (mth==4)
                mth="May";
        else if (mth==5)
                mth="June";
        else if (mth==6)
                mth="July";
        else if (mth==7)
                mth="August";
        else if (mth==8)
                mth="September";
        else if (mth==9)
                mth="October";
        else if (mth==10)
                mth="November";
        else if (mth==11)
                mth="December";
return mth;
}

</SCRIPT>
</HEAD>
```

```
<body>
<script language="JavaScript">

var dd, mth, yy, day, hh, ctime;;

today = new Date();
dd=today.getDate();
mth=today.getMonth()
yy=today.getYear();
day=today.getDay();
hh=today.getHours();
min=today.getMinutes();


clock= wkdy(day) + ", " + dd + " " +month(mth) +" " +yy + ", ";
clock += " " +hh + ":" + min;
document.write(clock, "<br>");

</SCRIPT>
</body>
<HTML>
```

Fig 8-2: Result of the above script

Tic-tac-toe with Loop

```
<HTML>

<HEAD>
<TITLE>Listing 7.4</TITLE>

<SCRIPT LANGUAGE="JavaScript">
<!-- HIDE FROM OTHER BROWSERS

var row = 0;
var col = 0;
```

```
var playerSymbol = "X";
var computerSymbol = "O";
board = new createArray(3,3);

function createArray(row,col) {

	var index = 0;
	this.length = (row * 10) + col;
	for (var x = 1; x <= row; x ++) {
		for (var y = 1; y <= col; y++) {
			index = (x*10) + y;
			this[index] = "";
		}
	}

}

function buildBoard(form) {

	var index = 0;
	for (var field = 0; field <= 8; field ++) {
		index = eval(form.elements[field].name);
		form.elements[field].value = board[index];
	}

}

function clearBoard(form) {
var index = 0;
	for (var field = 0; field <= 8; field ++) {
		form.elements[field].value = "";
		index = eval(form.elements[field].name);
		board[index] = "";
	}

}

function win(index) {

	var win = false;

	// CHECK ROWS
	if ((board[index] == board[(index < 30) ? index + 10 : index
	   - 20]) &&
		(board[index] == board[(index >  11) ? index - 10 :
		   index + 20])) {
	win = true;
	}
```

```
        // CHECK COLUMNS
        if ((board[index] == board[(index%10 < 3) ? index + 1 :
           index - 2]) &&
              (board[index] == board[(index%10 > 1) ? index - 1 :
                 index + 2])) {
        win = true;
        }

        // CHECK DIAGONALS
        if (Math.round(index/10) == index%10) {
              if ((board[index] == board[(index < 30) ? index + 11 :
                 index - 22]) &&
                    (board[index] == board[(index >  11) ? index - 11 :
                       index + 22])) {
              win = true;
              }
              if (index == 22) {
                    if ((board[index] == board[13]) && (board[index]
                    == board[31])) {
                          win = true;
                    }
              }
        }
if ((index == 31) || (index == 13)) {
        if ((board[index] == board[(index < 30) ? index + 9 : index -
           18]) &&
              (board[index] == board[(index >  11) ? index - 9 : index
                 + 18])) {
                    win = true;
              }
        }

        // RETURN THE RESULTS
        return win;

}

function play(form,field) {

        var index = eval(field.name);
        var playIndex = 0;
        var winIndex = 0;
        var done = false;
        field.value = playerSymbol;
        board[index] = playerSymbol;

        //CHECK FOR PLAYER WIN
        if (win(index)) {
```

```
        // PLAYER WON
        alert("Good Play! You Win!");
        clearBoard(form);
} else {
// PLAYER LOST, CHECK FOR WINNING POSITION
for (row = 1; row <= 3; row++) {
        for (col = 1; col <= 3; col++) {
                index = (row*10) + col;
                if (board[index] == "") {
                        board[index] = computerSymbol;
                if(win(index)) {
                        playIndex = index;
                        done = true;
                        board[index] = "";
                        break;
                }
        board[index] = "";
}
}
        if (done)
                break;
}
// CHECK IF COMPUTER CAN WIN
if (done) {
        board[playIndex] = computerSymbol;
        buildBoard(form);
        alert("Computer Just Won!");
        clearBoard(form);
} else {
// CAN'T WIN, CHECK IF NEED TO STOP A WIN
for (row = 1; row <=3; row++) {
        for (col = 1; col <= 3; col++) {
                index = (row*10) + col;
                if (board[index] == "") {
                        board[index] = playerSymbol;
                        if (win(index)) {
                                playIndex = index;
                                        done = true;
                                board[index] = "";
                                break;
                        }
                        board[index] = "";
                }
        }
        if (done)
        break;
}
// CHECK IF DONE
```

```
            if (done) {
                board[playIndex] = computerSymbol;
                buildBoard(form);
            } else {
        // NOT DONE, CHECK FOR FIRST EMPTY SPACE
        for (row = 1; row <= 3; row ++) {
            for (col = 1; col <= 3; col ++) {
            index = (row*10) + col;
            if (board[index] == "") {
            playIndex = index;
            done = true;
            break;
          }
         }
         if (done)
           break;
       }
       board[playIndex] = computerSymbol;
       buildBoard(form);
     }
   }
  }

}

// STOP HIDING HERE -->
</SCRIPT>

</HEAD>

<BODY>

<FORM METHOD = POST>

<TABLE>

<TR>
<TD>
<INPUT TYPE=text SIZE=3 NAME="11"
      onFocus="if (this.value != '') {blur();}"
      onChange="play(this.form,this);">
</TD>
<TD>
<INPUT TYPE=text SIZE=3 NAME="12"
      onFocus="if (this.value != '') {blur();}"
      onChange="play(this.form,this);">
</TD>
<TD>
```

```
<INPUT TYPE=text SIZE=3 NAME="13"
      onFocus="if (this.value != '') {blur();}"
      onChange="play(this.form,this);">
</TD>
</TR>

<TR>
<TD>
<INPUT TYPE=text SIZE=3 NAME="21"
      onFocus="if (this.value != '') {blur();}"
      onChange="play(this.form,this);">
</TD>
<TD>
<INPUT TYPE=text SIZE=3 NAME="22"
      onFocus="if (this.value != '') {blur();}"
      onChange="play(this.form,this);">
</TD>
<TD>
<INPUT TYPE=text SIZE=3 NAME="23"
      onFocus="if (this.value != '') {blur();}"
      onChange="play(this.form,this);">
</TD>
</TR>

<TR>
<TD>
<INPUT TYPE=text SIZE=3 NAME="31"
      onFocus="if (this.value != '') {blur();}"
      onChange="play(this.form,this);">
</TD>
<TD>
<INPUT TYPE=text SIZE=3 NAME="32"
      onFocus="if (this.value != '') {blur();}"
      onChange="play(this.form,this);">
</TD>
<TD>
<INPUT TYPE=text SIZE=3 NAME="33"
      onFocus="if (this.value != '') {blur();}"
      onChange="play(this.form,this);">
</TD>
</TR>

</TABLE>


<INPUT TYPE=button VALUE="I'm Done-Your Go">
<INPUT TYPE=button VALUE="Start Over" onClick="clearBoard(this.form);">
</FORM>
```

```
</BODY>

</HTML>
```

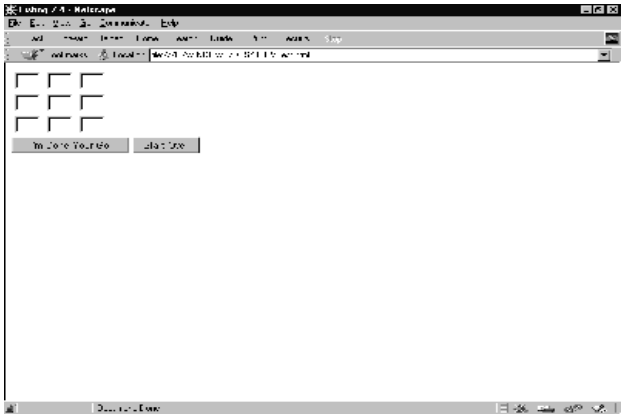


Fig 8-3: Result of the above script

Question that Probably Ponder in your Mind

18. Write while loops to emulate each of these for loop:

    g.
    ```
    for ( j = 4 ; j > 0; j -- ){
        document.writeln( j + "<BR>" );
    }
    ```

    h.
    ```
    for ( k = 1; k < = 99; k = k*2 ){
        k = k/1.5;
    }
    ```

    i.
    ```
    for ( num = 0; num < = 10; num ++ ){
        if ( num == 8 )
        break;
    }
    ```

FRAMES DOCUMENT AND WINDOWS

9.1 FRAME OBJECT(FRAMES ARRAY)

A window that can display multiple, independently scrollable frames on a single screen, each with its own distinct URL. Frames can point to different URLs and be targeted by other URLs, all within the same screen. A series of frames makes up a page.

Syntax

To define a frame object, use standard HTML syntax. The onLoad and onUnload event handlers are specified in the <FRAMESET> tag but are actually event handlers for the window object:

```
<FRAMESET
    ROWS="rowHeightList"
    COLS="columnWidthList"
    [onLoad="handlerText"]
    [onUnload="handlerText"]>
    [<FRAME SRC="locationOrURL" NAME="frameName">]
</FRAMESET>
```

ROWS="rowHeightList" is a comma-separated list of values specifying the row-height of the frame.

An optional suffix defines the units. Default units are pixels.

COLS="columnWidthList" is a comma-separated list of values specifying the column-width of the frame. An optional suffix defines the units. Default units are pixels. <FRAME> defines a frame.

SRC="locationOrURL" specifies the URL of the document to be displayed in the frame.

The URL cannot include an anchor name; for example <FRAME SRC="doc2.html#colors" NAME="frame2"> is invalid. See the location object for a description of the URL components.

NAME="frameName" specifies a name to be used as a target of hyperlink jumps.

To use a frame object's properties:

- [windowReference.]frameName.propertyName
- [windowReference.]frames[index].propertyName
- window.propertyName
- self.propertyName
- parent.propertyName

windowReference is a variable windowVar from a window definition (see window object), or one of the synonyms top or parent.

frameName is the value of the NAME attribute in the <FRAME> tag of a frame object.

index is an integer representing a frame object.

propertyName is one of the properties listed below.

Description

The <FRAMESET> tag is used in an HTML document whose sole purpose is to define the layout of frames that make up a page. Each frame is a window object.

If a <FRAME> tag contains SRC and NAME attributes, you can refer to that frame from a sibling frame by using parent.frameName or parent.frames[index]. For example, if the fourth frame in a set has NAME="homeFrame", sibling frames can refer to that frame using parent.homeFrame or parent.frames[3].

The self and window properties are synonyms for the current frame, and you can optionally use them to refer to the current frame. You can use these properties to make your code more readable. See the properties listed below for examples.

The top and parent properties are also synonyms that can be used in place of the frame name. Top refers to the top-most window that contains frames or nested framesets, and parent refers to the window containing the current frameset. See the top and parent properties.

9.1.1 The Frames Array

You can reference the frame objects in your code by using the frames array. This array contains an entry for each child frame (<FRAME> tag) in a window containing a <FRAMESET> tag in source order. For example, if a window contains three child frames, these frames are reflected as parent.frames[0], parent.frames[1], and parent.frames[2].

To use the frames array:

- [frameReference.]frames[index]
- [frameReference.]frames.length
- [windowReference.]frames[index]
- [windowReference.]frames.length

frameReference is a valid way of referring to a frame, as described in the frame object.

windowReference is a variable windowVar from a window definition (see window object), or one of the synonyms top or parent.

index is an integer representing a frame in a parent window.

To obtain the number of child frames in a window or frame, use the length property:

[windowReference.].frames.length

[frameReference.].frames.length

Elements in the frames array are read-only. For example, the statement

windowReference.frames[0]="frame1" has no effect.

The value of each element in the frames array is <object nameAttribute>, where

nameAttribute is the NAME attribute of the frame.

Properties

The frame object has the following properties:

- frames is an array reflecting all the frames in a window
- name reflects the NAME attribute of the <FRAME> tag
- length reflects the number of child frames within a frame
- parent is a synonym for the window or frame containing the current frameset
- self is a synonym for the current frame
- window is a synonym for the current frame

The frames array has the following properties:

- length reflects the number of child frames within a frame

Methods

- clearTimeout
- setTimeout

Event handlers

- None. The onLoad and onUnload event handlers are specified in the <FRAMESET> tag but are actually event handlers for the window object.

Property of

- The frame object is a property of window

- The frames array is a property of both frame and window

Examples

The following example creates two windows, each with four frames. In the first window, the first frame contains pushbuttons that change the background colors of the frames in both windows.

FRAMSET1.HTML, which defines the frames for the first window, contains the following code:

```
<HTML>
<HEAD>
<TITLE>Frames and Framesets: Window 1</TITLE>
</HEAD>
<FRAMESET ROWS="50%,50%" COLS="40%,60%" onLoad="alert('Hello, World.')">
<FRAME SRC=framcon1.html NAME="frame1">
<FRAME SRC=framcon2.html NAME="frame2">
<FRAME SRC=framcon2.html NAME="frame3">
<FRAME SRC=framcon2.html NAME="frame4">
</FRAMESET>
</HTML>
```

FRAMSET2.HTML, which defines the frames for the second window, contains the following code:

```
<HTML>
<HEAD>
<TITLE>Frames and Framesets: Window 2</TITLE>
</HEAD>
<FRAMESET ROWS="50%,50%" COLS="40%,60%">
<FRAME SRC=framcon2.html NAME="frame1">
<FRAME SRC=framcon2.html NAME="frame2">
<FRAME SRC=framcon2.html NAME="frame3">
<FRAME SRC=framcon2.html NAME="frame4">
</FRAMESET>
</HTML>
```

FRAMCON1.HTML, which defines the content for the first frame in the first window, contains the following code:

```
<HTML>
<BODY>
<A NAME="frame1"><H1>Frame1</H1></A>
<P><A HREF="framcon3.html" target=frame2>Click here</A> to load a different file into frame 2.
<SCRIPT>
window2=open("framset2.html","secondFrameset")
</SCRIPT>
<FORM>
<P><INPUT TYPE="button" VALUE="Change frame2 to teal"
      onClick="parent.frame2.document.bgColor='teal'">
```

```
<P><INPUT TYPE="button" VALUE="Change frame3 to slateblue"
      onClick="parent.frames[2].document.bgColor='slateblue'">
<P><INPUT TYPE="button" VALUE="Change frame4 to darkturquoise"
      onClick="top.frames[3].document.bgColor='darkturquoise'"
        >

<P><INPUT TYPE="button" VALUE="window2.frame2 to violet"
      onClick="window2.frame2.document.bgColor='violet'">
<P><INPUT TYPE="button" VALUE="window2.frame3 to fuchsia"
      onClick="window2.frames[2].document.bgColor='fuchsia'">
<P><INPUT TYPE="button" VALUE="window2.frame4 to deeppink"
      onClick="window2.frames[3].document.bgColor='deeppink'"
        >
</FORM>
</BODY>
</HTML>
```

FRAMCON2.HTML, which defines the content for the remaining frames, contains the following code:

```
<HTML>
<BODY>
<P>This is a frame.
</BODY>
</HTML>
```

FRAMCON3.HTML, which is referenced in a link object in FRAMCON1.HTML, contains the following code:

```
<HTML>
<BODY>
<P>This is a frame. What do you think?
</BODY>
</HTML>
```

9.2 Document Object

Contains information on the current document, and provides methods for displaying HTML output to the user.

Syntax

To define a document object, use standard HTML syntax:

```
<BODY
      BACKGROUND="backgroundImage"
      BGCOLOR="backgroundColor"
      TEXT="foregroundColor"
      LINK="unfollowedLinkColor"
```

```
        ALINK="activatedLinkColor"
        VLINK="followedLinkColor"
        [onLoad="handlerText"]
        [onUnload="handlerText"]>
</BODY>
```

BACKGROUND specifies an image that fills the background of the document.

BGCOLOR, TEXT, LINK, ALINK, and VLINK are color specifications expressed as a hexadecimal RGB triplet (in the format "rrggbb" or "#rrggbb") or as one of the string literals listed in Color Values.

To use a document object's properties and methods:

- document.propertyName
- document.methodName(parameters)

propertyName is one of the properties listed below.

methodName is one of the methods listed below.

Description

An HTML document consists of a <HEAD> and <BODY> tag. The <HEAD> includes information on the document's title and base (the absolute URL base to be used for relative URL links in the document). The <BODY> tag encloses the body of a document, which is defined by the current URL. The entire body of the document (all other HTML elements for the document) goes within the <BODY> tag.

You can reference the anchors, forms, and links of a document by using the anchors, forms, and links arrays. These arrays contain an entry for each anchor, form, or link in a document.

Properties

- alinkColor reflects the ALINK attribute
- anchors is an array reflecting all the anchors in a document
- bgColor reflects the BGCOLOR attribute
- cookie specifies a cookie
- fgColor reflects the TEXT attribute
- forms is an array reflecting all the forms in a document
- lastModified reflects the date a document was last modified
- linkColor reflects the LINK attribute

- links is an array reflecting all the links in a document
- location reflects the complete URL of a document
- referrer reflects the URL of the calling document
- title reflects the contents of the <TITLE> tag
- vlinkColor reflects the VLINK attribute

Methods

- clear
- close
- open
- write
- writeln

Event handlers

- None. The onLoad and onUnload event handlers are specified in the <BODY> tag but are actually event handlers for the window object.

Property of

- window

Examples

The following example creates two frames, each with one document. The document in the first frame contains links to anchors in the document of the second frame. Each document defines its colors.

DOC0.HTML, which defines the frames, contains the following code:

```
<HTML>
<HEAD>
<TITLE>Document object example</TITLE>
</HEAD>
<FRAMESET COLS="30%,70%">
<FRAME SRC="doc1.html" NAME="frame1">
```

```
<FRAME SRC="doc2.html" NAME="frame2">
</FRAMESET>
</HTML>
```

DOC1.HTML, which defines the content for the first frame, contains the following code:

```
<HTML>
<SCRIPT>
</SCRIPT>
<BODY
      BGCOLOR="antiquewhite"
      TEXT="darkviolet"
      LINK="fuchsia"
      ALINK="forestgreen"
      VLINK="navy">
<P><B>Some links</B>
<LI><A HREF="doc2.html#numbers" TARGET="frame2">Numbers</A>
<LI><A HREF="doc2.html#colors" TARGET="frame2">Colors</A>
<LI><A HREF="doc2.html#musicTypes" TARGET="frame2">Music types</A>
<LI><A HREF="doc2.html#countries" TARGET="frame2">Countries</A>
</BODY>
</HTML>
```

DOC2.HTML, which defines the content for the second frame, contains the following code:

```
<HTML>
<SCRIPT>
</SCRIPT>
<BODY
      BGCOLOR="oldlace" onLoad="alert('Hello, World.')"
      TEXT="navy">
<P><A NAME="numbers"><B>Some numbers</B></A>
<LI>one
<LI>two
<LI>three
<LI>four
<LI>five
<LI>six
<LI>seven
<LI>eight
<LI>nine
<P><A NAME="colors"><B>Some colors</B></A>
<LI>red
<LI>orange
<LI>yellow
<LI>green
<LI>blue
```

```
<LI>purple
<LI>brown
<LI>black
<P><A NAME="musicTypes"><B>Some music types</B></A>
<LI>R&B
<LI>Jazz
<LI>Soul
<LI>Reggae
<LI>Rock
<LI>Country
<LI>Classical
<LI>Opera
<P><A NAME="countries"><B>Some countries</B></A>
<LI>Afghanistan
<LI>Brazil
<LI>Canada
<LI>Finland
<LI>India
<LI>Italy
<LI>Japan
<LI>Kenya
<LI>Mexico
<LI>Nigeria
</BODY>
</HTML>
```

9.3 working with the status bar

To have a scrolling message at the status bar. Insert this code into your head (<head>Insert in between here</head>)

```
<script language="JavaScript">
<!-- Hide the script from old browsers --
function scrollit(seed)
{
      var m1  =  " THIS IS WHERE YOUR MESSAGE GOES  ";
      var msg=m1;   var out = " ";
      var c = 0;

      if (seed > 100)
            {
            seed--;
            var cmd="scrollit(" + seed + ")";
            timerTwo=window.setTimeout(cmd,7);
            }
      else
            if (seed <= 100 && seed > 0)
            {
                  for (c=0 ; c < seed ; c++)
```

```
                {
                    out+=" ";
                    }
                    out+=msg;
                    seed--;
                    var cmd="scrollit(" + seed + ")";
                    window.status=out;
                    timerTwo=window.setTimeout(cmd,7);
                    }
                else if (seed <= 0)
                {
                    if (-seed < msg.length)
{
                    out+=msg.substring(-seed,msg.length);
                    seed--;
                    var cmd="scrollit(" + seed + ")";
                    window.status=out;
                    timerTwo=window.setTimeout(cmd,7);
                    }
                else
                    {
                    window.status=" ";
                    timerTwo=window.setTimeout("scrollit(100)",
                       75);
                    }
                }
}
// --End Hiding Here -->
</script>
```

Now put this inside your body code.

```
<body onLoad="timerONE=window.setTimeout('scrollit(100)',500)">
```

Or

```
<BODY BGCOLOR="#FFFFFF" TEXT="#000000" LINK="#FF0000" VLINK="#000080"
ALINK="#000080" onLoad="timerONE=window.setTimeout('scrollit(100)',500)">
```

Fig 9-1: Working with the Status bar

9.3.1 To have a Ticker Tape

Insert this code **Insert in between** <head> and </head>

```
<script language="JavaScript">
<!-- Hide the script from old browsers --
var timerID = null;
var timerRunning = false;
var id,pause=0,position=0;

function ticker() {
                var i,k,msg="YOUR MESSAGE GOES HERE";
                k=(75/msg.length)+1;
                for(i=0;i<=k;i++) msg+=" "+msg;
                document.form2.ticker.value=msg.substring
                   (position,position+75);
                if(position++==38) position=0;
                id=setTimeout("ticker()",1000/10); }

function action() {
                if(!pause) {
                     clearTimeout(id);
                     pause=1; }

                else {
                     ticker();
                     pause=0; } }
// --End Hiding Here -->
</script>
```

Now put this inside your body code.

```
<body onLoad="ticker()">
```

OR

```
<BODY BGCOLOR="#FFFFFF" TEXT="#000000" LINK="#FF0000" VLINK="#000080"
ALINK="#000080" onLoad="ticker()">
```

Now put this where ever you want in your HTML document

```
<form name="form2">
<input type="text" name="ticker" size="75">
```

To change the size of the ticker tape, change the number "75" to what ever size you want in the line... <input type="text" name="ticker" size="75">

Fig 9-2: Ticker Tape

DYNAMIC HTML[DHTML] AND JAVASCRIPT

What is DHTML?

DHTML stands for Dynamic HTML. When implemented in full it allows complete control of all the structures and contents of a Web page by exposing every part of the Document Object Model (DOM) to control by JavaScript. Thus the position, visibility, color, size, content, etc of every aspect of a web page can be dynamically controlled.

List the three main components of Dynamic HTML.

1. **Positioning:** precisely placing blocks of content on the page and, if desired, moving these blocks around (strictly speaking, a subset of style modifications).
2. **Style modifications:** on-the-fly altering the aesthetics of content on the page.
3. **Event handling:** how to relate user events to changes in positioning or other style modifications.

Write short notes on layers.

In general, "layer" refers to elements that can be positioned at exact coordinates on the page. These elements can be defined with the DIV, SPAN, LAYER, or ILAYER tags. Layers created with DIV and SPAN are referred to as CSS layers because their properties are defined by

the Cascading Style Sheets specification by the World Wide web Consortium. This specification defines style properties (e.g. font, color, padding, margin, word-spacing) in addition to the positioning properties associated with layers (top, left, z-index, visibility).

How DHTML Does it Work?

Various elements of a web page such as images and blocks of text are organized into groups with the `<div>` or `<layer>` tags. The groups are then given a list of properties using the cascading style sheet specification and a name to distinguish them from each other. Then through the use of a scripting language, you can dynamically change the CSS attribute of each group. It's really quite neat how the three elements of web design can come together in a dynamic union.

What DHTML Can Do ?

DHTML gives you ability to position elements of a web page to precise (x,y,z) coordinates and dynamically change the position with script. Every property of a web page element can be altered with the use of a script language. Some properties include color, size, visibility, alignment, etc.. You can achieve some awesome effects with DHTML.

Write About The Technology Components Of DHTML.

The major components of Dynamic HTML technology are:-

- **Style Sheets** let you specify the stylistic attributes of the typographic elements of your web page. They let you change the color, size, or style of the text on a page *without* waiting for the screen to refresh.
- **Content Positioning** lets a web developer animate any element on a web page, moving pictures, text, and objects at will. It lets you ensure that pieces of content are displayed on the page exactly where you want them to appear, and you can modify their appearance and location after the page has been displayed.
- **Dynamic Content** actually changes the words, pictures, or multimedia on a page *without* another trip to the web Server.
- **Data Binding** lets you get all the information you need to ask questions, change elements, and get results *without* going back to the web server.

- **Downloadable Fonts** let you use the fonts of your choice to enhance the appearance of your text. Then you can package the fonts with the page so that the text is always displayed with your chosen fonts.

Introduction to JavaScript

JavaScript is used in millions of Web pages to improve the design, validate forms, detect browsers, create cookies, and much more.

JavaScript is the most popular scripting language on the internet, and works in all major browsers, such as Internet Explorer, Mozilla, Firefox, Netscape, Opera.

What is JavaScript?

- JavaScript was designed to add interactivity to HTML pages
- JavaScript is a scripting language (a scripting language is a lightweight programming language)
- A JavaScript consists of lines of executable computer code
- A JavaScript is usually embedded directly into HTML pages
- JavaScript is an interpreted language (means that scripts execute without preliminary compilation)
- Everyone can use JavaScript without purchasing a license

Are Java and JavaScript the Same?

NO!

Java and JavaScript are two completely different languages in both concept and design!

Java (developed by Sun Microsystems) is a powerful and much more complex programming language - in the same category as C and C++.

What can a JavaScript Do?

- **JavaScript gives HTML designers a programming tool -** HTML authors are normally not programmers, but JavaScript is a scripting language with a very simple syntax! Almost anyone can put small "snippets" of code into their HTML pages
- **JavaScript can put dynamic text into an HTML page -** A JavaScript statement like this: document.write("<h1>" + name + "</h1>") can write a variable text into an HTML page

- **JavaScript can react to events -** A JavaScript can be set to execute when something happens, like when a page has finished loading or when a user clicks on an HTML element
- **JavaScript can read and write HTML elements -** A JavaScript can read and change the content of an HTML element
- **JavaScript can be used to validate data -** A JavaScript can be used to validate form data before it is submitted to a server, this will save the server from extra processing
- **JavaScript can be used to detect the visitor's browser** - A JavaScript can be used to detect the visitor's browser, and - depending on the browser - load another page specifically designed for that browser
- **JavaScript can be used to create cookies** - A JavaScript can be used to store and retrieve information on the visitor's computer

How to Put a JavaScript Into an HTML Page

```
<html>
<body>
<script type="text/javascript">
document.write("Hello World!")
</script>
</body>
</html>
```

The code above will produce this output on an HTML page:

```
Hello World!
```

Example Explained

To insert a JavaScript into an HTML page, we use the <script> tag (also use the type attribute to define the scripting language).

So, the <script type="text/javascript"> and </script> tells where the JavaScript starts and ends:

```
<html>
<body>
<script type="text/javascript">
...
</script>
</body>
</html>
```

The word **document.write** is a standard JavaScript command for writing output to a page.

By entering the document.write command between the <script type="text/javascript"> and </script> tags, the browser will recognize it as a JavaScript command and execute the code line. In this case the browser will write Hello World! to the page:

```
<html>
<body>
<script type="text/javascript">
document.write("Hello World!")
</script>
</body>
</html>
```

Note: If we had not entered the <script> tag, the browser would have treated the document.write("Hello World!") command as pure text, and just write the entire line on the page.

Ending Statements With a Semicolon?

With traditional programming languages, like C++ and Java, each code statement has to end with a semicolon.

Many programmers continue this habit when writing JavaScript, but in general, semicolons are **optional**! However, semicolons are required if you want to put more than one statement on a single line.

How to Handle Older Browsers

Browsers that do not support JavaScript will display the script as page content. To prevent them from doing this, we may use the HTML comment tag:

```
<script type="text/javascript">
<!--
document.write("Hello World!")
//-->
</script>
```

The two forward slashes at the end of comment line (//) are a JavaScript comment symbol. This prevents the JavaScript compiler from compiling the line.

Where to Put the JavaScript

JavaScripts in the body section will be executed WHILE the page loads.

JavaScripts in the head section will be executed when CALLED.

JavaScripts in a page will be executed immediately while the page loads into the browser. This is not always what we want. Sometimes we want to execute a script when a page loads, other times when a user triggers an event.

Scripts in the head section: Scripts to be executed when they are called, or when an event is triggered, go in the head section. When you place a script in the head section, you will ensure that the script is loaded before anyone uses it.

```
<html>
<head>
```

```
<script type="text/javascript">
....
</script>
</head>
```

Scripts in the body section: Scripts to be executed when the page loads go in the body section. When you place a script in the body section it generates the content of the page.

```
<html>
<head>
</head>
<body>
<script type="text/javascript">
....
</script>
</body>
```

Scripts in both the body and the head section: You can place an unlimited number of scripts in your document, so you can have scripts in both the body and the head section.

```
<html>
<head>
<script type="text/javascript">
....
</script>
</head>
<body>
<script type="text/javascript">
....
</script>
</body>
```

Using an External JavaScript

Sometimes you might want to run the same JavaScript on several pages, without having to write the same script on every page.

To simplify this, you can write a JavaScript in an external file. Save the external JavaScript file with a .js file extension.

Note: The external script cannot contain the <script> tag!

To use the external script, point to the .js file in the "src" attribute of the <script> tag:

```
<html>
<head>
<script src="xxx.js"></script>
```

```
</head>
<body>
</body>
</html>
```

Note: Remember to place the script exactly where you normally would write the script!

Variables

A variable is a "container" for information you want to store. A variable's value can change during the script. You can refer to a variable by name to see its value or to change its value.

Rules for variable names:

- Variable names are case sensitive
- They must begin with a letter or the underscore character

IMPORTANT! JavaScript is case-sensitive! A variable named strname is not the same as a variable named STRNAME!

Declare a Variable

You can create a variable with the var statement:

```
var strname = some value
```

You can also create a variable without the var statement:

```
strname = some value
```

Assign a Value to a Variable

You can assign a value to a variable like this:

```
var strname = "Hege"
```

Or like this:

```
strname = "Hege"
```

The variable name is on the left side of the expression and the value you want to assign to the variable is on the right. Now the variable "strname" has the value "Hege".

Lifetime of Variables

When you declare a variable within a function, the variable can only be accessed within that function. When you exit the function, the variable is destroyed. These variables are called local variables. You can have local variables with the same name in different functions, because each is recognized only by the function in which it is declared.

If you declare a variable outside a function, all the functions on your page can access it. The lifetime of these variables starts when they are declared, and ends when the page is closed.

JavaScript If...Else Statements

Conditional statements in JavaScript are used to perform different actions based on different conditions.

Conditional Statements

Very often when you write code, you want to perform different actions for different decisions. You can use conditional statements in your code to do this.

In JavaScript we have the following conditional statements:

- **if statement** - use this statement if you want to execute some code only if a specified condition is true
- **if...else statement** - use this statement if you want to execute some code if the condition is true and another code if the condition is false
- **if...else if....else statement** - use this statement if you want to select one of many blocks of code to be executed
- **switch statement** - use this statement if you want to select one of many blocks of code to be executed

---

If Statement

You should use the if statement if you want to execute some code only if a specified condition is true.

Syntax

```
if (condition)
{
code to be executed if condition is true
}
```

Note that if is written in lowercase letters. Using uppercase letters (IF) will generate a JavaScript error!

Example 1

```
<script type="text/javascript">
//Write a "Good morning" greeting if
//the time is less than 10
var d=new Date()
var time=d.getHours()

if (time<10)
{
document.write("<b>Good morning</b>")
```

```
}
</script>
```

Example 2

```
<script type="text/javascript">
//Write "Lunch-time!" if the time is 11
var d=new Date()
var time=d.getHours()

if (time==11)
{
document.write("<b>Lunch-time!</b>")
}
</script>
```

Note: When **comparing** variables you must always use two equals signs next to each other (==)!

Notice that there is no ..else.. in this syntax. You just tell the code to execute some code **only if the specified condition is true**.

If...else Statement

If you want to execute some code if a condition is true and another code if the condition is not true, use the if....else statement.

Syntax

```
if (condition)
{
code to be executed if condition is true
}
else
{
code to be executed if condition is not true
}
```

Example

```
<script type="text/javascript">
//If the time is less than 10,
//you will get a "Good morning" greeting.
//Otherwise you will get a "Good day" greeting.
var d = new Date()
var time = d.getHours()

if (time < 10)
{
document.write("Good morning!")
}
```

```
else
{
document.write("Good day!")
}
</script>
```

If...else if...else Statement

You should use the if....else if...else statement if you want to select one of many sets of lines to execute.

Syntax

```
if (condition1)
{
code to be executed if condition1 is true
}
else if (condition2)
{
code to be executed if condition2 is true
}
else
{
code to be executed if condition1 and
condition2 are not true
}
```

Example

```
<script type="text/javascript">
var d = new Date()
var time = d.getHours()
if (time<10)
{
document.write("<b>Good morning</b>")
}
else if (time>10 && time<16)
{
document.write("<b>Good day</b>")
}
else
{
document.write("<b>Hello World!</b>")
}
</script>
```

Conditional statements in JavaScript are used to perform different actions based on different conditions.

The JavaScript Switch Statement

You should use the switch statement if you want to select one of many blocks of code to be executed.

Syntax

```
switch(n)
{
case 1:
  execute code block 1
  break
case 2:
  execute code block 2
  break
default:
  code to be executed if n is
  different from case 1 and 2
}
```

This is how it works: First we have a single expression *n* (most often a variable), that is evaluated once. The value of the expression is then compared with the values for each case in the structure. If there is a match, the block of code associated with that case is executed. Use **break** to prevent the code from running into the next case automatically.

Example

```
<script type="text/javascript">
//You will receive a different greeting based
//on what day it is. Note that Sunday=0,
//Monday=1, Tuesday=2, etc.
var d=new Date()
theDay=d.getDay()
switch (theDay)
{
case 5:
  document.write("Finally Friday")
  break
case 6:
  document.write("Super Saturday")
  break
case 0:
  document.write("Sleepy Sunday")
  break
default:
```

```
 document.write("I'm looking forward to this weekend!")
}
</script>
```

JavaScript Operators

Arithmetic Operators

Operator	Description	Example	Result
+	Addition	x=2 y=2 x+y	4
-	Subtraction	x=5 y=2 x-y	3
*	Multiplication	x=5 y=4 x*y	20
/	Division	15/5 5/2	3 2.5
%	Modulus (division remainder)	5%2 10%8 10%2	1 2 0
++	Increment	x=5 x++	x=6
--	Decrement	x=5 x--	x=4

Assignment Operators

Operator	Example	Is The Same As
=	x=y	x=y
+=	x+=y	x=x+y
-=	x-=y	x=x-y
*=	x*=y	x=x*y
/=	x/=y	x=x/y
%=	x%=y	x=x%y

Comparison Operators

Operator	Description	Example
==	is equal to	5==8 returns false
===	is equal to (checks for both value and type)	x=5 y="5" x==y returns true x===y returns false
!=	is not equal	5!=8 returns true

>	is greater than	5>8 returns false
<	is less than	5<8 returns true
>=	is greater than or equal to	5>=8 returns false
<=	is less than or equal to	5<=8 returns true

Logical Operators

Operator	Description	Example
&&	and	x=6 y=3 (x < 10 && y > 1) returns true
||	or	x=6 y=3 (x==5 || y==5) returns false
!	not	x=6 y=3 !(x==y) returns true

String Operator

A string is most often text, for example "Hello World!". To stick two or more string variables together, use the + operator.

```
txt1="What a very"
txt2="nice day!"
txt3=txt1+txt2
```

The variable txt3 now contains "What a verynice day!".

To add a space between two string variables, insert a space into the expression, OR in one of the strings.

```
txt1="What a very"
txt2="nice day!"
txt3=txt1+" "+txt2
or
txt1="What a very "
txt2="nice day!"
txt3=txt1+txt2
```

The variable txt3 now contains "What a very nice day!".

Conditional Operator

JavaScript also contains a conditional operator that assigns a value to a variable based on some condition.

Syntax

```
variablename=(condition)?value1:value2
```

Example

```
greeting=(visitor=="PRES")?"Dear President ":"Dear "
```

If the variable visitor is equal to PRES, then put the string "Dear President " in the variable named greeting. If the variable visitor is not equal to PRES, then put the string "Dear " into the variable named greeting.

JavaScript Popup Boxes

In JavaScript we can create three kind of popup boxes: Alert box, Confirm box, and Prompt box.

Alert Box

An alert box is often used if you want to make sure information comes through to the user.

When an alert box pops up, the user will have to click "OK" to proceed.

Syntax:

```
alert("sometext")
```

Confirm Box

A confirm box is often used if you want the user to verify or accept something.

When a confirm box pops up, the user will have to click either "OK" or "Cancel" to proceed.

If the user clicks "OK", the box returns true. If the user clicks "Cancel", the box returns false.

Syntax:

```
confirm("sometext")
```

Prompt Box

A prompt box is often used if you want the user to input a value before entering a page.

When a prompt box pops up, the user will have to click either "OK" or "Cancel" to proceed after entering an input value.

If the user clicks "OK" the box returns the input value. If the user clicks "Cancel" the box returns null.

Syntax:

```
prompt("sometext","defaultvalue")
```

JavaScript Functions

A function is a reusable code-block that will be executed by an event, or when the function is called.

JavaScript Functions

To keep the browser from executing a script as soon as the page is loaded, you can write your script as a function.

A function contains some code that will be executed only by an event or by a call to that function.

You may call a function from anywhere within the page (or even from other pages if the function is embedded in an external .js file).

Functions are defined at the beginning of a page, in the <head> section.

Example

```
<html>
<head>
<script type="text/javascript">
function displaymessage()
{
alert("Hello World!")
}
</script>
</head>
<body>
<form>
<input type="button" value="Click me!"
onclick="displaymessage()" >
</form>
</body>
</html>
```

If the line: alert("Hello world!!"), in the example above had not been written within a function, it would have been executed as soon as the line was loaded. Now, the script is not executed before the user hits the button. We have added an onClick event to the button that will execute the function displaymessage() when the button is clicked.

You will learn more about JavaScript events in the JS Events chapter.

How to Define a Function

The syntax for creating a function is:

```
function functionname(var1,var2,...,varX)
{
some code
}
```

var1, var2, etc are variables or values passed into the function. The { and the } defines the start and end of the function.

Note: A function with no parameters must include the parentheses () after the function name:

```
function functionname()
{
some code
}
```

Note: Do not forget about the importance of capitals in JavaScript! The word function must be written in lowercase letters, otherwise a JavaScript error occurs! Also note that you must call a function with the exact same capitals as in the function name.

The return Statement

The return statement is used to specify the value that is returned from the function.

So, functions that are going to return a value must use the return statement.

Example

The function below should return the product of two numbers (a and b):

```
function prod(a,b)
{
x=a*b
return x
}
```

When you call the function above, you must pass along two parameters:

```
product=prod(2,3)
```

The returned value from the prod() function is 6, and it will be stored in the variable called product.

JavaScript For Loop

Loops in JavaScript are used to execute the same block of code a specified number of times or while a specified condition is true.

JavaScript Loops

Very often when you write code, you want the same block of code to run over and over again in a row. Instead of adding several almost equal lines in a script we can use loops to perform a task like this.

In JavaScript there are two different kind of loops:

- **for** - loops through a block of code a specified number of times
- **while** - loops through a block of code while a specified condition is true

---

The for Loop

The for loop is used when you know in advance how many times the script should run.

Syntax

```
for (var=startvalue;var<=endvalue;var=var+increment)
{
   code to be executed
}
```

Example

Explanation: The example below defines a loop that starts with i=0. The loop will continue to run as long as **i** is less than, or equal to 10. **i** will increase by 1 each time the loop runs.

Note: The increment parameter could also be negative, and the <= could be any comparing statement.

```
<html>
<body>
<script type="text/javascript">
var i=0
for (i=0;i<=10;i++)
{
document.write("The number is " + i)
document.write("<br />")
}
</script>
</body>
</html>
```

Result

```
The number is 0
The number is 1
The number is 2
The number is 3
The number is 4
The number is 5
The number is 6
The number is 7
The number is 8
The number is 9
The number is 10
```

JavaScript While Loop

Loops in JavaScript are used to execute the same block of code a specified number of times or while a specified condition is true.

The while loop

The while loop is used when you want the loop to execute and continue executing while the specified condition is true.

```
while (var<=endvalue)
{
   code to be executed
}
```

Note: The <= could be any comparing statement.

Example

Explanation: The example below defines a loop that starts with i=0. The loop will continue to run as long as **i** is less than, or equal to 10. **i** will increase by 1 each time the loop runs.

```
<html>
<body>
<script type="text/javascript">
var i=0
while (i<=10)
{
document.write("The number is " + i)
document.write("<br />")
i=i+1
}
</script>
</body>
</html>
```

Result

```
The number is 0
The number is 1
The number is 2
The number is 3
The number is 4
The number is 5
The number is 6
The number is 7
The number is 8
The number is 9
The number is 10
```

The do...while Loop

The do...while loop is a variant of the while loop. This loop will always execute a block of code ONCE, and then it will repeat the loop as long as the specified condition is true. This loop will

always be executed once, even if the condition is false, because the code is executed before the condition is tested.

```
do
{
   code to be executed
}
while (var<=endvalue)
```

Example

```
<html>
<body>
<script type="text/javascript">
var i=0
do
{
document.write("The number is " + i)
document.write("<br />")
i=i+1
}
while (i<0)
</script>
</body>
</html>
```

Result

```
The number is 0
```

JavaScript Break and Continue

JavaScript break and continue Statements

There are two special statements that can be used inside loops: break and continue.

Break

The break command will break the loop and continue executing the code that follows after the loop (if any).

Example

```
<html>
<body>
<script type="text/javascript">
var i=0
for (i=0;i<=10;i++)
{
if (i==3){break}
document.write("The number is " + i)
```

```
document.write("<br />")
}
</script>
</body>
</html>
```

Result

```
The number is 0
The number is 1
The number is 2
```

Continue

The continue command will break the current loop and continue with the next value.

Example

```
<html>
<body>
<script type="text/javascript">
var i=0
for (i=0;i<=10;i++)
{
if (i==3){continue}
document.write("The number is " + i)
document.write("<br />")
}
</script>
</body>
</html>
```

Result

```
The number is 0
The number is 1
The number is 2
The number is 4
The number is 5
The number is 6
The number is 7
The number is 8
The number is 9
The number is 10
```

JavaScript For...In Statement

The for...in statement is used to loop (iterate) through the elements of an array or through the properties of an object.

JavaScript For...In Statement

The for...in statement is used to loop (iterate) through the elements of an array or through the properties of an object.

The code in the body of the for ... in loop is executed once for each element/property.

Syntax

```
for (variable in object)
{
   code to be executed
}
```

The variable argument can be a named variable, an array element, or a property of an object.

Example

Using for...in to loop through an array:

```
<html>
<body>
<script type="text/javascript">
var x
var mycars = new Array()
mycars[0] = "Saab"
mycars[1] = "Volvo"
mycars[2] = "BMW"

for (x in mycars)
{
document.write(mycars[x] + "<br />")
}
</script>
</body>
</html>
```

JavaScript Events

Events are actions that can be detected by JavaScript.

Events

By using JavaScript, we have the ability to create dynamic web pages. Events are actions that can be detected by JavaScript.

Every element on a web page has certain events which can trigger JavaScript functions. For example, we can use the onClick event of a button element to indicate that a function will run when a user clicks on the button. We define the events in the HTML tags.

Examples of events:

- A mouse click
- A web page or an image loading
- Mousing over a hot spot on the web page
- Selecting an input box in an HTML form
- Submitting an HTML form
- A keystroke

The following table lists the events recognized by JavaScript:

Note: Events are normally used in combination with functions, and the function will not be executed before the event occurs!

onload and onUnload

The onload and onUnload events are triggered when the user enters or leaves the page.

The onload event is often used to check the visitor's browser type and browser version, and load the proper version of the web page based on the information.

Both the onload and onUnload events are also often used to deal with cookies that should be set when a user enters or leaves a page. For example, you could have a popup asking for the user's name upon his first arrival to your page. The name is then stored in a cookie. Next time the visitor arrives at your page, you could have another popup saying something like: "Welcome John Doe!".

---

onFocus, onBlur and onChange

The onFocus, onBlur and onChange events are often used in combination with validation of form fields.

Below is an example of how to use the onChange event. The checkEmail() function will be called whenever the user changes the content of the field:

```
<input type="text" size="30"
id="email" onchange="checkEmail()">;
```

---

onSubmit

The onSubmit event is used to validate ALL form fields before submitting it.

Below is an example of how to use the onSubmit event. The checkForm() function will be called when the user clicks the submit button in the form. If the field values are not accepted, the submit should be cancelled. The function checkForm() returns either true or false. If it returns true the form will be submitted, otherwise the submit will be cancelled:

```
<form method="post" action="xxx.htm"
```

```
onsubmit="return checkForm()">
```

---

onMouseOver and onMouseOut

onMouseOver and onMouseOut are often used to create "animated" buttons.

Below is an example of an onMouseOver event. An alert box appears when an onMouseOver event is detected:

```
<a href="http://www.w3schools.com"
onmouseover="alert('An onMouseOver event');return false">
<img src="sch.gif" width="100" height="30">
</a>
```

JavaScript Special Characters

In JavaScript you can add special characters to a text string by using the backslash sign.

Insert Special Characters

The backslash (\) is used to insert apostrophes, new lines, quotes, and other special characters into a text string.

Look at the following JavaScript code:

```
var txt="We are the so-called "Vikings" from the north."
document.write(txt)
```

In JavaScript, a string is started and stopped with either single or double quotes. This means that the string above will be chopped to: We are the so-called

To solve this problem, you must place a backslash (\) before each double quote in "Viking". This turns each double quote into a string literal:

```
var txt="We are the so-called \"Vikings\" from the north."
document.write(txt)
```

JavasScript will now output the proper text string: We are the so-called "Vikings" from the north.

Here is another example:

```
document.write ("You \& me are singing!")
```

The example above will produce the following output:

```
You & me are singing!
```

The table below lists other special characters that can be added to a text string with the backslash sign:

Code	Outputs
\'	single quote

\"	double quote
\&	ampersand
\\	backslash
\n	new line
\r	carriage return
\t	tab
\b	backspace
\f	form feed

JavaScript Guidelines

Some other important things to know when scripting with JavaScript.

JavaScript is Case Sensitive

A function named "myfunction" is not the same as "myFunction" and a variable named "myVar" is not the same as "myvar".

JavaScript is case sensitive - therefore watch your capitalization closely when you create or call variables, objects and functions.

White Space

JavaScript ignores extra spaces. You can add white space to your script to make it more readable. The following lines are equivalent:

```
name="Hege"
name = "Hege"
```

Break up a Code Line

You can break up a code line **within a text string** with a backslash. The example below will be displayed properly:

```
document.write("Hello \
World!")
```

However, you cannot break up a code line like this:

```
document.write \
("Hello World!")
```

Comments

You can add comments to your script by using two slashes //:

```
//this is a comment
document.write("Hello World!")
```

or by using /* and */ (this creates a multi-line comment block):

```
/* This is a comment
block. It contains
several lines */
document.write("Hello World!")
```

JavaScript Objects Introduction

JavaScript is an Object Oriented Programming (OOP) language.

An OOP language allows you to define your own objects and make your own variable types.

Object Oriented Programming

JavaScript is an Object Oriented Programming (OOP) language. An OOP language allows you to define your own objects and make your own variable types.

However, creating your own objects will be explained later, in the Advanced JavaScript section. We will start by looking at the built-in JavaScript objects, and how they are used. The next pages will explain each built-in JavaScript object in detail.

Note that an object is just a special kind of data. An object has properties and methods.

Properties

Properties are the values associated with an object.

In the following example we are using the length property of the String object to return the number of characters in a string:

```
<script type="text/javascript">
var txt="Hello World!"
document.write(txt.length)
</script>
```

The output of the code above will be:

```
12
```

Methods

Methods are the actions that can be performed on objects.

In the following example we are using the toUpperCase() method of the String object to display a text in uppercase letters:

```
<script type="text/javascript">
var str="Hello world!"
```

```
document.write(str.toUpperCase())
</script>
```

The output of the code above will be:

```
HELLO WORLD!
```

JavaScript String Object

The String object is used to manipulate a stored piece of text.

String object

The String object is used to manipulate a stored piece of text.

Examples of use:

The following example uses the length property of the String object to find the length of a string:

```
var txt="Hello world!"
document.write(txt.length)
```

The code above will result in the following output:

```
12
```

The following example uses the toUpperCase() method of the String object to convert a string to uppercase letters:

```
var txt="Hello world!"
document.write(txt.toUpperCase())
```

The code above will result in the following output:

```
HELLO WORLD!
```

Complete String Object Reference

For a complete reference of all the properties and methods that can be used with the String object, go to our complete String object reference.

The reference contains a brief description and examples of use for each property and method!

JavaScript Date Object

The Date object is used to work with dates and times.

Defining Dates

The Date object is used to work with dates and times.

We define a Date object with the new keyword. The following code line defines a Date object called myDate:

```
var myDate=new Date()
```

Note: The Date object will automatically hold the current date and time as its initial value!

Manipulate Dates

We can easily manipulate the date by using the methods available for the Date object.

In the example below we set a Date object to a specific date (14th January 2010):

```
var myDate=new Date()
myDate.setFullYear(2010,0,14)
```

And in the following example we set a Date object to be 5 days into the future:

```
var myDate=new Date()
myDate.setDate(myDate.getDate()+5)
```

Note: If adding five days to a date shifts the month or year, the changes are handled automatically by the Date object itself!

Comparing Dates

The Date object is also used to compare two dates.

The following example compares today's date with the 14th January 2010:

```
var myDate=new Date()
myDate.setFullYear(2010,0,14)
var today = new Date()
if (myDate>today)
  alert("Today is before 14th January 2010")
else
  alert("Today is after 14th January 2010")
```

Complete Date Object Reference

For a complete reference of all the properties and methods that can be used with the Date object, go to our complete Date object reference.

The reference contains a brief description and examples of use for each property and method!

JavaScript Array Object

The Array object is used to store a set of values in a single variable name.

Defining Arrays

The Array object is used to store a set of values in a single variable name.

We define an Array object with the new keyword. The following code line defines an Array object called myArray:

```
var myArray=new Array()
```

There are two ways of adding values to an array (you can add as many values as you need to define as many variables you require).

1:

```
var mycars=new Array()
mycars[0]="Saab"
mycars[1]="Volvo"
mycars[2]="BMW"
```

You could also pass an integer argument to control the array's size:

```
var mycars=new Array(3)
mycars[0]="Saab"
mycars[1]="Volvo"
mycars[2]="BMW"
```

2:

```
var mycars=new Array("Saab","Volvo","BMW")
```

Note: If you specify numbers or true/false values inside the array then the type of variables will be numeric or Boolean instead of string.

Accessing Arrays

You can refer to a particular element in an array by referring to the name of the array and the index number. The index number starts at 0.

The following code line:

```
document.write(mycars[0])
```

will result in the following output:

```
Saab
```

Modify Values in Existing Arrays

To modify a value in an existing array, just add a new value to the array with a specified index number:

```
mycars[0]="Opel"
```

Now, the following code line:

```
document.write(mycars[0])
```

will result in the following output:

```
Opel
```

Complete Array Object Reference

For a complete reference of all the properties and methods that can be used with the Array object, go to our complete Array object reference.

The reference contains a brief description and examples of use for each property and method!

JavaScript Boolean Object

The Boolean object is used to convert a non-Boolean value to a Boolean value (true or false).

Boolean Object

The Boolean object is an object wrapper for a Boolean value.

The Boolean object is used to convert a non-Boolean value to a Boolean value (true or false).

We define a Boolean object with the new keyword. The following code line defines a Boolean object called myBoolean:

```
var myBoolean=new Boolean()
```

Note: If the Boolean object has no initial value or if it is 0, -0, null, "", false, undefined, or NaN, the object is set to false. Otherwise it is true (even with the string "false")!

All the following lines of code create Boolean objects with an initial value of false:

```
var myBoolean=new Boolean()
var myBoolean=new Boolean(0)
var myBoolean=new Boolean(null)
var myBoolean=new Boolean("")
var myBoolean=new Boolean(false)
var myBoolean=new Boolean(NaN)
```

And all the following lines of code create Boolean objects with an initial value of true:

```
var myBoolean=new Boolean(true)
var myBoolean=new Boolean("true")
var myBoolean=new Boolean("false")
var myBoolean=new Boolean("Richard")
```

Complete Boolean Object Reference

For a complete reference of all the properties and methods that can be used with the Boolean object, go to our complete Boolean object reference.

The reference contains a brief description and examples of use for each property and method!

JavaScript Math Object

The Math object allows you to perform common mathematical tasks.

Math Object

The Math object allows you to perform common mathematical tasks.

The Math object includes several mathematical values and functions. You do not need to define the Math object before using it.

Mathematical Values

JavaScript provides eight mathematical values (constants) that can be accessed from the Math object. These are: E, PI, square root of 2, square root of 1/2, natural log of 2, natural log of 10, base-2 log of E, and base-10 log of E.

You may reference these values from your JavaScript like this:

```
Math.E
Math.PI
Math.SQRT2
Math.SQRT1_2
Math.LN2
Math.LN10
Math.LOG2E
Math.LOG10E
```

Mathematical Methods

In addition to the mathematical values that can be accessed from the Math object there are also several functions (methods) available.

Examples of functions (methods):

The following example uses the round() method of the Math object to round a number to the nearest integer:

```
document.write(Math.round(4.7))
```

The code above will result in the following output:

```
5
```

The following example uses the random() method of the Math object to return a random number between 0 and 1:

```
document.write(Math.random())
```

The code above can result in the following output:

```
0.8669997601016495
```

The following example uses the floor() and random() methods of the Math object to return a random number between 0 and 10:

```
document.write(Math.floor(Math.random()*11))
```

The code above can result in the following output:

```
3
```

Complete Math Object Reference

For a complete reference of all the properties and methods that can be used with the Math object, go to our complete Math object reference.

The reference contains a brief description and examples of use for each property and method!

The Navigator Object

The JavaScript Navigator object contains all information about the visitor's browser. We are going to look at two properties of the Navigator object:

- appName - holds the name of the browser
- appVersion - holds, among other things, the version of the browser

Example

```
<html>
<body>
<script>
with(navigator)
{
  document.writeln("Browser Code Name:"+appCodeName+"<br>")
  document.writeln("Browser Name:"+appName+"<br>")
  document.writeln("Browser Version:"+appVersion+"<br>")
  document.writeln("Language:"+language+"<br>")
  document.writeln("Operating Platform:"+platform+"<br>")
  document.writeln("User Agent:"+userAgent)
}
</script>
</body>
</html>
```

```
<html>
<body>
<script>
with(navigator)
{
  document.writeln("Browser Code Name:"+appCodeName+"<br>")
```

```
  document.writeln("Browser Name:"+appName+"<br>")
  document.writeln("Browser Version:"+appVersion+"<br>")
  document.writeln("Language:"+language+"<br>")
  document.writeln("Operating Platform:"+platform+"<br>")
  document.writeln("User Agent:"+userAgent)
}
</script>
</body>
</html>
```

The variable browser in the example above holds the name of the browser, i.e. "Netscape" or "Microsoft Internet Explorer".

The appVersion property in the example above returns a string that contains much more information than just the version number, but for now we are only interested in the version number. To pull the version number out of the string we are using a function called parseFloat(), which pulls the first thing that looks like a decimal number out of a string and returns it.

IMPORTANT! The version number is WRONG in IE 5.0 or later! Microsoft start the appVersion string with the numbers 4.0. in IE 5.0 and IE 6.0!!! Why did they do that??? However, JavaScript is the same in IE6, IE5 and IE4, so for most scripts it is ok.

Example

The script below displays a different alert, depending on the visitor's browser:

```
<html>
<head>
<script type="text/javascript">
function detectBrowser()

{

var browser=navigator.appName
var b_version=navigator.appVersion
var version=parseFloat(b_version)
if ((browser=="Netscape"||browser=="Microsoft Internet Explorer")
&& (version>=4))

  {alert("Your browser is good enough!")}
else
  {alert("It's time to upgrade your browser!")}

}
</script>
</head>
```

```
<body onload="detectBrowser()">
</body>
</html>
```

JavaScript Timing Events

With JavaScript, it is possible to execute some code NOT immediately after a function is called, but after a specified time interval. This is called timing events.

JavaScript Timing Events

With JavaScript, it is possible to execute some code NOT immediately after a function is called, but after a specified time interval. This is called timing events.

It's very easy to time events in JavaScript. The two key methods that are used are:

- setTimeout() - executes a code some time in the future
- clearTimeout() - cancels the setTimeout()

Note: The setTimeout() and clearTimeout() are both methods of the HTML DOM Window object.

---

setTimeout()

Syntax

```
var t=setTimeout("javascript statement",milliseconds)
```

The setTimeout() method returns a value - In the statement above, the value is stored in a variable called t. If you want to cancel this setTimeout(), you can refer to it using the variable name.

The first parameter of setTimeout() is a string that contains a JavaScript statement. This statement could be a statement like "alert('5 seconds!')" or a call to a function, like "alertMsg()".

The second parameter indicates how many milliseconds from now you want to execute the first parameter.

Note: There are 1000 milliseconds in one second.

clearTimeout()

Syntax

```
clearTimeout(setTimeout_variable)
```

Example

When the button is clicked in the example below, an alert box will be displayed after 5 seconds.

```
<html>
<head>
<script>
```

```
var c=setTimeout("alert('Press the OK button to continue')",5000);
function  clr()
{
  clearTimeout(c);
  alert("The setTimeout() method was cancelled");
}
</script>
</head>
<body>
<form>
<input type="button" value="OK" onclick="clr()">
</form>
</html>
```

JavaScript Summary

This tutorial has taught you how to add JavaScript to your HTML pages, to make your web site more dynamic and interactive.

You have learned how to create responses to events, validate forms and how to make different scripts run in response to different scenarios.

You have also learned how to create and use objects, and how to use JavaScript's built-in objects.

HTML Styles - CSS

CSS stands for Cascading Style Sheets.

CSS saves a lot of work. It can control the layout of multiple web pages all at once.

CSS = Styles and Colors

What is CSS?

Cascading Style Sheets (CSS) is used to format the layout of a webpage.

With CSS, you can control the color, font, the size of text, the spacing between elements, how elements are positioned and laid out, what background images or background colors are to be used, different displays for different devices and screen sizes, and much more!

Tip: The word **cascading** means that a style applied to a parent element will also apply to all children elements within the parent. So, if you set the color of the body text to "blue", all headings, paragraphs, and other text elements within the body will also get the same color (unless you specify something else)!

Using CSS

CSS can be added to HTML documents in 3 ways:

- **Inline** - by using the `style` attribute inside HTML elements
- **Internal** - by using a `<style>` element in the `<head>` section
- **External** - by using a `<link>` element to link to an external CSS file

The most common way to add CSS, is to keep the styles in external CSS files. However, in this tutorial we will use inline and internal styles, because this is easier to demonstrate, and easier for you to try it yourself.

Inline CSS

An inline CSS is used to apply a unique style to a single HTML element.

An inline CSS uses the `style` attribute of an HTML element.

The following example sets the text color of the `<h1>` element to blue, and the text color of the `<p>` element to red:

Example

```
<h1 style="color:blue;">A Blue Heading</h1>

<p style="color:red;">A red paragraph.</p>
```

Try it Yourself »

Internal CSS

An internal CSS is used to define a style for a single HTML page.

An internal CSS is defined in the `<head>` section of an HTML page, within a `<style>` element.

The following example sets the text color of ALL the `<h1>` elements (on that page) to blue, and the text color of ALL the `<p>` elements to red. In addition, the page will be displayed with a "powderblue" background color:

Example

```
<!DOCTYPE html>
<html>
<head>
<style>
body {background-color: powderblue;}
h1   {color: blue;}
p    {color: red;}
</style>
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

External CSS

An external style sheet is used to define the style for many HTML pages.

To use an external style sheet, add a link to it in the `<head>` section of each HTML page:

Example

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="styles.css">
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

The external style sheet can be written in any text editor. The file must not contain any HTML code, and must be saved with a .css extension.

Here is what the "styles.css" file looks like:

"styles.css":

```
body {
  background-color: powderblue;
}
h1 {
  color: blue;
}
p {
  color: red;
}
```

Tip: With an external style sheet, you can change the look of an entire web site, by changing one file!

CSS Colors, Fonts and Sizes

Here, we will demonstrate some commonly used CSS properties. You will learn more about them later.

The CSS `color` property defines the text color to be used.

The CSS `font-family` property defines the font to be used.

The CSS `font-size` property defines the text size to be used.

Example

Use of CSS color, font-family and font-size properties:

```
<!DOCTYPE html>
<html>
```

```
<head>
<style>
h1 {
  color: blue;
  font-family: verdana;
  font-size: 300%;
}
p {
  color: red;
  font-family: courier;
  font-size: 160%;
}
</style>
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

CSS Border

The CSS `border` property defines a border around an HTML element.

Tip: You can define a border for nearly all HTML elements.

Example

Use of CSS border property:

```
p {
  border: 2px solid powderblue;
}
```

CSS Padding

The CSS `padding` property defines a padding (space) between the text and the border.

Example

Use of CSS border and padding properties:

```
p {
  border: 2px solid powderblue;
  padding: 30px;
}
```

CSS Margin

The CSS `margin` property defines a margin (space) outside the border.

Example

Use of CSS border and margin properties:

```
p {
  border: 2px solid powderblue;
  margin: 50px;
}
```

Link to External CSS

External style sheets can be referenced with a full URL or with a path relative to the current web page.

Example

This example uses a full URL to link to a style sheet:

```
<link rel="stylesheet" href="https://www.w3schools.com/html/styles.css">
```

Example

This example links to a style sheet located in the html folder on the current web site:

```
<link rel="stylesheet" href="/html/styles.css">
```

Example

This example links to a style sheet located in the same folder as the current page:

```
<link rel="stylesheet" href="styles.css">
```

You can read more about file paths in the chapter HTML File Paths.

Chapter Summary

- Use the HTML `style` attribute for inline styling
- Use the HTML `<style>` element to define internal CSS
- Use the HTML `<link>` element to refer to an external CSS file
- Use the HTML `<head>` element to store <style> and <link> elements
- Use the CSS `color` property for text colors
- Use the CSS `font-family` property for text fonts
- Use the CSS `font-size` property for text sizes
- Use the CSS `border` property for borders
- Use the CSS `padding` property for space inside the border
- Use the CSS `margin` property for space outside the border

Tip: You can learn much more about CSS in our CSS Tutorial.

HTML Exercises

Exercise:

Use CSS to set the background color of the document (body) to yellow.

```
<!DOCTYPE html>
<html>
<head>
<style>
[illegible] [illegible]
  [illegible]:yellow;
[illegible]
</style>
</head>
<body>

<h1>My Home Page</h1>

</body>
</html>
```

Submit Answer »

Start the Exercise

HTML Style Tags

Tag	Description
<style>	Defines style information for an HTML document
<link>	Defines a link between a document and an external resource

For a complete list of all available HTML tags, visit our HTML Tag Reference.

REFERENCES.

https://www.tutorialspoint.com/jsp/index.htm
https://www.javatpoint.com/html-tutorial
https://www.javatpoint.com/javascript-tutorial
https://www.javatpoint.com/css-tutorial
https://www.javatpoint.com/jsp-tutorial
https://www.javatpoint.com/asp-net-tutorial